

Multi-Target Tracking Training Workshop

Module 1 · Introduction & Tracking Fundamentals

2026-09-17

Table of contents

1	Introduction & Tracking Fundamentals	3
	Learning objectives	3
1.1	What target tracking is	3
	The recursive tracking loop	4
1.2	Where tracking sits: from detection to C2	4
	From detection to command and control	4
	From Wiener filtering to state-space estimation	4
1.3	Target motion models: CV, CA, CT	5
	Three motion models, one start state	6
1.4	Process noise: the model-mismatch budget	6
	Process noise as an uncertainty budget	6
1.5	Radar measurements: range, azimuth, elevation, range-rate	6
	Radar measurement geometry	7
1.6	Angle-only measurements: bearings	7
	Bearings without range	8
1.7	Linear vs nonlinear measurement models	8
	Where nonlinearity enters	9
1.8	Coordinate systems and frames	9
	One target, three frames	10
1.9	The tracking problem statement	10
	The discrete-time recursion	11
	Key takeaways	11
	Exercise 1: Explore the Motion Models	11

Module 1: Introduction & Tracking Fundamentals

Student Guide

Learning objectives

This module sets the vocabulary and the state-space framing that every later module reuses. By the end of it you will be able to describe the recursive tracking loop (predict, associate, update) and where tracking sits in a sensor system; write the three motion models the course carries forward (constant velocity, constant acceleration, and coordinated turn) and explain what real targets each one suits; read process noise as the budget for everything the motion model leaves out; write the radar and angle-only measurement models and recognize where they become nonlinear; name the coordinate frames a tracker moves between (sensor-relative, local-level, and Earth-centered) and see why frame choice can make a problem linear or not; and state the discrete-time tracking problem in the notation used through the rest of the workshop. Everything here reappears, in more depth, in Modules 2 through 13. This is the shared foundation.

1.1 What target tracking is

Target tracking is the recursive estimation of a moving object's state, usually its position and velocity, from a stream of noisy sensor measurements. The word recursive is the key: a tracker does not re-solve the whole problem every time a measurement arrives. Instead it carries a running estimate and updates it one step at a time. Each cycle has three parts. It predicts the state forward to the time of the next measurement using a motion model; it associates an incoming measurement with the predicted track (deciding which return, if any, belongs to this target); and it updates the estimate by blending the prediction with the associated measurement. The updated estimate then becomes the starting point for the next prediction, and the loop repeats. Holding onto a running estimate plus its uncertainty, rather than a pile of raw detections, is what lets a tracker work in real time and ride through missed detections and clutter.

The recursive tracking loop

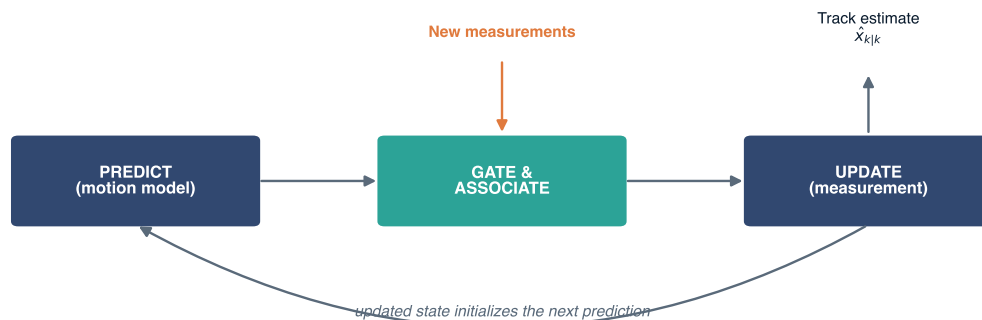


Figure 1.1: The recursive tracking loop: predict ahead with a motion model, gate and associate an incoming measurement, update the estimate, and feed it back as the next prediction.

1.2 Where tracking sits: from detection to C2

Tracking is one stage in a longer sensor-to-decision chain, and it helps to see the neighbors. Upstream, detection (the signal-processing stage) turns raw sensor returns into discrete measurements, often called plots or contacts: a range and angle here, a bearing there, usually with false alarms mixed in. Tracking takes that stream of plots and produces tracks: for each target, an estimate of its state and the uncertainty in that estimate, maintained over time. Fusion combines tracks (or measurements) from several sensors into a single, consistent picture, the subject of Day 3. Finally, command and control (C2) uses that picture to make decisions. This workshop lives in the tracking and fusion stages: Days 1 and 2 build single-sensor tracking from the filter up through data association, and Day 3 fuses multiple sensors. Detection and C2 are the boundaries we hand off to.

From detection to command and control

From Wiener filtering to state-space estimation

It is worth a moment on where this recursive, model-based style of estimation came from, because it frames the whole course. In the 1940s Norbert Wiener formulated the problem of optimally filtering a signal out of noise, working in the frequency domain and assuming stationary statistics. Wiener's filter was a milestone, but its frequency-domain formulation was hard to apply to the nonstationary, maneuvering targets a tracker faces, and it did not lend itself to running online as data arrives. The breakthrough of the late 1950s and 1960 was to recast estimation in the state-space and time-domain form: describe the target with a state vector and a dynamics model, describe the sensor with a measurement model, and estimate the state recursively as each measurement comes in.

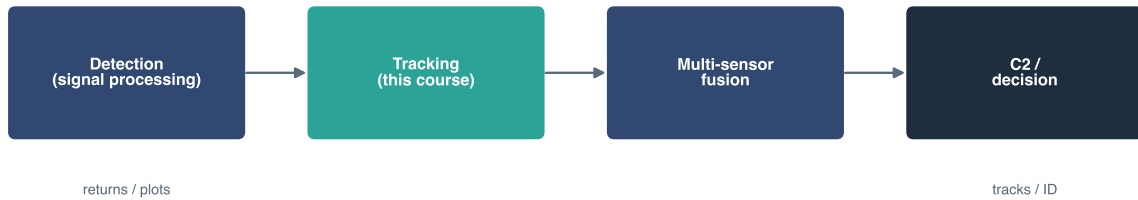


Figure 1.2: Where tracking sits in the sensor-to-decision chain: detection produces plots, tracking produces tracks, fusion merges them across sensors, and command and control acts on the result.

That reformulation, which Module 2 develops as the Kalman filter, is what turned estimation into something a computer can do step by step in real time, and it is the backbone of every tracker in this workshop. We defer the filter itself, and the biography of its originator, to Module 2.

1.3 Target motion models: CV, CA, CT

Between measurements, the tracker propagates the state with a motion model, a statement of how the target is assumed to move. Three models cover most of what this course needs. The constant-velocity (CV) model assumes the target moves in a straight line at fixed speed; its state is position and velocity, and it is the default for benign, non-maneuvering targets. The constant-acceleration (CA) model adds an acceleration term to the state, capturing targets that are speeding up, slowing down, or gently curving. The coordinated-turn (CT) model assumes constant speed and a constant turn rate, tracing a circular arc, which is a good description of an aircraft in a banked turn. For a linear discrete-time model the state propagates as

$$\mathbf{x}_k = \mathbf{F} \mathbf{x}_{k-1} + \mathbf{w}_{k-1} \quad (1.1)$$

where $\mathbf{x}_k$ is the state at time step k , $\mathbf{F}$ is the state-transition matrix set by the motion model and the time step, and $\mathbf{w}_{k-1}$ is the process noise (next section). For a two-dimensional CV target the state is $\mathbf{x} = [p_x, v_x, p_y, v_y]^\top$ (position and velocity in each axis), and over a step of length T the transition applies $p \leftarrow p + vT$ to each axis. CA extends the state with acceleration; CT makes $\mathbf{F}$ depend on the turn rate ω , which makes the coordinated turn a nonlinear model. The figure propagates all three from the same start state so the difference is visible at a glance.

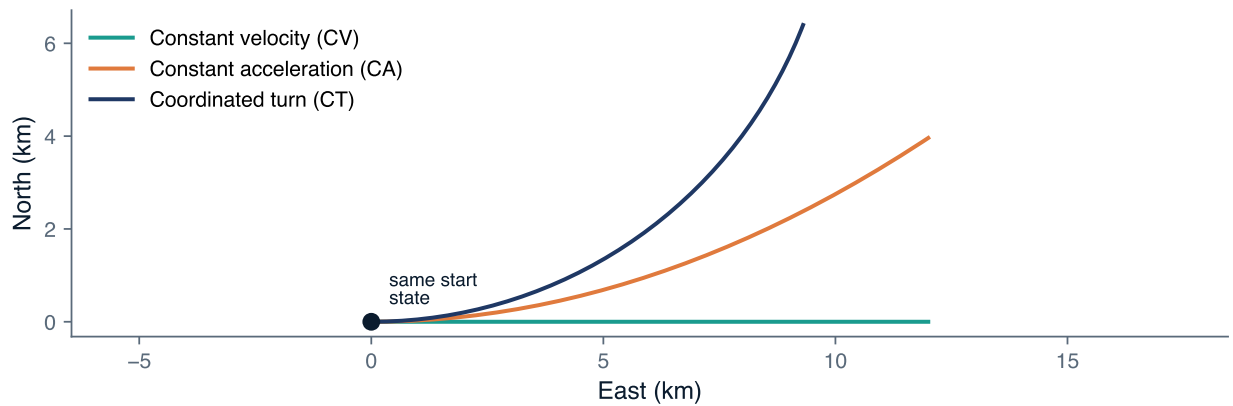


Figure 1.3: The CV, CA, and CT motion models propagated from the same start state: a straight line, a parabola, and a turning arc.

Three motion models, one start state

1.4 Process noise: the model-mismatch budget

No motion model is exactly right. A CV filter meets a target that turns; a CA filter meets one that holds course. Process noise is how the tracker budgets for that mismatch. In the dynamics of Equation 1.1 the term $\mathbf{w}_{k-1}$ is modeled as zero-mean Gaussian noise with covariance $\mathbf{Q}$,

$$\mathbf{w}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}), \quad (1.2)$$

and $\mathbf{Q}$ encodes how much the true motion is expected to depart from the assumed model over one step (physically, the size of the unmodeled accelerations). The choice of $\mathbf{Q}$ is a design lever with a direct trade-off. A small $\mathbf{Q}$ tells the filter to trust its motion model: the track is smooth and precise on a benign target but lags and can diverge when the target maneuvers. A large $\mathbf{Q}$ tells the filter to expect surprises: the track reacts quickly to maneuvers but is noisier and less precise in steady flight. The figure shows the idea as an ensemble: with the same nominal constant-velocity path but unknown accelerations, the possible trajectories fan out over time, and the width of that fan is exactly what $\mathbf{Q}$ sizes. We return to tuning $\mathbf{Q}$ against measurement noise in Module 2, and to adapting it for maneuvers in Module 5.

Process noise as an uncertainty budget

1.5 Radar measurements: range, azimuth, elevation, range-rate

A tracker reasons about a target in Cartesian state (position and velocity), but a sensor reports what it can actually measure, and a radar measures in its own polar frame. A monostatic radar typically reports range r (from the round-trip time of flight), azimuth θ and elevation ϕ (the pointing angles

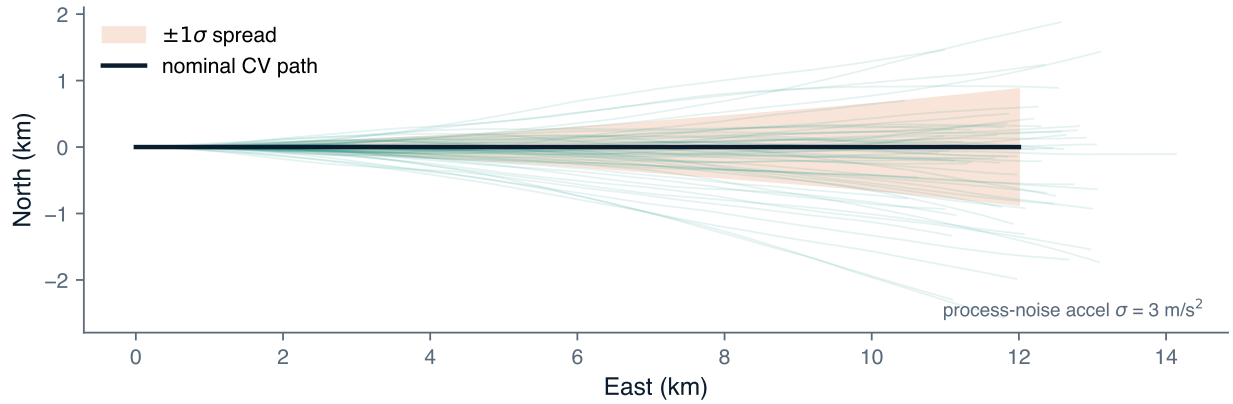


Figure 1.4: Process noise as an uncertainty budget: an ensemble of tracks with the same nominal CV path but unknown accelerations, spreading over time.

of the return), and often range-rate $\dot{r}$ (the radial velocity, from the Doppler shift). If the target position relative to the sensor is (p_x, p_y, p_z) , these measurements are

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2}, \quad \theta = \text{atan2}(p_y, p_x), \quad \phi = \arcsin\left(\frac{p_z}{r}\right), \quad (1.3)$$

and the range-rate is the projection of the velocity onto the line of sight,

$$\dot{r} = \frac{\mathbf{p} \cdot \mathbf{v}}{\|\mathbf{p}\|}, \quad (1.4)$$

with $\mathbf{p}$ and $\mathbf{v}$ the relative position and velocity vectors. Range-rate is valuable because it senses velocity directly rather than inferring it from successive positions; Module 7 puts it to work. Notice that every one of these expressions is a nonlinear function of the Cartesian state, a fact the next two sections build on.

Radar measurement geometry

1.6 Angle-only measurements: bearings

Not every sensor measures range. Passive sensors, electronic-support-measures receivers, infrared trackers, and passive sonar, measure angle only: a bearing (and possibly elevation) to the target, with no direct range. A single bearing defines a line of sight, and the target may be anywhere along it, so from one fixed sensor the range is fundamentally unobservable. Range becomes observable only when the geometry changes, typically when the observer maneuvers so that successive bearings are taken from different positions and can be triangulated. The bearing measurement is

$$\beta = \text{atan2}(p_y, p_x), \quad (1.5)$$

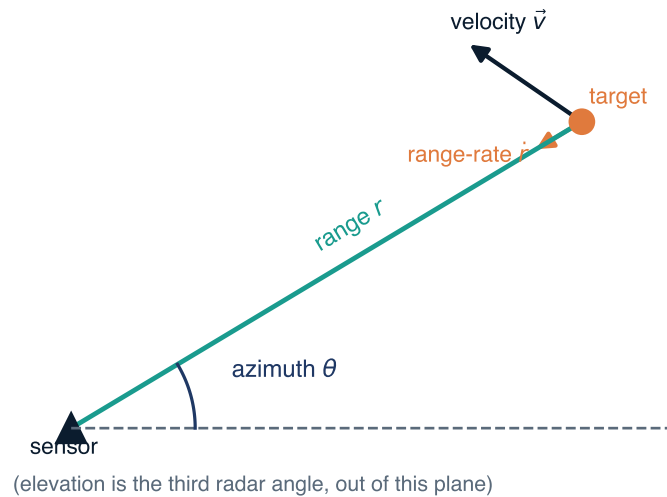


Figure 1.5: Radar measurement geometry: range along the line of sight, azimuth from a reference direction, and range-rate as the radial component of the target velocity.

a nonlinear function of position with no range information at all. Angle-only tracking is important enough, and awkward enough, to get its own treatment in Module 4; here it establishes the second major measurement type alongside radar.

Bearings without range

1.7 Linear vs nonlinear measurement models

The measurement model links the state to what the sensor reports. In general,

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \quad (1.6)$$

where $\mathbf{z}_k$ is the measurement, $h(\cdot)$ is the measurement function, and $\mathbf{v}_k$ is measurement noise with covariance $\mathbf{R}$. When the sensor measures a quantity that is a linear function of the state, for example a position measured directly in Cartesian coordinates, the model collapses to $\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$ with a constant matrix $\mathbf{H}$, and the whole estimation problem stays linear (the Kalman filter of Module 2 is then exact and optimal). The radar and angle-only measurements of the last two sections are not linear: range, azimuth, elevation, and bearing are all nonlinear functions of the Cartesian state. Nonlinearity matters because it breaks the neat Gaussian bookkeeping the linear filter relies on. A Gaussian uncertainty in the state, pushed through a nonlinear measurement, no longer maps to a Gaussian, as the figure shows: a compact ellipse of state uncertainty becomes a curved, banana-shaped cloud in measurement space. Handling that distortion is exactly the job of the nonlinear filters (EKF, UKF, particle filter) in Module 3.

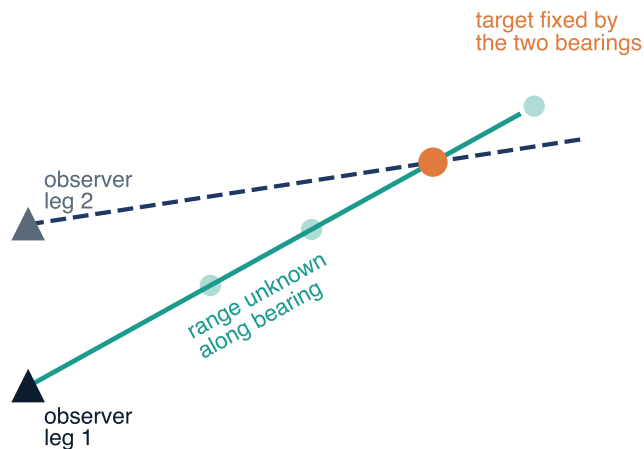


Figure 1.6: Angle-only geometry: a single bearing leaves range unknown along the line of sight; a second bearing from a moved observer fixes the target.

Where nonlinearity enters

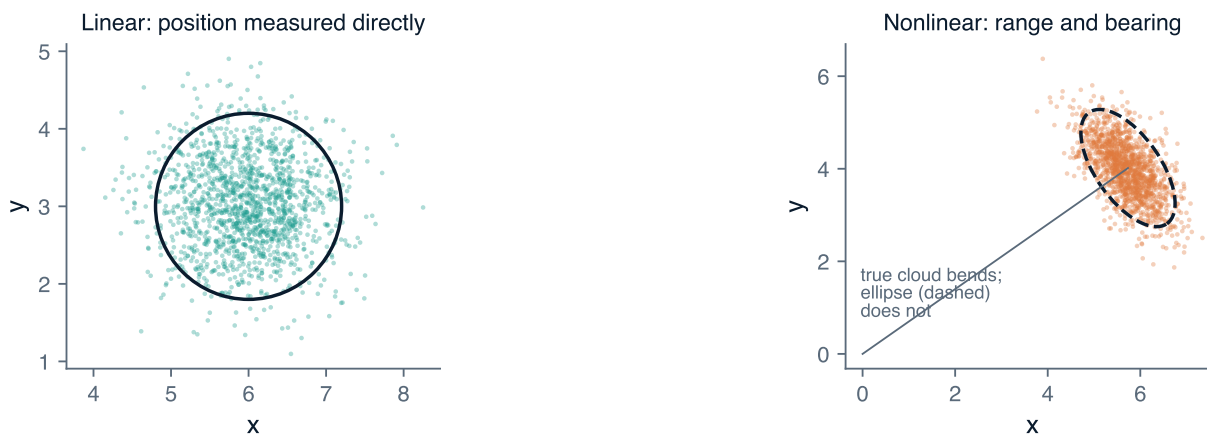


Figure 1.7: A position measurement keeps a Gaussian Gaussian (left); a range-and-bearing measurement bends it into a curved cloud the linearized ellipse does not capture (right).

1.8 Coordinate systems and frames

A tracker constantly moves between coordinate frames, and choosing the right one is half the battle. Measurements are born in the sensor frame, a sensor-relative polar or RUV (range, direction-cosine) frame. Tracking is usually done in a local-level Cartesian frame such as East-North-Up (ENU), where a constant-velocity target really does move in a straight line, so the dynamics are linear and simple. When sensors are far apart, or the curvature of the Earth matters, tracks are carried in

an Earth-centered, Earth-fixed (ECEF) frame, a global Cartesian frame, and converted to each sensor's local frame as needed. The important idea for this module is that frame choice interacts with linearity: the same constant-velocity motion is linear in ENU but its radar measurement is nonlinear in polar, and a poor frame choice can introduce nonlinearity or bias that a better one avoids. Converting measurements between frames cleanly, without introducing bias, is the precondition for combining sensors, which is why Module 9 is devoted to coordinate conversion and why it opens the fusion half of the course.

One target, three frames

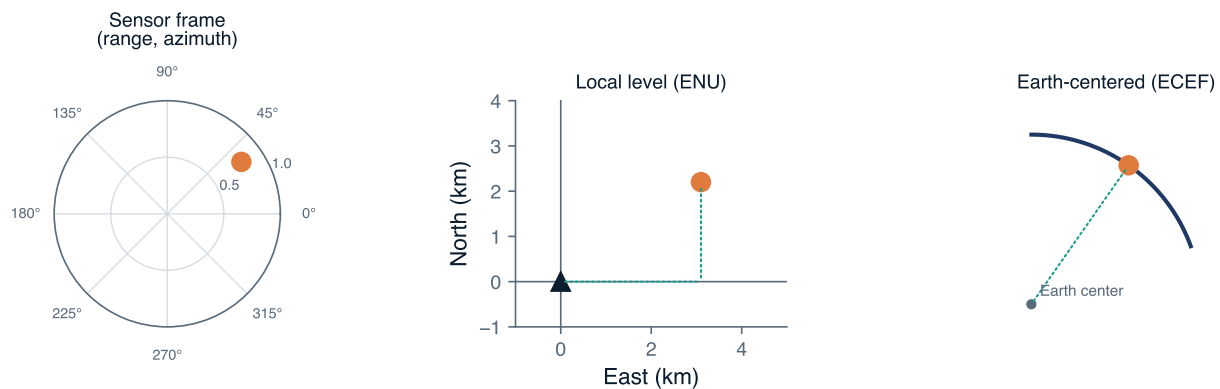


Figure 1.8: The same target expressed in three coordinate frames: the sensor polar frame, a local-level ENU frame, and an Earth-centered frame.

1.9 The tracking problem statement

Putting the pieces together gives the problem statement the rest of the course solves. We describe the target by a state $\mathbf{x}_k$ (at minimum position and velocity), evolving in discrete time under a motion model with process noise, and observed through a measurement model with measurement noise:

$$\mathbf{x}_k = \mathbf{F} \mathbf{x}_{k-1} + \mathbf{w}_{k-1}, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad (1.7)$$

with $\mathbf{w}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ and $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$. The task is to estimate the state $\hat{\mathbf{x}}_k$ and the covariance $\mathbf{P}_k$ that quantifies its uncertainty, recursively, as each measurement arrives. The estimator does this in the two half-steps of the tracking loop: a predict step pushes $\hat{\mathbf{x}}_{k-1|k-1}$ and $\mathbf{P}_{k-1|k-1}$ forward through the dynamics to $\hat{\mathbf{x}}_{k|k-1}$, $\mathbf{P}_{k|k-1}$ (uncertainty grows), and an update step folds in the measurement $\mathbf{z}_k$ to produce $\hat{\mathbf{x}}_{k|k}$, $\mathbf{P}_{k|k}$ (uncertainty shrinks). The subscript $k|k-1$ reads as “the estimate at time k given measurements through $k-1$.” This notation, the symbols $\mathbf{x}, \mathbf{F}, \mathbf{Q}, \mathbf{z}, h, \mathbf{H}, \mathbf{R}, \mathbf{P}$ and the predict-update indexing, is used unchanged from here through Module 13. Module 2 fills in the

actual predict and update equations that make it the Kalman filter.

The discrete-time recursion

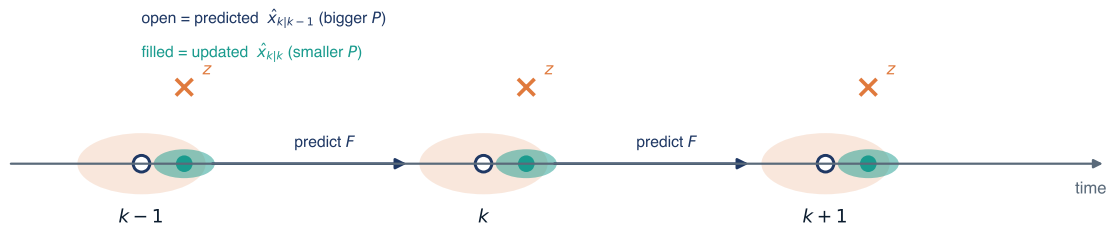


Figure 1.9: The discrete-time recursion in time: each predict step grows the covariance, each measurement update shrinks it.

Key takeaways

Target tracking is recursive estimation: a predict, associate, and update loop that maintains a state and its uncertainty over time rather than re-solving from scratch. Prediction is driven by a motion model (constant velocity, constant acceleration, or coordinated turn) together with process noise $\mathbf{Q}$, which budgets for everything the model leaves out. Measurements come mainly in two forms, radar (range, angles, and range-rate) and angle-only (bearings), and both are nonlinear functions of the Cartesian state, which is what forces the nonlinear filters of Module 3. The frames a tracker works in, sensor-relative, local-level, and Earth-centered, interact with that linearity and must be made consistent before sensors can be fused. All of this is captured in one discrete-time state-space statement whose notation the rest of the workshop uses without change. Module 2 turns the loop into the Kalman filter.

Exercise 1: Explore the Motion Models

Exercise 1: Explore the Motion Models (25 min)

No coding is required. This exercise uses the interactive companion app: you learn by changing its controls and reading the result.

Start the app from the project root, in the course environment:

```
conda activate radar
streamlit run exercises/ex01_motion_models.py
```

It opens in your browser with sliders for speed, heading, turn rate, process-noise level, and duration, a set of ready-made scenario presets, and a live plot of a constant-velocity (CV) path against a coordinated-turn (CT) path from the same start state.

Work through these steps and note what you see:

1. Set Turn rate = 0. The two paths land on top of each other. Explain in one sentence why the coordinated-turn model becomes the constant-velocity model when the turn rate is zero.
2. Set the turn rate to about $3^\circ/\text{s}$ and read the CV-CT separation at 60 s. This is roughly the error a CV tracker would build up on this maneuvering target. Say where that error comes from.
3. Slide the process-noise level from 0 up to about 8 m/s^2 and watch the paths. Describe how they change, and name one real source of that uncertainty (something the motion model leaves out).
4. Load the Benign airliner and Fighter in a hard bank presets. For each, say which motion model (CV or CT) fits better, and why.

Deliverable: short written answers to the four items above (a few sentences each), plus one screenshot of the plot for the fighter-in-a-bank case with the CV-model error highlighted.

Go further (optional, for those who code): open `exercises/ex01_motion_models.py` and `figures/mod01.py` to see the CV, CA, and CT propagation in a few lines of Python, and try changing a model or adding constant acceleration.

Multi-Target Tracking Training Workshop

Module 2 · Linear Filtering: Kalman & Other Linear Filters

2026-09-17

Table of contents

2	Linear Filtering: Kalman & Other Linear Filters	3
	Learning objectives	3
	Where Module 1 left us	3
2.1	Least squares: the batch estimate	3
	Recursive least squares	4
	From batch to recursive	4
2.2	The Kalman filter: the model	4
	The predict step	5
	The update step	5
	Deriving the Kalman gain	6
	The filter tracking a target	6
	The covariance settles	6
2.3	Reading the Kalman gain	6
	Gain vs measurement noise	8
2.4	Constant-gain filters: alpha-beta	8
	Choosing the gains, and alpha-beta-gamma	8
	Alpha-beta vs the Kalman filter	9
2.5	Table look-up and gain-scheduled filters	9
	The gain schedule is precomputable	10
2.6	Comparative performance	10
	Accuracy, cost, and effort	10
	The person behind it	10
	Key takeaways	11
	Exercise 2a: Tune the Filter	12
	Exercise 2b: Match Alpha-Beta to the Kalman Filter	12

Module 2: Linear Filtering: Kalman & Other Linear Filters

Student Guide

Learning objectives

Module 1 set up the discrete-time tracking problem: a state that evolves under a motion model with process noise, observed through a measurement model with measurement noise, to be estimated recursively. This module fills in the actual estimator for the linear case. We start from least-squares estimation, make it recursive, and show that the result is the Kalman filter. We derive the filter's predict and update equations and its gain, read the gain physically as the balance between the process-noise covariance $\mathbf{Q}$ and the measurement-noise covariance $\mathbf{R}$, and then place the cheaper fixed-gain alternatives (the alpha-beta and alpha-beta-gamma filters, and gain-scheduled / table look-up filters) beside it, ending with a head-to-head comparison. This is the analytical core of Day 1; everything on Days 2 and 3 assumes you can run and reason about a linear filter.

Where Module 1 left us

Recall the linear-Gaussian tracking model from Module 1: the state propagates as $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{w}_{k-1}$ with process noise $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, and is observed through $\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$ with measurement noise $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$. The task is to estimate the state $\hat{\mathbf{x}}_k$ and its covariance $\mathbf{P}_k$ recursively, in a predict step that grows the uncertainty and an update step that shrinks it. This module derives exactly what those two steps are when the models are linear.

2.1 Least squares: the batch estimate

Before the recursive machinery, start with the batch problem. Suppose we have collected m measurements that depend linearly on a fixed unknown state, stacked as $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{v}$, where $\mathbf{z}$ is the m -vector of measurements, $\mathbf{H}$ is the $m \times n$ observation matrix, and $\mathbf{v}$ is measurement noise. Ordinary least squares chooses the state that minimizes the sum of squared residuals $J(\mathbf{x}) = \|\mathbf{z} - \mathbf{H}\mathbf{x}\|^2$.

Setting the gradient to zero gives the normal equations $\mathbf{H}^\top \mathbf{H} \hat{\mathbf{x}} = \mathbf{H}^\top \mathbf{z}$, so

$$\hat{\mathbf{x}} = (\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{z}. \quad (2.1)$$

If the measurements have unequal or correlated noise with covariance $\mathbf{R}$, weighted least squares minimizes $(\mathbf{z} - \mathbf{H}\mathbf{x})^\top \mathbf{R}^{-1}(\mathbf{z} - \mathbf{H}\mathbf{x})$ instead, giving

$$\hat{\mathbf{x}} = (\mathbf{H}^\top \mathbf{R}^{-1} \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{z}, \quad (2.2)$$

which down-weights noisy measurements. This batch estimate is optimal but requires all the data at once and an $n \times n$ inversion; a tracker instead needs to update as each measurement arrives.

Recursive least squares

Recursive least squares (RLS) computes the same estimate incrementally. Writing $\mathbf{P}_{k-1} = (\mathbf{H}_{k-1}^\top \mathbf{R}^{-1} \mathbf{H}_{k-1})^{-1}$ for the estimate covariance after $k-1$ measurements, the arrival of one more scalar measurement z_k with row $\mathbf{h}_k^\top$ and variance r_k updates the estimate through a gain:

$$\mathbf{K}_k = \mathbf{P}_{k-1} \mathbf{h}_k (\mathbf{h}_k^\top \mathbf{P}_{k-1} \mathbf{h}_k + r_k)^{-1}, \quad (2.3)$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_{k-1} + \mathbf{K}_k (z_k - \mathbf{h}_k^\top \hat{\mathbf{x}}_{k-1}), \quad \mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{h}_k^\top) \mathbf{P}_{k-1}. \quad (2.4)$$

The term $z_k - \mathbf{h}_k^\top \hat{\mathbf{x}}_{k-1}$ is the innovation, the part of the measurement the current estimate did not predict, and the gain $\mathbf{K}_k$ decides how much of it to believe. This is exactly the Kalman update equations for a state that does not move: the Kalman filter is RLS plus a prediction step that lets the state evolve and lets uncertainty grow between measurements. That is the one idea that turns a batch estimator into a tracker.

From batch to recursive

2.2 The Kalman filter: the model

The Kalman filter is the optimal recursive estimator for the linear-Gaussian model. It represents the state estimate as a Gaussian, tracked by its mean $\hat{\mathbf{x}}$ and covariance $\mathbf{P}$, and updates that Gaussian in two half-steps per cycle. We use the standard conditional notation: $\hat{\mathbf{x}}_{k|k-1}$ and $\mathbf{P}_{k|k-1}$ are the mean and covariance at time k given all measurements through time $k-1$ (the prediction), while $\hat{\mathbf{x}}_{k|k}$ and $\mathbf{P}_{k|k}$ are the same quantities after folding in the measurement at time k (the update). The next two slides derive each half-step.

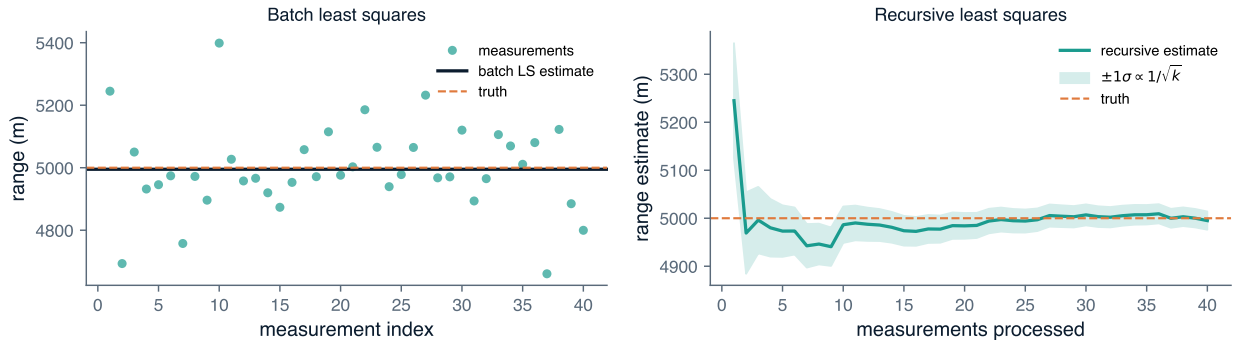


Figure 2.1: Estimating a fixed quantity: the batch least-squares estimate is the mean of all measurements; the recursive estimate reaches the same value incrementally, its uncertainty shrinking like $1/\sqrt{k}$.

The predict step

The predict step pushes the estimate forward to the next measurement time using the dynamics $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{w}_{k-1}$. Because $\mathbf{w}$ has zero mean, the predicted mean is just the transported previous mean,

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F} \hat{\mathbf{x}}_{k-1|k-1}. \quad (2.5)$$

The predicted covariance follows from propagating the estimation error $\tilde{\mathbf{x}}_{k-1} = \mathbf{x}_{k-1} - \hat{\mathbf{x}}_{k-1|k-1}$ through $\mathbf{F}$ and adding the independent process noise: $\mathbf{P}_{k|k-1} = \text{Cov}(\mathbf{F}\tilde{\mathbf{x}}_{k-1} + \mathbf{w}_{k-1})$, which gives

$$\mathbf{P}_{k|k-1} = \mathbf{F} \mathbf{P}_{k-1|k-1} \mathbf{F}^\top + \mathbf{Q}. \quad (2.6)$$

The first term rotates and stretches the old uncertainty through the dynamics; the second term, $\mathbf{Q}$, adds the uncertainty from everything the motion model leaves out. Prediction therefore always makes the estimate less certain, which is the growth you saw in the Module 1 recursion figure.

The update step

The update step folds in the measurement $\mathbf{z}_k$. The innovation is the difference between the measurement and its prediction, $\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}$, and its covariance is

$$\mathbf{S}_k = \mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R}, \quad (2.7)$$

the predicted measurement uncertainty plus the measurement noise. The filter corrects the state by a gain-weighted innovation,

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k, \quad (2.8)$$

and the covariance shrinks to

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_{k|k-1}. \quad (2.9)$$

The only quantity not yet justified is the gain $\mathbf{K}_k$; the next slide shows it is the choice that makes the updated estimate as accurate as possible.

Deriving the Kalman gain

The Kalman gain is not assumed; it is derived as the gain that minimizes the expected squared estimation error, equivalently the trace of the posterior covariance $\text{tr}(\mathbf{P}_{k|k})$. For an arbitrary gain $\mathbf{K}$, the posterior covariance in the numerically robust Joseph form is

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_{k|k-1}(\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top. \quad (2.10)$$

Differentiating the trace with respect to $\mathbf{K}$ and using $\partial \text{tr}(\mathbf{A}\mathbf{B}\mathbf{A}^\top)/\partial \mathbf{A} = 2\mathbf{A}\mathbf{B}$ for symmetric $\mathbf{B}$ gives $-2(\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_{k|k-1}\mathbf{H}^\top + 2\mathbf{K}\mathbf{R} = \mathbf{0}$. Solving for $\mathbf{K}$,

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R})^{-1} = \mathbf{P}_{k|k-1} \mathbf{H}^\top \mathbf{S}_k^{-1}, \quad (2.11)$$

the gain we used above. Substituting this optimal gain back into the Joseph form collapses it to the compact $\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_{k|k-1}$. So the Kalman filter is the linear estimator that, at every step, leaves the smallest possible posterior uncertainty.

The filter tracking a target

The covariance settles

2.3 Reading the Kalman gain

The gain is the filter's entire character in one term. In the scalar case with $\mathbf{H} = 1$ it reduces to $K = P^-/(P^- + R)$, where P^- is the predicted (prior) variance and R the measurement variance. Two limits make the behavior clear. When the sensor is noisy, $R \gg P^-$, the gain approaches zero and the filter barely moves off its prediction: it trusts the model. When the prediction is uncertain, $P^- \gg R$, the gain approaches one and the filter follows the measurement: it trusts the sensor. Because P^- is grown by the process-noise term $\mathbf{Q}$ in the predict step, raising $\mathbf{Q}$ raises the gain and raising $\mathbf{R}$ lowers it. Filter tuning is therefore fundamentally about the ratio $\mathbf{Q}/\mathbf{R}$: too little $\mathbf{Q}$ and

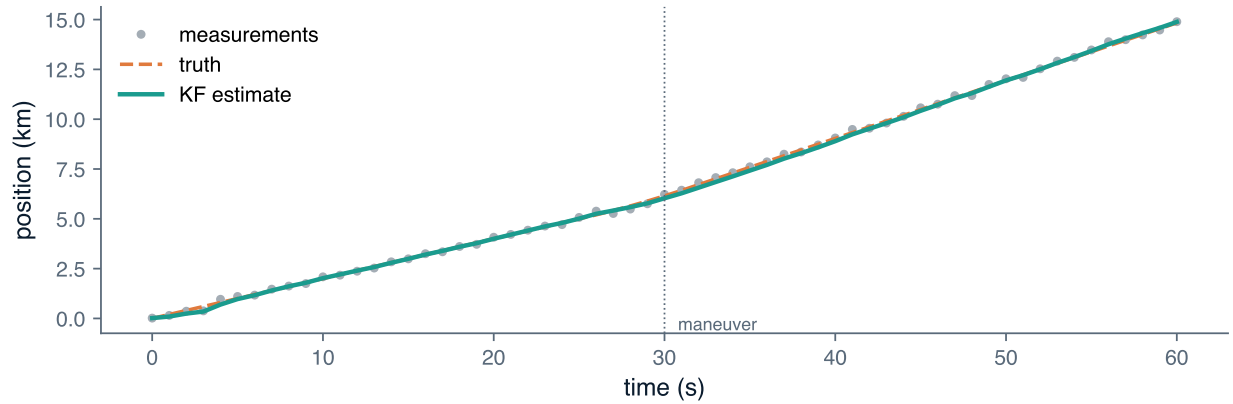


Figure 2.2: A Kalman filter tracking a noisy target that maneuvers at mid-track: the estimate rejects most of the measurement noise and re-converges after the maneuver.

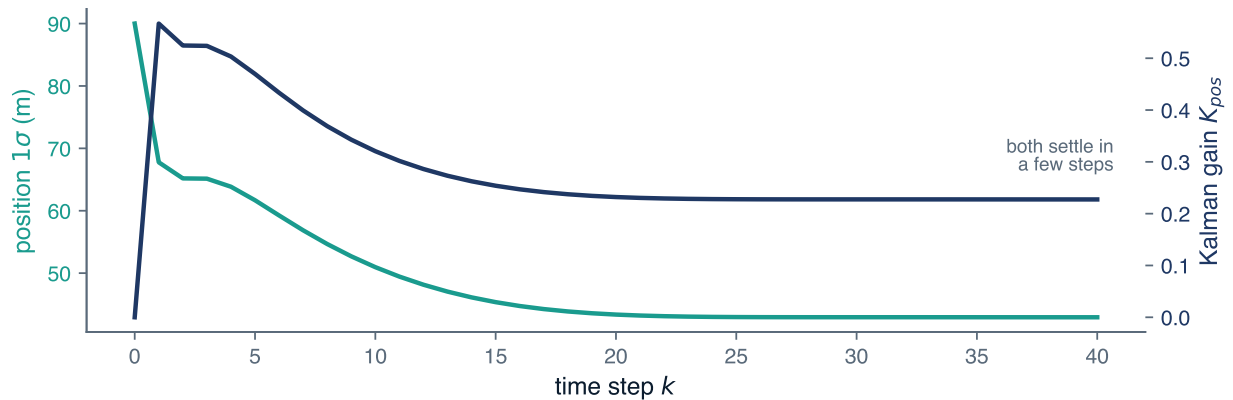


Figure 2.3: For a fixed model, Q , and R , the position uncertainty and the Kalman gain both converge to a steady state within a few steps.

the filter lags maneuvers; too much and it passes sensor noise straight through. You will feel this trade-off directly in Exercise 2a.

Gain vs measurement noise

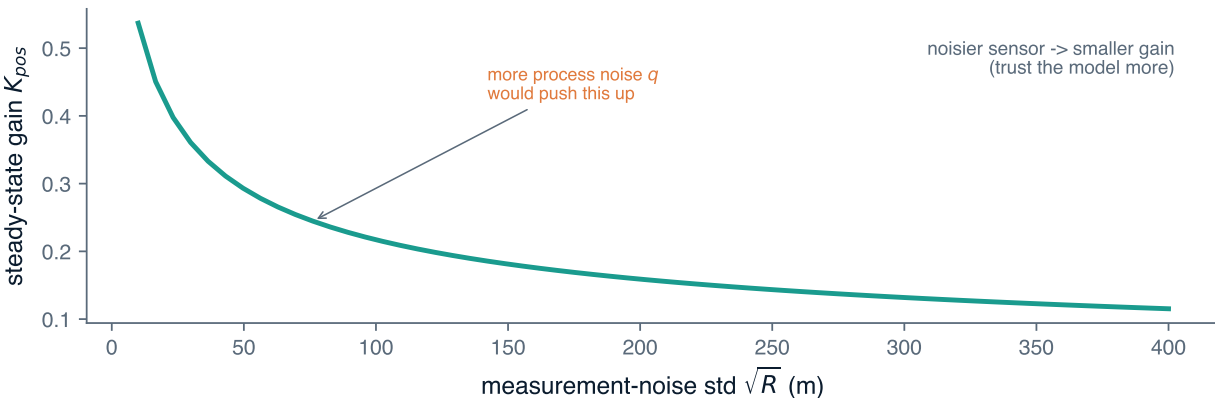


Figure 2.4: Steady-state Kalman gain against measurement-noise level: a noisier sensor drives the gain down, so the filter leans harder on its motion model.

2.4 Constant-gain filters: alpha-beta

The last figure showed that for a constant-velocity target with fixed $\mathbf{Q}$ and $\mathbf{R}$ the Kalman gain converges to a constant. The alpha-beta filter simply uses that steady-state gain from the start, skipping the covariance recursion entirely. With state (position, velocity) and sample interval T , it predicts $p^- = \hat{p}_{\text{prev}} + T\hat{v}_{\text{prev}}$ and then updates with two fixed gains,

$$\hat{p} = p^- + \alpha(z - p^-), \quad \hat{v} = v^- + \frac{\beta}{T}(z - p^-), \quad (2.12)$$

where α is the position gain and β the velocity gain. Comparing with the Kalman update, α is exactly the steady-state position gain and β/T the steady-state velocity gain. Because there is no matrix inversion and no covariance to propagate, the alpha-beta filter costs a handful of multiplies per step, which is why it is still used in high-rate and embedded trackers.

Choosing the gains, and alpha-beta-gamma

The two gains are not free: for the filter to be stable they must satisfy $0 < \alpha < 1$, $0 < \beta \leq 2$, and $4 - 2\alpha - \beta > 0$. In practice a single design parameter, a maneuvering or tracking index that compares the expected target acceleration to the measurement noise, fixes both gains through a standard relation. The most common is the Benedict-Bordner criterion, which minimizes a blend

of steady-state error and transient response and gives $\beta = \alpha^2/(2 - \alpha)$, reducing the design to one number. When targets accelerate rather than merely drift, the alpha-beta-gamma filter adds an acceleration term to the state and a third gain γ ; it is the constant-gain counterpart of a constant-acceleration Kalman filter, and the same steady-state argument links its gains to that filter. The trade is always the same: fixed gains cost almost nothing but cannot adapt when the situation changes.

Alpha-beta vs the Kalman filter

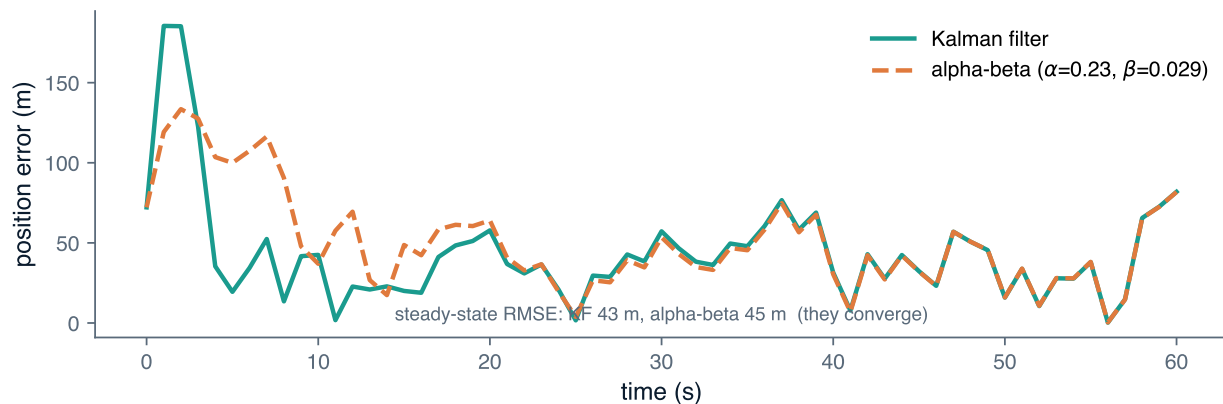


Figure 2.5: Position error of the Kalman filter and a matched alpha-beta filter on the same track: they agree once settled, and the Kalman filter converges faster from a cold start.

2.5 Table look-up and gain-scheduled filters

There is a middle option between the full Kalman filter and a single fixed gain. Notice from the derivation that the covariance recursion, and therefore the gain sequence $\mathbf{K}_0, \mathbf{K}_1, \dots$, depends only on the model matrices $\mathbf{F}, \mathbf{H}, \mathbf{Q}, \mathbf{R}$ and not on the actual measured values. The gains can therefore be computed once, offline, and stored in a table; online, the filter looks up the gain for the current step and applies it with no matrix inversion at all. This table look-up filter achieves the full Kalman filter's time-varying accuracy, including the fast initial convergence a single fixed gain lacks, at a small fraction of the runtime cost, trading memory for computation. The related idea of gain scheduling stores several gain sets for different operating regimes, for example different signal-to-noise ratios or sensor-target geometries, and switches between them as conditions change. Both are workhorses in compute-limited or high-rate systems.

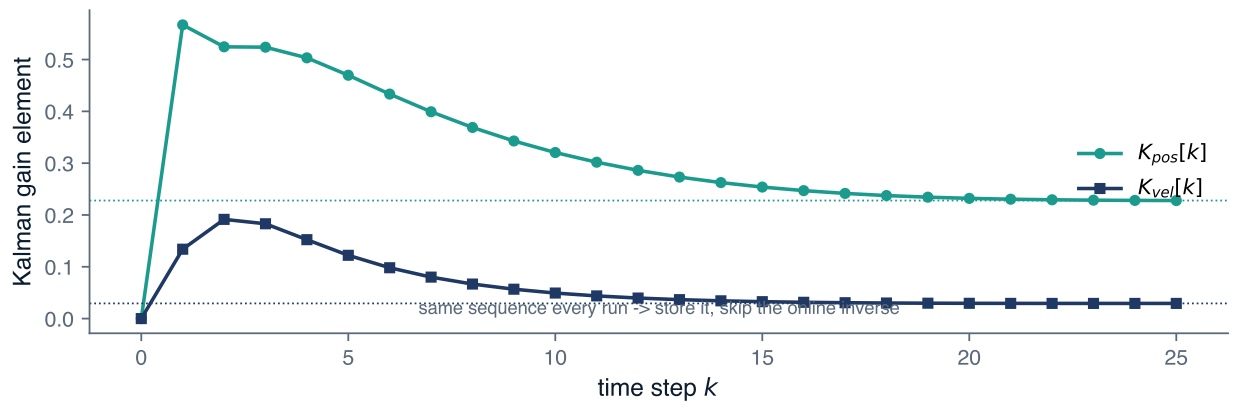


Figure 2.6: The Kalman gain sequence depends only on the model, not the data, so it can be computed once and stored: the basis of the table look-up filter.

The gain schedule is precomputable

2.6 Comparative performance

The three filter classes trade the same three currencies: accuracy, online compute, and tuning or design effort. The full Kalman filter gives the best accuracy and adapts its gain to the current uncertainty, including fast convergence after initialization or a maneuver, but it pays a matrix inversion every step and demands that $\mathbf{Q}$ and $\mathbf{R}$ be tuned to the target and sensor. The alpha-beta and alpha-beta-gamma filters are the cheapest and simplest to implement and tune (one index), but their fixed gains are suboptimal during transients and cannot react when conditions change. The table look-up / gain-scheduled filter recovers the Kalman filter's time-varying accuracy at low online cost by precomputing gains, at the price of storage and an assumption that the noise environment matches the one the table was built for. The right choice follows from the compute budget and how much the scenario varies: a stable, high-rate sensor favors fixed or scheduled gains, while a maneuvering-target, variable-noise problem favors the full filter.

Accuracy, cost, and effort

The person behind it

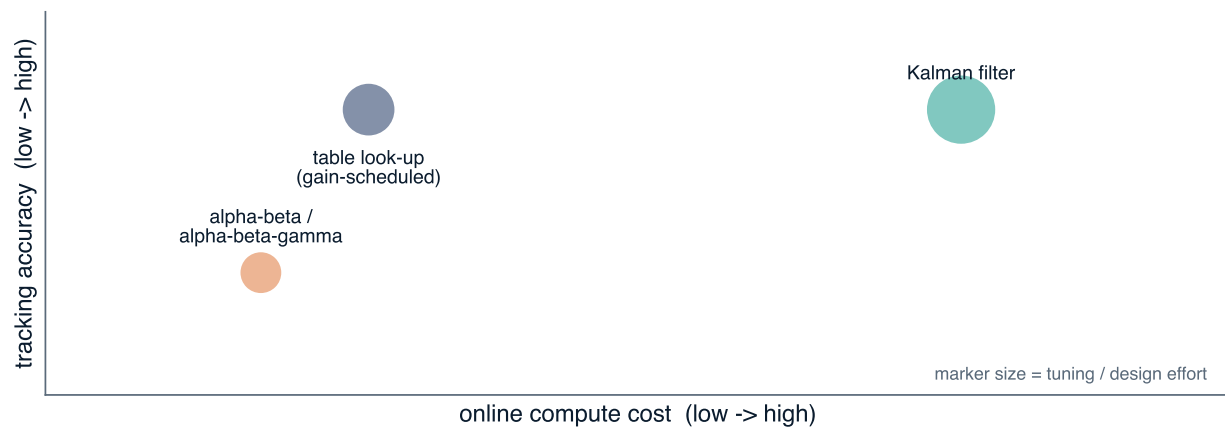


Figure 2.7: The three linear-filter classes on online compute cost versus tracking accuracy; marker size hints at the tuning and design effort each demands.



THE PERSON BEHIND IT

Rudolf E. Kalman (1930-2016)

Hungarian-born American engineer and mathematician. His 1960 paper "A New Approach to Linear Filtering and Prediction Problems" recast estimation in state-space, time-domain form as a recursive, model-based algorithm, superseding Wiener's frequency-domain filter for the problems trackers face. Stanley Schmidt at NASA Ames adapted it (as the extended Kalman filter of Module 3) for on-board Apollo navigation, its first great application. Kalman received the National Medal of Science in 2009.

Key takeaways

The Kalman filter is recursive weighted least squares with a prediction step that lets the state move and the uncertainty grow. Its predict step propagates the mean through $\mathbf{F}$ and inflates the covariance by $\mathbf{Q}$; its update step corrects the state by a gain-weighted innovation and shrinks the covariance. The gain is derived as the minimizer of the posterior covariance, and it reads physically as the balance between process noise $\mathbf{Q}$ (model trust) and measurement noise $\mathbf{R}$ (sensor trust). When that gain settles, freezing it gives the cheap alpha-beta and alpha-beta-gamma filters; precomputing the whole gain sequence gives the table look-up filter, which recovers the full filter's accuracy at low online cost. Module 3 keeps this structure but confronts the nonlinearity Module 1 flagged, where $\mathbf{H}$ becomes a nonlinear $h(\cdot)$ and the plain Kalman filter no longer applies.

Exercise 2a: Tune the Filter

Exercise 2a: Tune the Filter (15 min)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex02_linear_filters.py
```

Select the Maneuvering target preset and keep the filter set to Kalman.

1. Turn the process noise Q down to its lowest setting. Describe what the estimate does when the target maneuvers, and why (link it to the gain).
2. Turn Q up to its highest setting. Describe what now happens between maneuvers, and why.
3. Find a Q that balances the two. Note the position RMSE you achieve.
4. Now raise the measurement noise R and watch the steady-state gain read-out. Which way does the gain move, and what does that mean the filter is doing?

Deliverable: the Q you chose and its RMSE, plus short answers to items 1, 2, and 4 (a sentence each).

Exercise 2b: Match Alpha-Beta to the Kalman Filter

Exercise 2b: Match Alpha-Beta to the Kalman Filter (15 min)

No coding required. In the same app, keep the target settings from Exercise 2a and note the Kalman filter's steady-state position RMSE and its steady-state gain (shown in the read-out).

1. Switch the filter to alpha-beta. Two gain sliders, α and β , appear.
2. Adjust α and β until the alpha-beta position RMSE matches the Kalman filter's. Record the values you found.
3. Compare your α to the Kalman filter's steady-state position gain from the read-out. What do you notice?
4. Watch the first few seconds of each run. Which filter settles faster from a cold start, and why? (Think about what the Kalman gain does early on.)

Deliverable: the α, β you found and the matched RMSE, plus a sentence on how α relates to the Kalman gain and a sentence on the transient difference.

Multi-Target Tracking Training Workshop

Module 3 · Nonlinear Filters: EKF, Unscented & Particle

2026-09-17

Table of contents

3	Nonlinear Filters: EKF, Unscented & Particle	3
	Learning objectives	3
	Where Module 2 left us	3
	3.1 Why linearity breaks	3
	A Gaussian through a nonlinearity	4
	3.2 The Extended Kalman Filter: linearize	4
	The EKF equations	5
	What linearization costs	5
	3.3 Where the EKF degrades	6
	3.4 The unscented transform	6
	Sigma points and weights	6
	Sigma points beat the tangent	7
	The Unscented Kalman Filter	7
	3.5 Particle filters	7
	Sequential importance sampling	8
	Resampling and degeneracy	8
	Particles: reweight and resample	9
	3.6 Choosing among them	9
	EKF, UKF, and PF on one track	9
	The fidelity-cost ladder	9
	The people behind it	9
	Key takeaways	11
	Exercise 3: EKF vs UKF on Range/Bearing	11

Module 3: Nonlinear Filters: EKF, Unscented & Particle

Student Guide

Learning objectives

Module 2 gave the optimal recursive estimator for linear-Gaussian models. But Module 1 already showed the catch: real measurements (range, azimuth, bearing) are nonlinear functions of the Cartesian state, and some dynamics (the coordinated turn) are nonlinear too. This module extends the linear machinery to that reality along a ladder of fidelity and cost. The Extended Kalman Filter linearizes with Jacobians and reuses the Kalman equations. The Unscented Kalman Filter drops the Jacobians and instead pushes a small set of sample points through the true nonlinearity. The particle filter drops the Gaussian assumption entirely and represents the posterior with weighted samples. We derive each, show where it works and where it breaks, and end with how to choose.

Where Module 2 left us

The Kalman filter of Module 2 is exact and optimal, but only under two assumptions: linear models and Gaussian noise. Tracking violates the linearity assumption immediately, because a sensor measures range and angles, which are nonlinear functions of the Cartesian state, $z = h(\mathbf{x}) + v$. As Module 1 showed with the banana-shaped cloud, a Gaussian state distribution pushed through a nonlinear measurement is no longer Gaussian, so the constant measurement matrix $\mathbf{H}$ and the clean covariance bookkeeping of the linear filter no longer hold. The three filters in this module are the standard ways to cope.

3.1 Why linearity breaks

Write the general nonlinear system with a possibly nonlinear dynamics function f and a nonlinear measurement function h :

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}) + \mathbf{w}_{k-1}, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad (3.1)$$

with $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ and $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ as before. The difficulty is that pushing a Gaussian through h produces a non-Gaussian result: the distribution curves, and its true mean is generally not h of the input mean, $\mathbb{E}[h(\mathbf{x})] \neq h(\mathbb{E}[\mathbf{x}])$, a bias that grows with the curvature of h and the size of the covariance. There are two broad responses. Approximate the function: replace h by a linear approximation and keep the Kalman machinery (the EKF). Or approximate the density: represent the distribution by carefully chosen points (the unscented transform and UKF) or by random samples (the particle filter), and push those through the exact nonlinearity. The rest of the module develops both responses.

A Gaussian through a nonlinearity

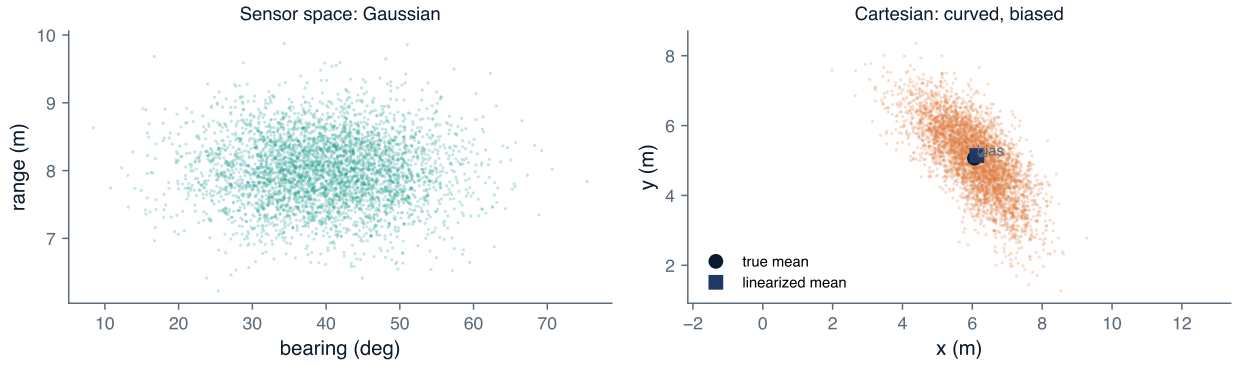


Figure 3.1: A Gaussian in range and bearing converts to a curved, biased cloud in Cartesian coordinates: the true mean is not the linearized mean.

3.2 The Extended Kalman Filter: linearize

The Extended Kalman Filter keeps the Kalman filter exactly, but replaces the constant matrices by first-order Taylor linearizations evaluated at the current estimate. For the measurement, expand h about the prediction $\hat{\mathbf{x}}_{k|k-1}$:

$$h(\mathbf{x}) \approx h(\hat{\mathbf{x}}_{k|k-1}) + \mathbf{H}_k (\mathbf{x} - \hat{\mathbf{x}}_{k|k-1}), \quad \mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}}, \quad (3.2)$$

where $\mathbf{H}_k$ is the measurement Jacobian, the matrix of partial derivatives of each measurement component with respect to each state component. When the dynamics f are also nonlinear, they are linearized the same way with the dynamics Jacobian $\mathbf{F}_k = \partial f / \partial \mathbf{x}$ evaluated at $\hat{\mathbf{x}}_{k-1|k-1}$. With these local linear approximations in hand, the filter runs the ordinary Kalman predict and update equations.

The EKF equations

The EKF predicts the mean through the true (nonlinear) dynamics but propagates the covariance through the Jacobian:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}), \quad \mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^\top + \mathbf{Q}. \quad (3.3)$$

The update computes the innovation through the true measurement function, but the innovation covariance, gain, and covariance update all use the Jacobian:

$$\mathbf{y}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1}), \quad \mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R}, \quad (3.4)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k, \quad \mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}. \quad (3.5)$$

Structurally this is the Module 2 Kalman filter; only $\mathbf{F}$ and $\mathbf{H}$ have become state-dependent Jacobians re-evaluated every step. For a sensor at the origin measuring range and bearing, $h = [\sqrt{p_x^2 + p_y^2}, \text{atan2}(p_y, p_x)]^\top$, the measurement Jacobian (with $r^2 = p_x^2 + p_y^2$) is

$$\mathbf{H} = \begin{bmatrix} p_x/r & 0 & p_y/r & 0 \\ -p_y/r^2 & 0 & p_x/r^2 & 0 \end{bmatrix}, \quad (3.6)$$

evaluated at the predicted state. Computing and coding such Jacobians correctly is part of the EKF's cost.

What linearization costs

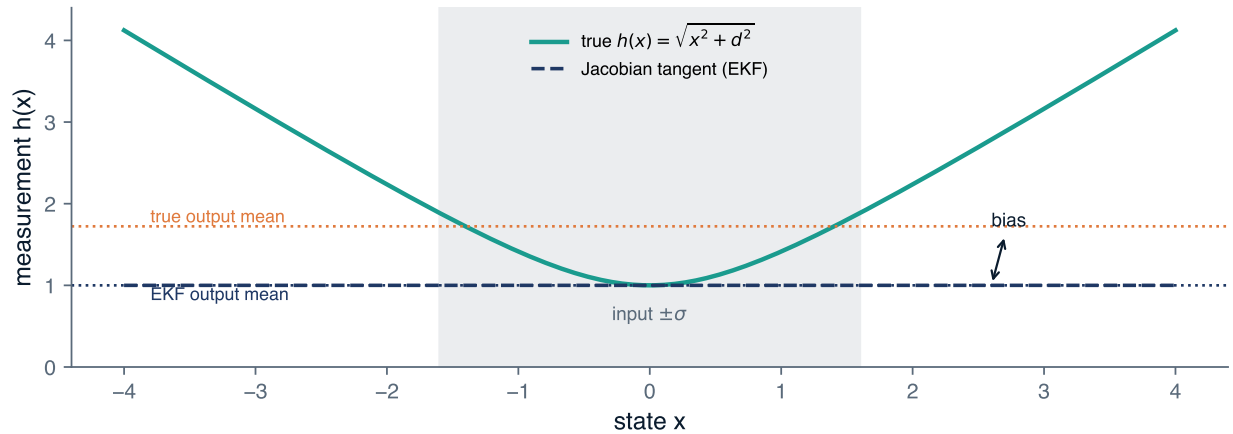


Figure 3.2: A nonlinear measurement with its Jacobian tangent: a symmetric input spread maps to a skewed output, so the EKF (tangent) output mean is biased.

3.3 Where the EKF degrades

The EKF is the workhorse nonlinear filter, but its single approximation, a tangent line at the current estimate, has predictable failure modes. The linearization error grows with the product of the function's curvature and the size of the covariance: when the uncertainty spans a region over which h curves appreciably, the tangent is a poor stand-in. Because the tangent passes through $h(\hat{\mathbf{x}})$ rather than the true transformed mean, the update is biased, as the previous figure showed. Worse, the covariance update uses the same tangent, so the filter can report a covariance that is too small, become overconfident, stop trusting measurements, and diverge. And every model needs a hand-derived, correctly coded Jacobian, which is a maintenance and correctness burden. The EKF is a good choice when h is only mildly curved over the state uncertainty, and a risky one when it is not, which motivates the derivative-free alternatives.

3.4 The unscented transform

The unscented transform rests on a simple observation: it is easier to approximate a probability distribution with a few points than to approximate an arbitrary nonlinear function. Rather than linearize h , it chooses a small set of $2n + 1$ deterministic sigma points whose sample mean and covariance equal those of the state, pushes each point through the true nonlinear function, and then reads the transformed mean and covariance back off the moved points. No derivatives are taken, and the result captures the mean and covariance to second order in the Taylor sense (third order for Gaussian inputs), which is strictly better than the EKF's first-order tangent.

Sigma points and weights

Concretely, for an n -dimensional state with mean $\bar{\mathbf{x}}$ and covariance $\mathbf{P}$, and a spread parameter $\kappa \geq 0$, the sigma points are the mean plus and minus the matrix square-root columns of $(n + \kappa)\mathbf{P}$:

$$\mathcal{X}_0 = \bar{\mathbf{x}}, \quad \mathcal{X}_i = \bar{\mathbf{x}} + \left(\sqrt{(n + \kappa)\mathbf{P}} \right)_i, \quad \mathcal{X}_{i+n} = \bar{\mathbf{x}} - \left(\sqrt{(n + \kappa)\mathbf{P}} \right)_i, \quad (3.7)$$

for $i = 1, \dots, n$, with weights $W_0 = \kappa/(n + \kappa)$ and $W_i = 1/[2(n + \kappa)]$. After mapping every point through the true function, $\mathcal{Y}_i = h(\mathcal{X}_i)$, the transformed mean and covariance are the weighted sample moments

$$\bar{\mathbf{y}} = \sum_{i=0}^{2n} W_i \mathcal{Y}_i, \quad \mathbf{P}_{yy} = \sum_{i=0}^{2n} W_i (\mathcal{Y}_i - \bar{\mathbf{y}})(\mathcal{Y}_i - \bar{\mathbf{y}})^\top. \quad (3.8)$$

Because the points straddle the mean, the reconstructed mean captures the curvature-induced shift that the EKF's tangent misses. (Scaled variants add parameters α, β to tune the spread and to inject knowledge that the input is Gaussian; the plain form above is enough for this course.)

Sigma points beat the tangent

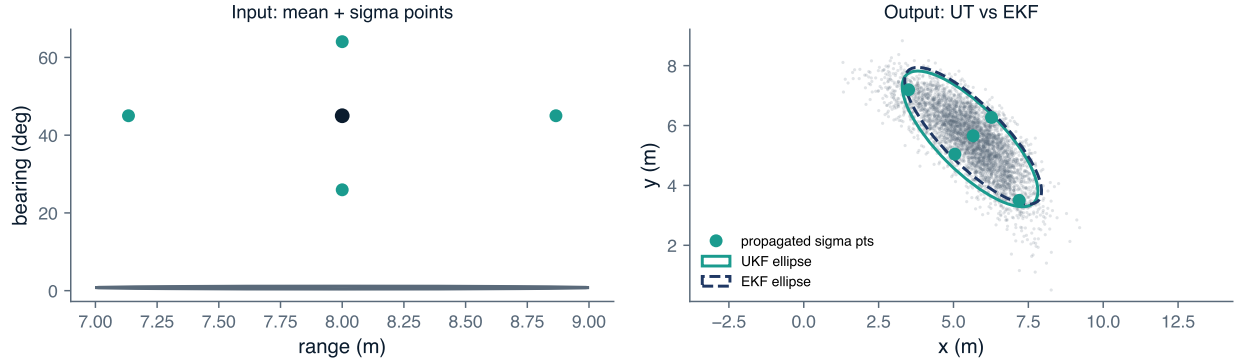


Figure 3.3: Sigma points carried through the true nonlinearity: the unscented ellipse matches the transformed cloud, while the EKF linearized ellipse does not.

The Unscented Kalman Filter

The Unscented Kalman Filter simply applies the unscented transform in each half-step of the Kalman recursion. In predict, the sigma points are pushed through the dynamics f and the predicted mean and covariance are recovered from them, with $\mathbf{Q}$ added. In update, the (regenerated) sigma points are pushed through h to give the predicted measurement mean, the innovation covariance $\mathbf{S}_k = \mathbf{P}_{yy} + \mathbf{R}$, and the state-measurement cross-covariance $\mathbf{P}_{xz} = \sum_i W_i (\mathcal{X}_i - \hat{\mathbf{x}}_{k|k-1})(y_i - \bar{y})^\top$. The gain and update then read

$$\mathbf{K}_k = \mathbf{P}_{xz} \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k, \quad \mathbf{P}_{k|k} = \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top. \quad (3.9)$$

The UKF costs the same order as the EKF (a handful of function evaluations and one small matrix inverse per step) but needs no derivatives, avoids the tangent bias, and tends to be more consistent, that is, its reported covariance better matches its actual error. For angle measurements the only subtlety is to average and difference bearings on the circle (wrap to $[-\pi, \pi]$), which the course code handles.

3.5 Particle filters

The particle filter abandons the Gaussian assumption altogether and represents the full posterior density by a set of N weighted samples, or particles:

$$p(\mathbf{x}_k \mid \mathbf{z}_{1:k}) \approx \sum_{i=1}^N w_k^i \delta(\mathbf{x}_k - \mathbf{x}_k^i), \quad (3.10)$$

where δ is the Dirac delta and the weights sum to one. Because the representation is a general cloud of points rather than a single ellipse, a particle filter can track non-Gaussian and even multimodal posteriors, exactly the ambiguous, hard-nonlinear situations (such as the two-fold bearing ambiguity of Module 1) that defeat a Gaussian filter. The price is computational: enough particles to populate the region where the target might be, which grows quickly with the dimension of the state.

Sequential importance sampling

The particles are updated by importance sampling. Each particle is propagated forward by sampling from a proposal distribution q , and its weight is multiplied by the ratio of the true target density to the proposal:

$$w_k^i \propto w_{k-1}^i \frac{p(\mathbf{z}_k | \mathbf{x}_k^i) p(\mathbf{x}_k^i | \mathbf{x}_{k-1}^i)}{q(\mathbf{x}_k^i | \mathbf{x}_{k-1}^i, \mathbf{z}_k)}. \quad (3.11)$$

The most common choice, the bootstrap filter, proposes from the motion model itself, $q = p(\mathbf{x}_k | \mathbf{x}_{k-1})$, so the transition densities cancel and the weight update reduces to the measurement likelihood, $w_k^i \propto w_{k-1}^i p(\mathbf{z}_k | \mathbf{x}_k^i)$: propagate each particle through the motion model with process noise, then weight it by how well it predicts the new measurement. The weights are then normalized to sum to one, and the state estimate is their weighted mean.

Resampling and degeneracy

Sequential importance sampling has a built-in failure: after a few steps almost all the weight concentrates on a single particle while the rest carry negligible weight, so the cloud stops representing the posterior. This degeneracy is measured by the effective sample size

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N (w_k^i)^2}, \quad (3.12)$$

which falls toward one as the weights concentrate. The cure is resampling: when N_{eff} drops below a threshold (commonly $N/2$), draw a new set of equally weighted particles in proportion to the current weights, duplicating the heavy particles and discarding the light ones. Resampling fixes degeneracy but creates the opposite problem, sample impoverishment, where the duplicated particles collapse onto a few identical points; a small amount of roughening (jitter) added after resampling restores diversity. The figure shows the three moves: a prior cloud, reweighting by a measurement, and resampling.

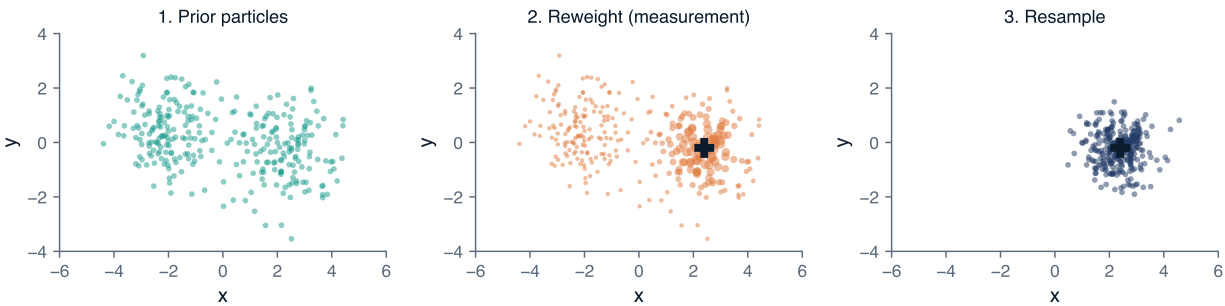


Figure 3.4: Sequential Monte Carlo in three moves: a prior particle cloud, reweighting by a measurement, and resampling onto the high-likelihood region.

Particles: reweight and resample

3.6 Choosing among them

The three filters trade fidelity against cost. The EKF is the cheapest and most widely deployed, but needs correct Jacobians and assumes the nonlinearity is mild over the state uncertainty. The UKF costs the same order, needs no Jacobians, and represents the transformed mean and covariance more faithfully, so it is the safer default whenever the nonlinearity is more than mild. The particle filter is the most general, handling non-Gaussian and multimodal posteriors, but it is far more expensive and its required particle count grows sharply with state dimension, so a basic bootstrap filter can struggle when part of the state (such as velocity) is only weakly observed. For the range/bearing problem of this module, the EKF and UKF are often close in accuracy, with the UKF's advantage widening as the bearing uncertainty (and hence the nonlinearity) grows; the truly dramatic separation appears in the strongly nonlinear, observability-limited setting of bearing-only tracking, which is Module 4.

EKF, UKF, and PF on one track

The fidelity-cost ladder

The people behind it

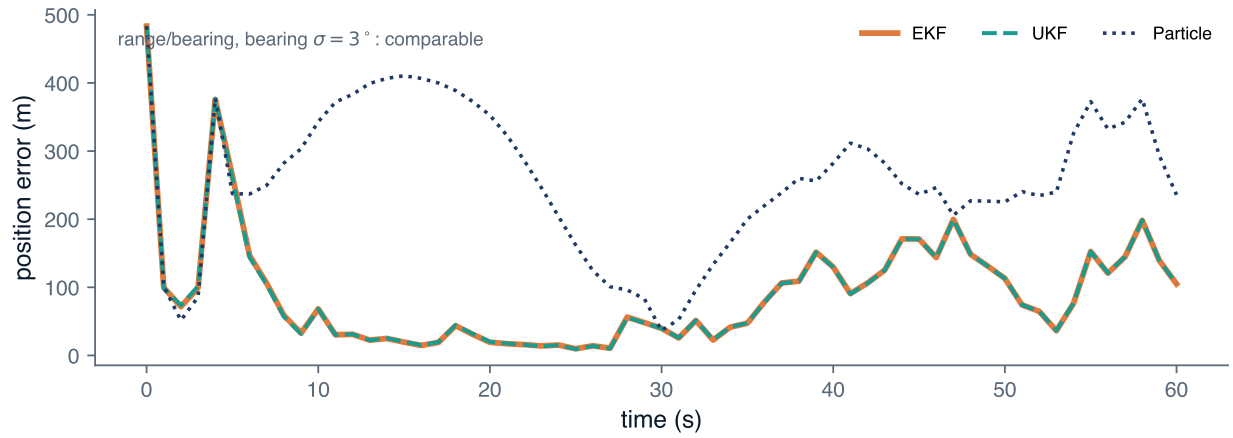


Figure 3.5: EKF, UKF, and particle-filter position error over time on one common range/bearing track: at this moderate nonlinearity the three are comparable.

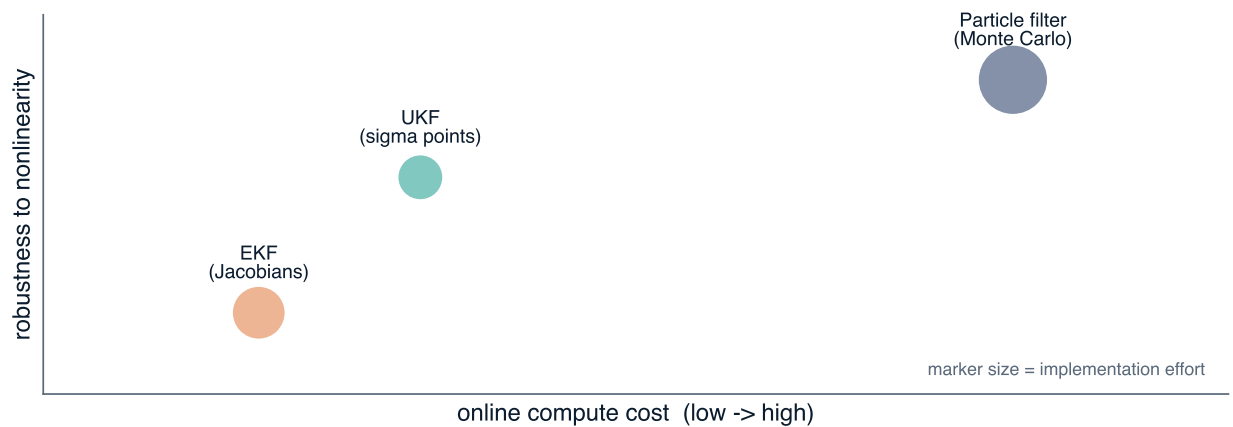
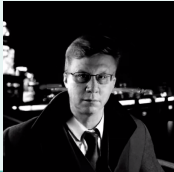


Figure 3.6: The three nonlinear filters on online compute cost versus robustness to nonlinearity; marker size hints at implementation effort.



THE PERSON BEHIND IT

Simon Julier & Jeffrey Uhlmann (unscented transform, 1997)



Working at the University of Oxford, Julier and Uhlmann introduced the unscented transform and the Unscented Kalman Filter to avoid the Extended Kalman Filter's two weaknesses: the burden of deriving Jacobians and the bias and inconsistency of linearization. Their insight, that a small set of deterministically chosen points can carry a distribution through a nonlinearity more faithfully than a tangent line, made derivative-free nonlinear filtering practical and is now standard across tracking and navigation.

The historical bridge runs back to Module 2. Rudolf Kalman's 1960 filter was linear; it was Stanley F. Schmidt at NASA Ames who linearized it for the nonlinear dynamics of spaceflight, producing the Extended Kalman Filter (still sometimes called the Kalman-Schmidt filter) that flew on Apollo. The EKF was the nonlinear workhorse for three decades, and the unscented transform was the response to its limitations.

Key takeaways

A nonlinear measurement function breaks the linear filter's Gaussian bookkeeping, introducing a mean bias and a curved, non-Gaussian spread. The Extended Kalman Filter copes by linearizing with Jacobians: cheap and familiar, but degrading as curvature and covariance grow, and demanding hand-coded derivatives. The Unscented Kalman Filter pushes a small set of sigma points through the true model, avoiding Jacobians and capturing the transformed mean and covariance more faithfully and consistently, at the same order of cost. The particle filter represents the posterior with weighted samples and handles non-Gaussian and multimodal problems, at a computational cost that scales poorly with dimension. The engineering task is to match the filter to the strength of the nonlinearity, the state dimension, and the compute budget. Module 4 takes the hardest case, bearing-only tracking, where observability itself is at stake and these choices bite hardest.

Exercise 3: EKF vs UKF on Range/Bearing

Exercise 3: EKF vs UKF on Range/Bearing (30 min)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex03_nonlinear_filters.py
```

The app tracks a constant-velocity target past a fixed sensor that reports range and bearing, running an EKF and a UKF on the same measurements. Controls set the sensor geometry (how close the target passes), the range and bearing noise, and the process noise; the plot shows the truth, the measurements, and both filter estimates, with position RMSE and a

covariance-consistency read-out for each.

Work through these steps and note what you see:

1. Start with a precise bearing sensor. Confirm the EKF and UKF give essentially the same track and RMSE. Why should they agree here?
2. Increase the bearing noise in stages. Watch the RMSE and the consistency read-out for each filter. Which degrades first, and which holds up better?
3. Load the close, fast pass preset (strong nonlinearity). Compare the two filters' accuracy and their reported covariance. Where does the EKF become overconfident (reported covariance smaller than its actual error)?
4. The UKF uses no Jacobians. Given what you saw, when is the EKF still the right choice, and when would you reach for the UKF?

Deliverable: the bearing-noise level at which the EKF and UKF visibly diverge, the RMSE and consistency you recorded for each, and one paragraph describing the EKF failure mode you observed.

Multi-Target Tracking Training Workshop

Module 4 · Angle-Only Tracking

2026-09-17

Table of contents

4	Angle-Only Tracking	3
	Learning objectives	3
	Where Module 3 left us	3
4.1	The angle-only problem	3
	The observer maneuver that unlocks range	4
4.2	Observability: why a maneuver is required	4
	Range stays lost until the turn	5
4.3	EKF in Cartesian coordinates	5
	Why the Cartesian EKF goes inconsistent	5
	The covariance collapses below the error	6
4.4	The pseudo-linear estimator	6
4.5	Modified polar coordinates	6
	MPC keeps the filter honest	7
4.6	Practical comparison	7
	Three estimators, one requirement	8
	The people behind it	8
	Key takeaways	8
	Exercise 4: Observer Maneuver and Range Observability	9

Module 4: Angle-Only Tracking

Student Guide

Learning objectives

This is the first true application module, and it takes the hardest single-sensor case: tracking a target from bearings alone, with no range. It is where the Module 3 warning about the Extended Kalman Filter's optimism becomes a practical failure, and where a new issue, observability, moves to the foreground: from a single non-maneuvering platform the target range simply cannot be recovered, no matter how good the filter. We establish the observability requirement (the observer must maneuver), formulate the Cartesian EKF and show its covariance-consistency failure, introduce the pseudo-linear estimator and its bias, and then the modified-polar-coordinate formulation that decouples the unobservable range and keeps the filter well behaved. Everything reuses the nonlinear-filtering machinery of Module 3.

Where Module 3 left us

Module 3 built the nonlinear filters and flagged the Extended Kalman Filter's tendency to report a covariance smaller than its true error. Bearings-only tracking is the setting where that tendency becomes a genuine failure, because the measurement carries no range information at all. On top of the nonlinearity, a structural issue appears: observability. From a single platform that moves at constant velocity, the target's range is not merely hard to estimate, it is mathematically unrecoverable. This module treats observability first, then the estimators that cope with it.

4.1 The angle-only problem

In angle-only tracking the sensor reports only the bearing to the target,

$$\beta_k = \text{atan2}(T_y - O_{y,k}, T_x - O_{x,k}) + v_k, \quad v_k \sim \mathcal{N}(0, \sigma_\beta^2), \quad (4.1)$$

where $\mathbf{T} = (T_x, T_y)$ is the target position and $\mathbf{O}_k$ is the (known) observer position. A single bearing defines a line of sight from the observer; the target lies somewhere along it, at an unknown range.

This is the native situation for passive sensors, passive sonar, electronic support measures, and infrared search and track, which detect a target's emissions or radiation but have no cooperative ranging. The central difficulty, developed next, is that a single platform travelling at constant velocity cannot recover the range at all.

The observer maneuver that unlocks range

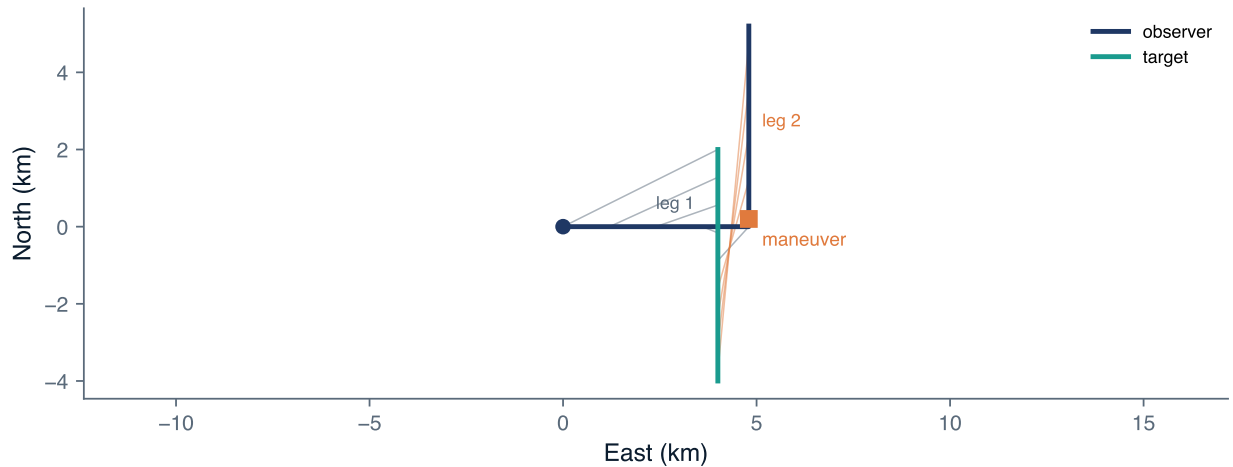


Figure 4.1: Bearings-only geometry: lines of sight from the first (straight) leg are consistent with many target ranges; after the observer turns, the second-leg lines of sight cross the first and fix the target.

4.2 Observability: why a maneuver is required

The observability result is the heart of this module. If the observer moves at constant velocity, the bearings-only problem is unobservable in range: for any candidate target trajectory that fits the measured bearing history, a one-parameter family of scaled trajectories (different range, with a correspondingly different velocity) produces exactly the same bearings. No estimator can choose among them, because they are indistinguishable in the data. The cure is an observer maneuver. When the observer changes course or speed, the relative geometry on the second leg is consistent with only one of those scaled trajectories, so range becomes observable. The practical rule is that the observer must out-maneuver the target, executing motion of higher order than the target's, and the sharper and better-timed the maneuver, the stronger the observability. The figure shows range error staying large on a straight leg and tightening only after the turn.

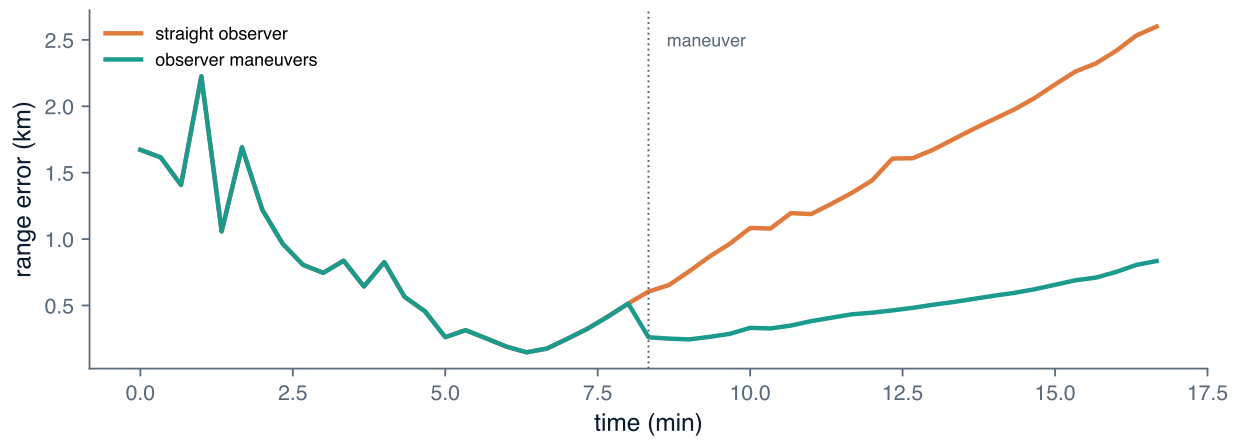


Figure 4.2: Range error over time for a straight observer versus one that maneuvers: on a single leg the range is barely constrained; the estimate tightens only after the observer turns.

Range stays lost until the turn

4.3 EKF in Cartesian coordinates

The direct approach is the Module 3 Extended Kalman Filter in Cartesian coordinates. The target state is $\mathbf{x} = [T_x, \dot{T}_x, T_y, \dot{T}_y]^\top$ with a constant-velocity model, and the bearing measurement of Equation 4.1 is linearized by its Jacobian, evaluated at the current estimate,

$$\mathbf{H}_k = \begin{bmatrix} -\frac{\Delta y}{r^2} & 0 & \frac{\Delta x}{r^2} & 0 \end{bmatrix}, \quad \Delta = \mathbf{T} - \mathbf{O}_k, \quad r = \|\Delta\|, \quad (4.2)$$

after which the ordinary EKF predict and update equations run unchanged. It is the natural first implementation and is simple to code, but on bearings-only problems it has a well-known and serious weakness.

Why the Cartesian EKF goes inconsistent

The trouble is that the Jacobian Equation 4.2 is evaluated at the current estimate, which, in range, is essentially a guess early on. Linearizing the bearing at a range-biased point makes the update reduce the covariance along the range direction more than the information actually warrants. The filter therefore becomes overconfident: its reported covariance collapses well below its true error, it stops trusting new measurements, and it can lock onto a wrong range and diverge. This is exactly the Module 3 inconsistency, amplified because range is the weakly observable direction. It is at its worst precisely when observability is weakest, on a single leg or before the observer has maneuvered, as the next figure shows.

The covariance collapses below the error

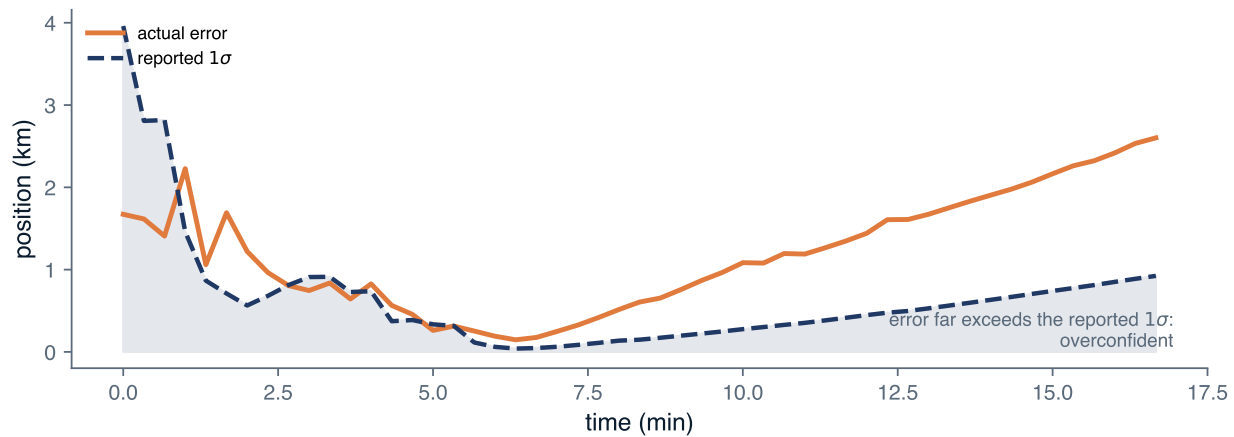


Figure 4.3: A straight-leg bearings-only run: the Cartesian EKF’s reported one-sigma uncertainty collapses far below its actual error, the classic optimistic (inconsistent) covariance.

4.4 The pseudo-linear estimator

A clever alternative removes the linearization entirely. The measured bearing says the target lies on the line of sight, which can be written as a linear constraint on the target position,

$$\sin \beta_k (T_x - O_{x,k}) - \cos \beta_k (T_y - O_{y,k}) = 0. \quad (4.3)$$

With a constant-velocity target, $\mathbf{T}_k = \mathbf{T}_0 + t_k \mathbf{V}$, each measurement becomes a linear equation in the unknown $\theta = [\mathbf{T}_0, \mathbf{V}]$, and stacking the measurements gives an overdetermined linear system $\mathbf{A}\theta = \mathbf{c}$ solved by ordinary or recursive least squares. The pseudo-linear estimator needs no Jacobian and cannot diverge the way the EKF does, which makes it attractive. Its drawback is bias: the rows of $\mathbf{A}$ are built from the noisy bearings themselves, so the regressor is correlated with the measurement noise (an errors-in-variables problem), and the least-squares solution is biased toward the observer, increasingly so at long range and low bearing signal-to-noise. Instrumental-variable and total-least-squares variants reduce the bias at extra cost.

4.5 Modified polar coordinates

Modified polar coordinates (MPC) are the formulation that finally behaves. The state is written as

$$\mathbf{y} = [\beta, \dot{\beta}, \dot{r}/r, 1/r]^\top, \quad (4.4)$$

the bearing, the bearing rate, the range-rate-over-range, and the inverse range. Two properties make this the right coordinate system. First, the bearing measurement is now exactly linear in the state, $z_k = \beta = \mathbf{H}\mathbf{y}$ with $\mathbf{H} = [1 \ 0 \ 0 \ 0]$, so the update introduces no linearization error, the very error that made the Cartesian EKF optimistic. Second, the one component that is unobservable on a straight leg is precisely the inverse range $1/r$, and in MPC it is decoupled from the three observable components $(\beta, \dot{\beta}, \dot{r}/r)$. Its large uncertainty therefore stays contained and does not corrupt the observable states, so the filter remains consistent even before a maneuver and converges cleanly after one. The state propagation between measurements is nonlinear (the modified-polar dynamics, integrated with the known observer motion), but the crucial measurement update is linear. This formulation is due to Aidala and Hammel (1983) and remains the standard well-behaved bearings-only filter.

MPC keeps the filter honest

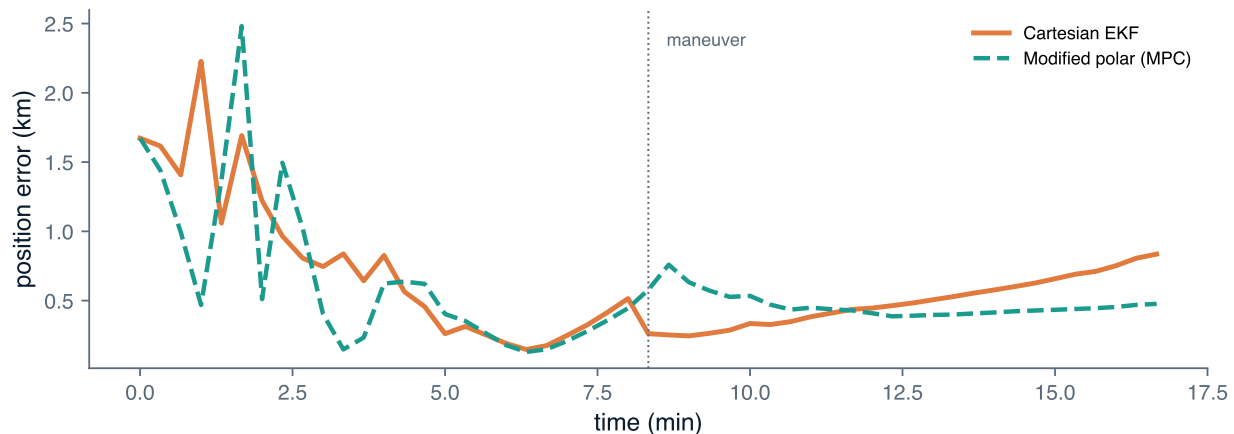


Figure 4.4: Position error on the maneuvering-observer scenario: the modified-polar filter tracks at least as well as the Cartesian EKF and stays consistent, where the EKF is noisier and optimistic.

4.6 Practical comparison

The three estimators trade simplicity against robustness. The Cartesian EKF is the easiest to build and integrates directly with the rest of a tracker, but it is optimistic and prone to range divergence, so it needs care (larger process noise, covariance inflation, or careful initialization). The pseudo-linear estimator avoids divergence and is computationally cheap, but its bias grows at long range and low bearing signal-to-noise, so it is best as an initializer or when its bias is tolerable. Modified polar coordinates give the best-behaved filter, with a linear measurement and a decoupled unobservable range, at the cost of a more involved propagation, and are the usual choice for a dedicated bearings-only tracker. The one thing none of them can escape is the observability

requirement: whatever the estimator, range is recoverable only if the observer maneuvers.

Three estimators, one requirement

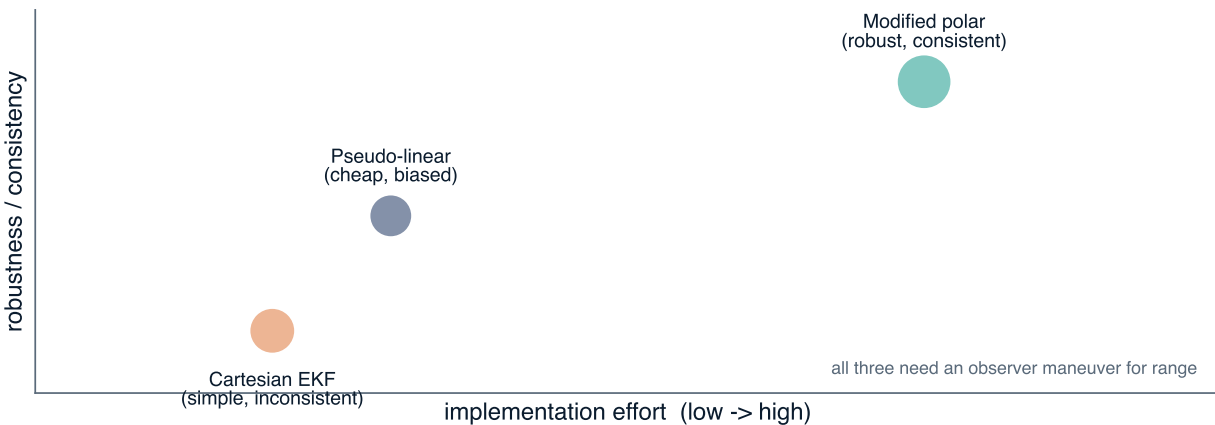


Figure 4.5: The bearings-only estimators on implementation effort versus robustness; all three still need an observer maneuver to make range observable.

The people behind it

photo	THE PERSON BEHIND IT
	V. J. Aidala & S. E. Hammel (modified polar coordinates, 1983) At the Naval Underwater Systems Center, Aidala and Hammel showed why the Cartesian Extended Kalman Filter is inconsistent on bearings-only problems and introduced the modified-polar-coordinate formulation as the fix. Their key insight was to choose a state in which the bearing measurement is linear and the single unobservable quantity, inverse range, is decoupled from the observable states, so its uncertainty cannot corrupt them. Their 1983 paper is the standard reference for well-behaved bearings-only tracking, and the observability analysis that accompanies it underpins how observer maneuvers are planned to this day.

Key takeaways

Angle-only tracking adds observability to the estimation problem: a single bearing is a line of sight, and range is recoverable only when the observer maneuvers, out- moving the target so the later geometry rules out the wrong ranges. The Cartesian EKF is the simplest estimator but is optimistic, collapsing its reported covariance below its true error and risking range divergence, worst where observability is weak. The pseudo-linear estimator trades that divergence for a range-dependent bias. Modified polar coordinates resolve both problems by making the bearing measurement linear

and decoupling the unobservable inverse range, and are the preferred bearings-only filter. Module 5 turns from the sensor problem to the target: maneuvering targets, and the adaptive and multiple-model filters that track them.

Exercise 4: Observer Maneuver and Range Observability

Exercise 4: Observer Maneuver and Range Observability (30 min)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex04_angle_only.py
```

A passive observer measures only the bearing to a constant-velocity target. The app runs a Cartesian EKF (and, optionally, the modified-polar filter) and plots the geometry and the range error over time. You control whether and when the observer maneuvers, the maneuver, the bearing noise, and the geometry.

Work through these steps and note what you see:

1. Set the observer to a straight course (no maneuver). Watch the range error over time. Does the filter ever pin the range? What does the reported uncertainty do compared with the actual error?
2. Turn the observer maneuver on at mid-track. Describe what happens to the range error at and after the turn. Record the final range error with and without the maneuver.
3. Make the maneuver earlier or sharper. How does the timing and size of the maneuver change how quickly range is resolved?
4. Switch the filter to modified polar and compare its reported uncertainty against the Cartesian EKF's. Which is honest about what it knows?

Deliverable: the two range-error curves (straight vs maneuver) with the final range error for each, and a short paragraph explaining, in observability terms, why the maneuver is what makes range recoverable.

Multi-Target Tracking Training Workshop

Module 5 · Maneuvering Targets: Adaptive & Multiple-Model

2026-09-17

Table of contents

5	Maneuvering Targets: Adaptive & Multiple-Model	3
	Learning objectives	3
	Where Module 4 left us	3
5.1	The maneuvering-target problem	4
	A single model lags the turn	4
5.2	The innovation as a maneuver signal	4
	Innovations flag the maneuver	5
5.3	Adaptive filtering: detect and react	5
5.4	The multiple-model idea	6
5.5	The IMM estimator: one cycle	6
	One IMM cycle, in a picture	7
	Mode probabilities track the maneuver	7
5.6	IMM versus a single model	8
	The IMM keeps its error low throughout	8
5.7	Designing an IMM in practice	8
	The people behind it	9
	Key takeaways	9
	Exercise 5: Maneuver Detection and the IMM	10

Module 5: Maneuvering Targets: Adaptive & Multiple-Model

Student Guide

Learning objectives

The previous modules assumed the target obeys one motion model, usually constant velocity. Real targets accelerate, turn, and then straighten out again, and a filter tuned for constant velocity cannot follow those transitions: it lags at the maneuver and, if its process noise is raised enough to keep up, it is noisy everywhere else. This module is about tracking targets that change how they move. It starts with why the single-model filter fails and how the filter's own innovation sequence reveals a maneuver in progress, then develops two families of response. The adaptive approach detects the maneuver and reacts, by inflating the process noise or by estimating the unknown acceleration as an input. The multiple-model approach runs several models at once and blends them by how well each currently fits, and its modern form, the Interacting Multiple Model (IMM) estimator, is the workhorse of maneuvering-target tracking. We build the IMM cycle step by step and show it tracking cleanly through a turn that defeats a single model.

Where Module 4 left us

Modules 2 through 4 built the estimation machinery, the linear Kalman filter, the nonlinear EKF, UKF, and particle filter, and the bearings-only filters, all on the assumption that the target follows a single motion model. Module 4 added the observability problem but still tracked a constant-velocity target. The assumption that breaks now is the motion model itself. A maneuvering target switches between regimes, and the tuning that makes a filter smooth on a straight leg makes it slow to respond to a turn, while the tuning that makes it agile at a turn makes it jittery on the straight legs. No single fixed model resolves that tension, which is what motivates everything in this module.

5.1 The maneuvering-target problem

Consider a target that flies straight, executes a coordinated turn, and then flies straight again. A constant-velocity filter models the target as moving in a straight line with small random accelerations budgeted by its process-noise term q . During the turn the true acceleration is large, sustained, and structured (it is centripetal, always turning the velocity), which is exactly what the CV model does not represent, so the filter's prediction consistently falls behind the truth and the estimate swings wide of the turn. The obvious fix, raising q so the filter trusts the measurements more, does catch the turn faster, but it also makes the filter noisy on the straight legs where the extra process noise is not needed. Lowering q gives clean straight-leg tracking but a large, slow error through the maneuver. A single fixed process noise cannot be right in both regimes, and the figure shows the resulting corner-cutting lag.

A single model lags the turn

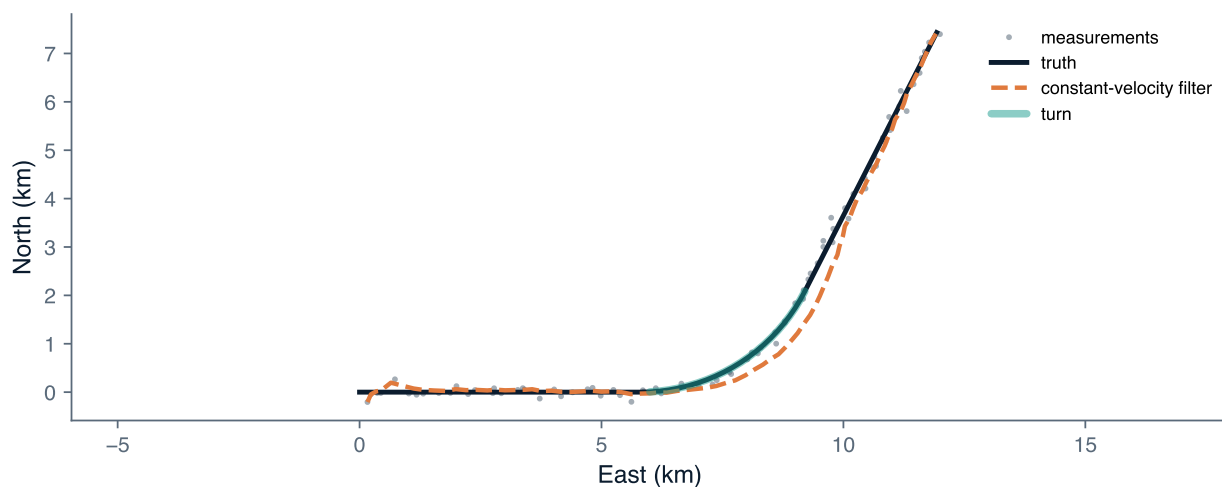


Figure 5.1: A constant-velocity filter tracking a target that turns: tight on the straight legs, but its estimate swings wide of the truth through the maneuver, because the model has no term for the turn.

5.2 The innovation as a maneuver signal

The filter tells you when its model has gone wrong, through the innovation, the difference between the measurement and the filter's prediction of it,

$$\nu_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}, \quad \mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R}. \quad (5.1)$$

When the filter's model matches the target, the innovation sequence is zero-mean and white (uncor-

related in time), with covariance $\mathbf{S}_k$. A maneuver violates both properties at once: the predictions lag in a consistent direction, so the innovations acquire a bias and become correlated. The standard scalar test statistic is the normalized innovation squared (NIS),

$$\epsilon_k = \nu_k^\top \mathbf{S}_k^{-1} \nu_k, \quad (5.2)$$

which, for a consistent filter, follows a chi-square distribution with n_z degrees of freedom (the measurement dimension), so its expected value is n_z . Comparing ϵ_k , or its average over a short sliding window, against a chi-square threshold gives a maneuver detector: as long as NIS sits near n_z the model is fine, and a sustained excursion above the threshold flags a maneuver. The figure shows NIS resting near its expected value on the straight legs and spiking well past the threshold through the turn.

Innovations flag the maneuver

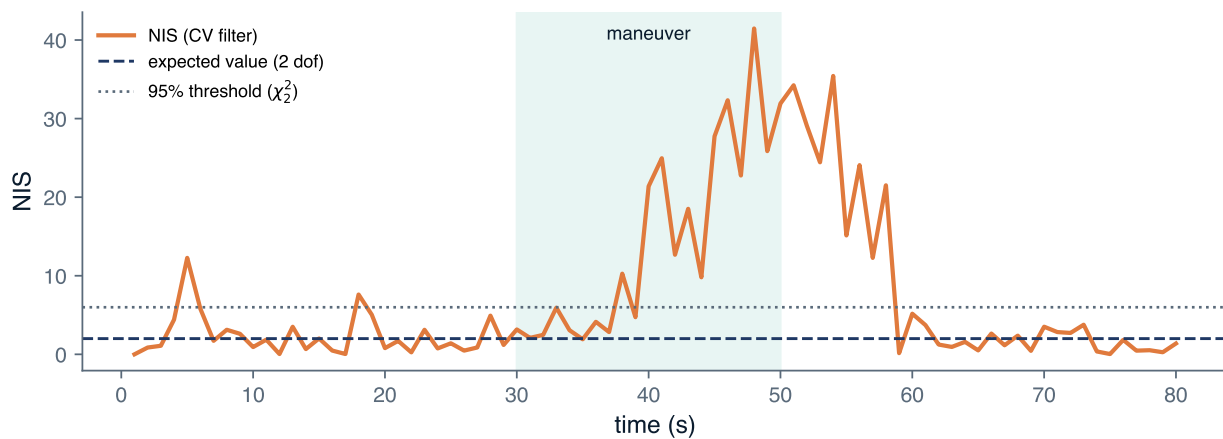


Figure 5.2: Normalized innovation squared (NIS) for the constant-velocity filter: near its expected value of 2 on the straight legs, then a sustained spike past the chi-square threshold through the turn. That excursion is the adaptive maneuver detector.

5.3 Adaptive filtering: detect and react

The adaptive family keeps a single filter but changes it in response to the detector. The simplest version is process-noise switching: run a low- q filter for smooth straight-leg tracking, and when the NIS test fires, inflate q (or directly inflate the covariance) so the filter becomes agile, then restore the low value once the innovations settle. Input estimation, due to Chan, Hu, and Plant, treats the maneuver as an unknown constant acceleration over a sliding window, estimates that input by least squares from the innovation sequence, and corrects the state and covariance retroactively. The variable-state-dimension approach of Bar-Shalom and Birmingham keeps a constant-velocity

model in quiescent tracking and switches to a constant-acceleration model when a maneuver is detected, switching back when it ends. All of these work and all share one structural limitation: they commit to a single model at each instant and depend on a hard detection decision, which can be late (a short maneuver may end before it is declared) or wrong (a false alarm inflates the covariance needlessly). The multiple-model approach removes the hard decision.

5.4 The multiple-model idea

The multiple-model idea is to stop choosing. Instead of one model plus a decision, run a bank of filters, each with a different motion model, for instance a constant-velocity model for the straight legs and one or more coordinated-turn models for the maneuvers. Every filter processes the same measurement, and each reports a likelihood, how probable that measurement was under its model, computed from its own innovation and innovation covariance. These likelihoods update a set of mode probabilities, the probability that each model is the one currently in force, and the output is the probability-weighted blend of the filters. The model in force is treated as a Markov chain: a transition-probability matrix $\pi_{ij} = \Pr(\text{model } j \text{ now} \mid \text{model } i \text{ last})$ encodes how readily the target switches regimes. Because the mode probabilities move continuously as the fit changes, there is no hard switch and no detection delay; the bank simply shifts its weight toward whichever model is fitting best. The practical form of this idea is the Interacting Multiple Model estimator.

5.5 The IMM estimator: one cycle

The Interacting Multiple Model estimator, introduced by Blom and Bar-Shalom in 1988, runs one cycle per measurement in four steps. First, mixing (the “interaction” that names the method): rather than each filter continuing from its own last estimate, every filter is restarted from a mixture of all the filters’ estimates, weighted by the mixing probabilities

$$\mu_{i|j} = \frac{\pi_{ij} \mu_i}{\bar{c}_j}, \quad \bar{c}_j = \sum_i \pi_{ij} \mu_i, \quad (5.3)$$

so a model that is about to become active inherits a sensible state from the models that were active. Second, model-matched filtering: each filter runs an ordinary predict-and-update on its mixed input, and returns the likelihood of the measurement under its model,

$$\Lambda_j = \mathcal{N}(\nu_j; \mathbf{0}, \mathbf{S}_j). \quad (5.4)$$

Third, the mode-probability update combines the predicted mode weight $\bar{c}_j$ with the likelihood,

$$\mu_j = \frac{\bar{c}_j \Lambda_j}{\sum_\ell \bar{c}_\ell \Lambda_\ell}. \quad (5.5)$$

Fourth, combination: the output estimate and covariance are the mode-probability- weighted blend of the individual filters' updated estimates. The cost is only that of the filters in the bank plus the light mixing algebra, and the mixing step is what gives the IMM its excellent transient response, far better than running the same filters independently.

One IMM cycle, in a picture

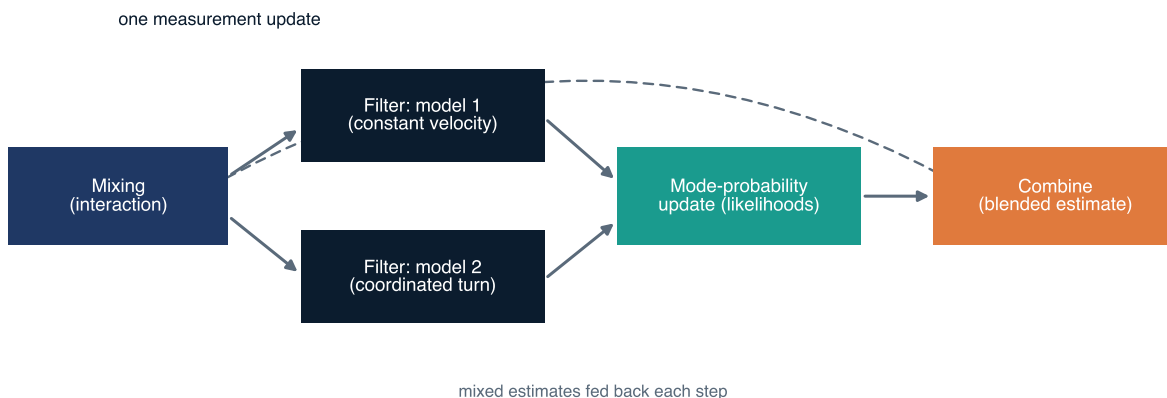


Figure 5.3: One cycle of the IMM estimator: the mode-conditioned estimates are mixed by the transition probabilities, each model filters the same measurement, the innovation likelihoods update the mode probabilities, and the outputs are blended into a single estimate.

Mode probabilities track the maneuver

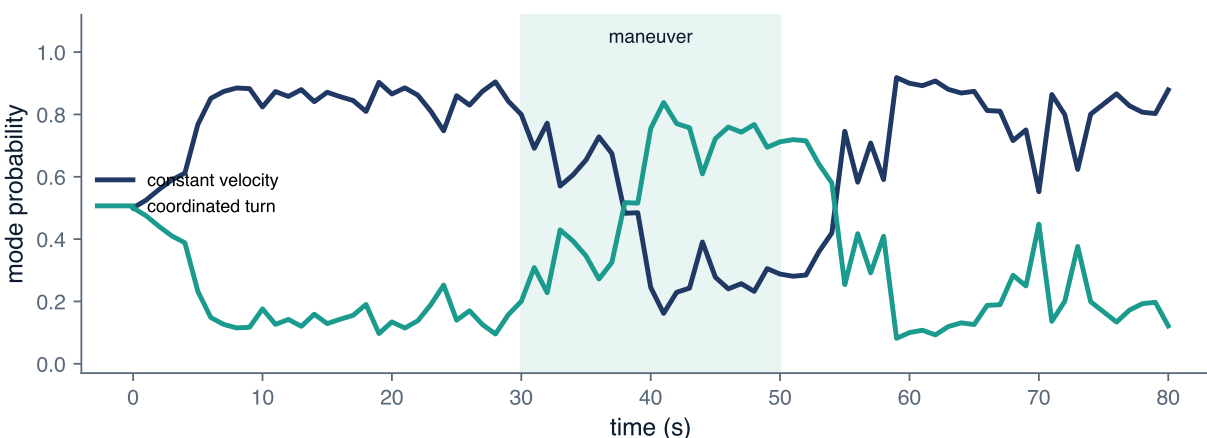


Figure 5.4: IMM mode probabilities over time: the constant-velocity model holds the weight on the straight legs, the coordinated-turn model takes over through the maneuver, and the weight swings back afterward, all with no hard switch.

5.6 IMM versus a single model

Put the single-model filter and the IMM on the same data and the difference is clear. The single constant-velocity filter tracks well on the straight legs and its position error spikes through the turn, exactly the lag seen earlier. The IMM, with a constant-velocity model and a coordinated-turn model in its bank, keeps its error low throughout: on the straight legs the constant-velocity model holds most of the mode probability and the IMM is as smooth as the single filter, and through the turn the weight shifts to the turn model, giving the IMM that model's agility precisely when it is needed and then handing the weight back. It achieves the agility of a high-process-noise filter at the maneuver and the smoothness of a low-process-noise filter on the straight legs, with no hard switch and no detection delay. The next figure quantifies the improvement.

The IMM keeps its error low throughout

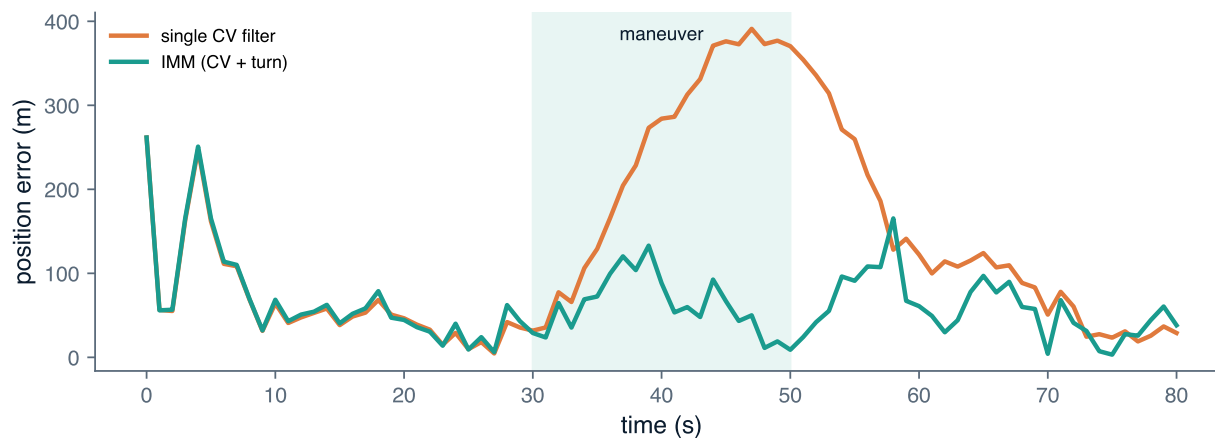


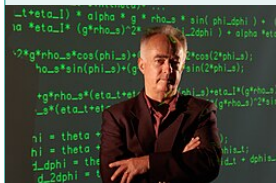
Figure 5.5: Position error over time: the single constant-velocity filter spikes through the turn, while the IMM stays close throughout by shifting weight to the turn model and back. It matches the single filter on the straight legs and beats it at the maneuver.

5.7 Designing an IMM in practice

Designing an IMM is mostly choosing the model set and the transition matrix. The model set should span the target's regimes with as few models as possible: a common bank is a constant-velocity model plus one or two coordinated-turn models at representative turn rates, and sometimes a constant-acceleration model for speed changes. More models are not better, because a redundant or near-duplicate model splits the mode probability and can slow the response; two or three well-separated models usually outperform a large bank. The transition-probability matrix encodes how persistent the modes are: diagonal entries in the range 0.9 to 0.98 say the target usually stays in its current regime, and the off-diagonal entries set how quickly the filter is willing to switch,

with a larger off-diagonal giving faster response but more jitter. A useful rule of thumb is that the expected sojourn time in a mode is about $1/(1 - \pi_{ii})$ steps, which you match to the expected maneuver duration. Finally, tune and validate against consistency, the NIS of the combined output over a chi-square gate, not error alone, so that the IMM is honest about its own uncertainty as well as accurate.

The people behind it



THE PERSON BEHIND IT

Henk A. P. Blom & Yaakov Bar-Shalom (the IMM estimator, 1988)

Henk Blom, then at the Dutch National Aerospace Laboratory, and Yaakov Bar-Shalom, of the University of Connecticut, introduced the Interacting Multiple Model estimator in a 1988 paper. Their contribution was the mixing (interaction) step: earlier multiple-model schemes either kept the number of model histories growing exponentially or reset the filters too crudely, and Blom's insight was to reinitialize each model each step from a probability-weighted mixture of all the models' estimates. This gives near-optimal performance at the cost of just one filter per model, and it made multiple-model tracking practical. Bar-Shalom's books and decades of work established the IMM, with data association, as the standard approach to maneuvering-target tracking; it is now used across air traffic control, radar, and defense systems worldwide.

Key takeaways

Maneuvering targets break the single-model assumption that carried Modules 2 through 4: no fixed process noise is both smooth on the straight legs and agile at a turn. The filter's own innovation sequence detects the mismatch, since a maneuver makes the normalized innovation squared jump above its chi-square expectation, and the adaptive filters act on that signal by inflating process noise, estimating the maneuver as an input, or switching the state dimension, all committing to one model plus a hard decision. The multiple-model approach avoids the decision by running a bank of models and weighting them by how well each fits, and the Interacting Multiple Model estimator is its practical form: mix, filter each model, update the mode probabilities from the likelihoods, and combine. The IMM matches a single filter on the straight legs and a maneuver-tuned filter through the turn, at the cost of one filter per model plus light mixing algebra, and its design reduces to a small model set and a transition matrix. Module 6 turns to the next problem in a real tracker: deciding which measurement belongs to which track, the data-association problem.

Exercise 5: Maneuver Detection and the IMM

Exercise 5: Maneuver Detection and the IMM (two parts, about 15 min each)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex05_maneuvering.py
```

The app simulates a target that flies straight, turns, and straightens out, and runs either a single Kalman filter or a two-model IMM on the same measurements. Use the view selector at the top to switch between the two parts.

Part A - Maneuver detector (single filter). Run the constant-velocity filter and watch its normalized innovation squared (NIS) against the chi-square threshold.

1. Note the NIS on the straight legs. Is it near its expected value (2 for a 2-D position measurement)?
2. Find the smallest detection threshold and window length that flag the turn without firing before it. Record the detection delay (how many steps after the turn starts the detector fires). Note that the raised NIS just after the turn is the filter recovering, not a false alarm.
3. Raise the measurement noise. What happens to the detector's reliability, and why?

Part B - Two-model IMM. Switch to the IMM view (constant velocity plus a coordinated-turn model).

1. Watch the mode probabilities through the run. When does the weight shift to the turn model, and how quickly does it shift back?
2. Change the turn-model rate so it does or does not match the truth. How does a mismatched model affect the switch and the error?
3. Adjust the transition-probability matrix (the stay-in-mode probability). How do stickier vs looser transitions trade response speed against jitter?
4. Compare the IMM position error against the single CV filter from Part A on the same scenario.

Deliverable: for Part A, your chosen threshold and window with the detection delay; for Part B, the mode-probability plot annotated with the turn, and a short paragraph on why the IMM needs no hard detection decision.

Multi-Target Tracking Training Workshop

Module 6 · Single-Target Correlation & Association

2026-09-17

Table of contents

6	Single-Target Correlation & Association	3
	Learning objectives	3
	Where Module 5 left us	3
6.1	Why association is needed	3
6.2	Gating: rectangular and ellipsoidal	4
	The chi-square gate	4
	An ellipsoidal gate around the prediction	5
6.3	Correlation distance: the Mahalanobis metric	5
	Euclidean versus statistical distance	6
6.4	Nearest-neighbor association	6
	Nearest-neighbor breaks down in clutter	7
6.5	Assignment framing: association as a cost	7
	The origins of gating	8
	Key takeaways	8
	Exercise 6: Gating and Nearest-Neighbor in Clutter	8

Module 6: Single-Target Correlation & Association

Student Guide

Learning objectives

Day 1 built the estimators; Day 2 is about keeping a real tracker fed with the right measurements. Every filter in Modules 2 through 5 assumed the measurement it processed came from the target. In a real sensor that is not given: a scan returns several detections, of which at most one is the target and the rest are clutter, and sometimes the target is missed entirely. Choosing which return updates the track is the data-association problem, and this module treats it in the simplest setting, a single target, so the machinery, gating, statistical distance, and the assignment view, is clear before the multi-target combinatorics of Module 8. Every piece here is built directly on Day 1: the innovation and its covariance, which Module 2 derived, are exactly what define the gate and the association metric.

Where Module 5 left us

By the end of Day 1 we had a filter that, once handed the target's measurement, predicts the next measurement $\hat{\mathbf{z}}_{k|k-1} = \mathbf{H}\hat{\mathbf{x}}_{k|k-1}$ and quantifies how uncertain that prediction is through the innovation covariance $\mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R}$. Those two quantities are all we need for this module. What Day 1 did not address is that a real sensor scan produces a whole set of detections, only one of which (at most) is the target, and the filter must be told which one. This module supplies that logic for a single target.

6.1 Why association is needed

At each scan the sensor reports a set of returns $\{\mathbf{z}^1, \dots, \mathbf{z}^m\}$ whose origin is unknown: any one of them might be the target detection, and the remainder are clutter, spurious returns from noise, reflections, or other objects. The target itself is detected only with probability P_D , which is below

one, so some scans contain no target return at all. This is the difficulty the Day 1 filters silently assumed away. Their update equation takes the measurement as a true observation of the target, so associating a clutter return to the track injects a false observation and pulls the estimate toward the clutter, while failing to associate the true return leaves the track to coast on its prediction with a growing covariance. Clutter is usually modeled as a Poisson process with a spatial density β (expected false returns per unit measurement volume), and the missed-detection behavior by the detection probability P_D ; these two numbers govern how hard the association problem is. The rest of the module builds the logic that makes the choice.

6.2 Gating: rectangular and ellipsoidal

The first step is to throw away returns that are too far from the prediction to plausibly be the target, which is called gating. For a candidate return $\mathbf{z}$ the relevant quantity is the innovation it would produce, $\nu = \mathbf{z} - \hat{\mathbf{z}}_{k|k-1}$, together with the innovation covariance $\mathbf{S}_k$ the filter already computes. Two gate shapes are common. The rectangular gate accepts the return when each innovation component satisfies $|\nu_i| \leq \sqrt{\gamma S_{ii}}$, an axis-aligned box that needs only the diagonal of $\mathbf{S}_k$ and no matrix inverse, so it is cheap enough to apply to every return as a first screen. The ellipsoidal gate is the statistically correct test, accepting when the normalized squared innovation $\nu^\top \mathbf{S}_k^{-1} \nu \leq \gamma$. The rectangle is exactly the bounding box of that ellipse, so it admits everything the ellipse does plus the corners; it is used as a fast pre-gate, with the ellipsoidal test applied only to the survivors. The threshold γ is set next.

The chi-square gate

The threshold has a clean statistical meaning. If the candidate is the true target detection, its innovation is zero-mean Gaussian with covariance $\mathbf{S}_k$ by construction, $\nu \sim \mathcal{N}(\mathbf{0}, \mathbf{S}_k)$, and the normalized squared innovation is therefore chi-square distributed with n_z degrees of freedom, where n_z is the measurement dimension,

$$d^2 = \nu^\top \mathbf{S}_k^{-1} \nu \sim \chi_{n_z}^2. \quad (6.1)$$

To size the gate we pick a gate probability P_G , the fraction of true detections we want to keep, and read the threshold off the chi-square distribution,

$$\Pr(d^2 \leq \gamma) = P_G \implies \gamma = F_{\chi_{n_z}^2}^{-1}(P_G). \quad (6.2)$$

For a two-dimensional position measurement ($n_z = 2$) the chi-square distribution is an exponential and the inverse is closed form, $\gamma = -2 \ln(1 - P_G)$, giving $\gamma = 5.99$ at $P_G = 0.95$ and $\gamma = 9.21$ at $P_G = 0.99$. The choice of P_G is a trade: a larger P_G makes the gate bigger so it rarely rejects the true return, but a bigger gate also admits more clutter, which is exactly what makes the

association downstream harder. The gate is the first and cheapest defense; it does not decide among the survivors, only removes the implausible.

An ellipsoidal gate around the prediction

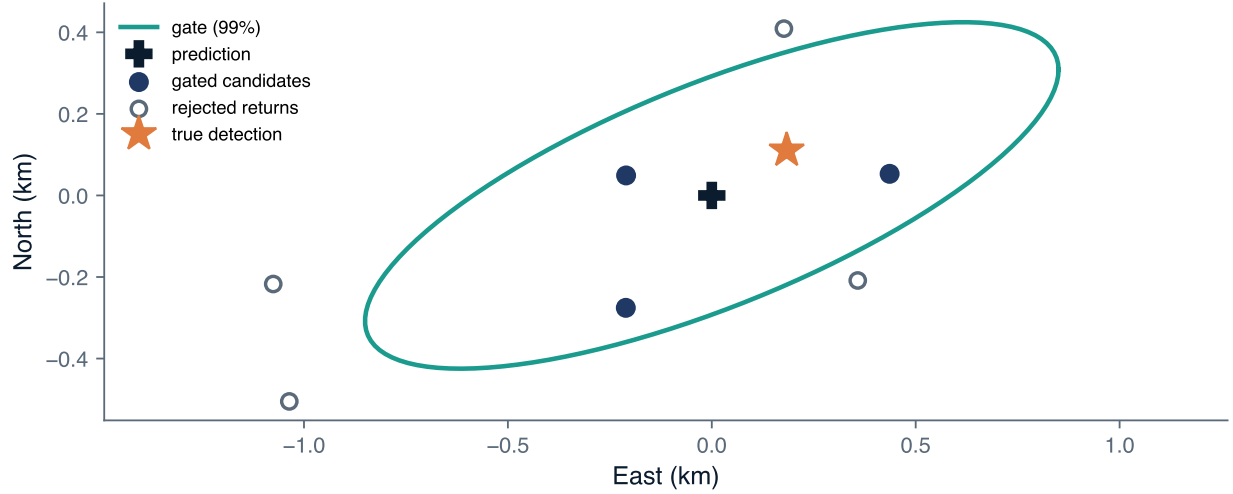


Figure 6.1: An ellipsoidal (chi-square) gate around a predicted measurement: returns inside the gate survive as association candidates, returns outside are rejected, and the true detection sits comfortably inside. The gate is set from the innovation covariance and a gate probability.

6.3 Correlation distance: the Mahalanobis metric

Gating leaves a handful of plausible returns; the association metric ranks them. The right metric is the statistical, or Mahalanobis, distance, the same normalized squared innovation used for the gate,

$$d^2(\mathbf{z}) = \nu^\top \mathbf{S}_k^{-1} \nu, \quad \nu = \mathbf{z} - \hat{\mathbf{z}}_{k|k-1}. \quad (6.3)$$

Its justification is that it is the exponent of the Gaussian measurement likelihood: the probability density of observing $\mathbf{z}$ if it came from the target is

$$\Lambda(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \hat{\mathbf{z}}_{k|k-1}, \mathbf{S}_k) \propto \exp\left(-\frac{1}{2}d^2(\mathbf{z})\right), \quad (6.4)$$

so the return with the smallest Mahalanobis distance is the maximum-likelihood association. Compare this with the plain Euclidean distance $\|\nu\|$, which ignores $\mathbf{S}_k$ entirely and treats every direction as equally uncertain. When the innovation covariance is elongated or correlated, as it is for a sensor with good range accuracy but poor cross-range accuracy, the two metrics disagree: a return that is

nearest in plain distance can lie along the well-known direction, where a small displacement is statistically very unlikely, while a return that is farther in plain distance lies along the poorly-known direction, where the same displacement is entirely ordinary. The Mahalanobis distance normalizes each direction by its own uncertainty and gets the ranking right; the next figure shows a case where it and the Euclidean rule pick different returns.

Euclidean versus statistical distance

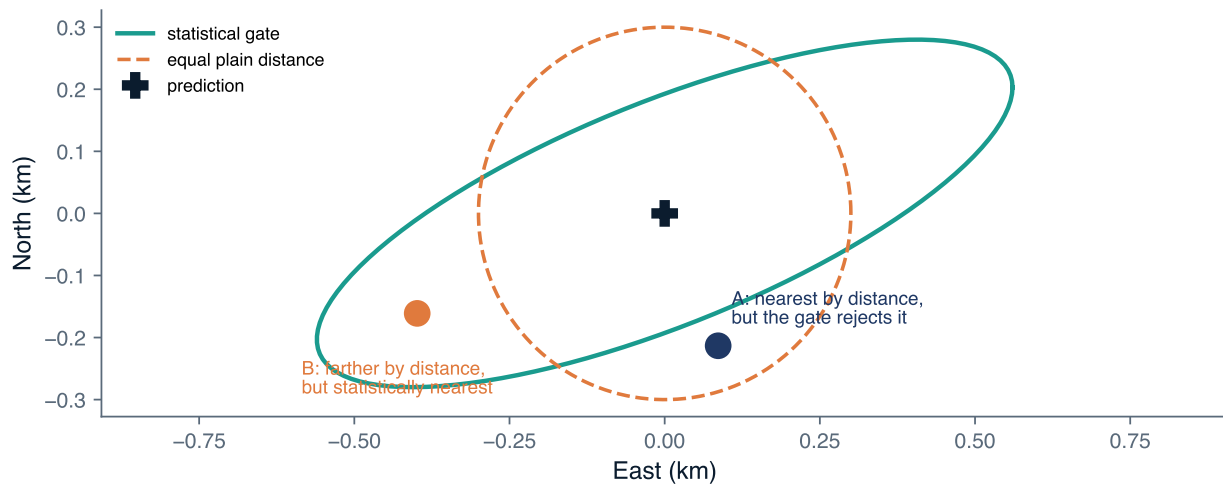


Figure 6.2: Euclidean versus Mahalanobis distance with an elongated innovation covariance: return A is nearest in plain distance but falls outside the statistical gate, while return B is farther in plain distance but is statistically nearest and admitted. The two metrics associate to different returns.

6.4 Nearest-neighbor association

The simplest decision rule is nearest-neighbor: among the returns that survive the gate, update the filter with the one of smallest Mahalanobis distance,

$$\mathbf{z}^* = \arg \min_{\mathbf{z}^j \in \text{gate}} (\mathbf{z}^j - \hat{\mathbf{z}}_{k|k-1})^\top \mathbf{S}_k^{-1} (\mathbf{z}^j - \hat{\mathbf{z}}_{k|k-1}). \quad (6.5)$$

Because the Mahalanobis distance is the likelihood exponent, this is the maximum-likelihood association under the assumption that exactly one of the gated returns is the target, and it costs only a scan over the survivors. If no return falls in the gate, there is nothing to associate and the track coasts on its prediction, with its covariance growing until a detection reappears. Nearest-neighbor is the standard first choice and is adequate when clutter is light, but it commits irrevocably to a single return each scan, and in clutter the nearest return is frequently a false one. The next figure quantifies how quickly that happens.

Nearest-neighbor breaks down in clutter

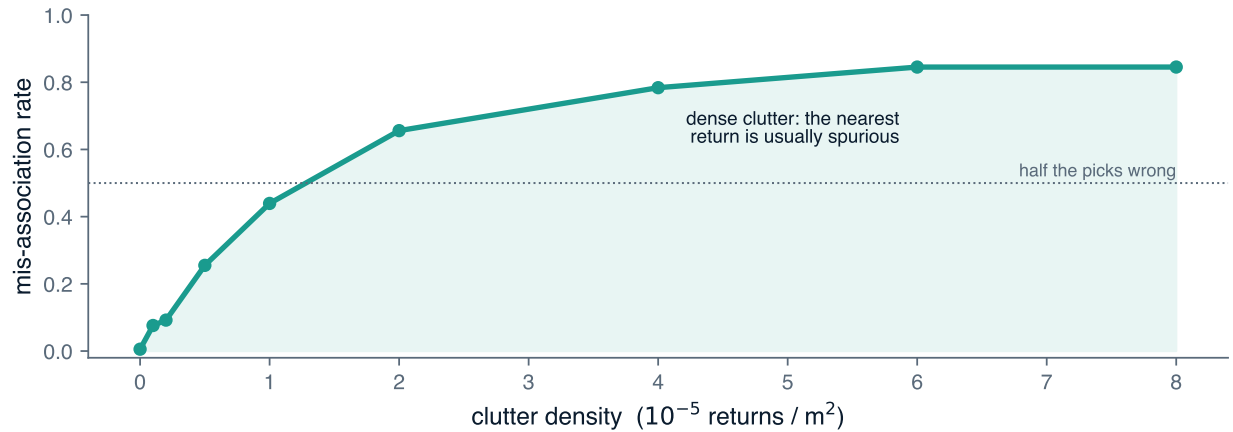


Figure 6.3: Nearest-neighbor mis-association rate as clutter density rises: with little clutter the tracker almost always picks the true return, but the rate climbs steeply as clutter fills the gate, until in dense clutter the nearest return is usually spurious and the single-target rule fails.

6.5 Assignment framing: association as a cost

Nearest-neighbor is worth restating as an optimization, because that is the form that generalizes. Give each gated return a cost and pick the return of least cost; the natural cost is the negative log-likelihood of the association,

$$c_j = \frac{1}{2}d^2(\mathbf{z}^j) + \frac{1}{2}\ln|2\pi\mathbf{S}_k|, \quad (6.6)$$

whose data-dependent part is just the Mahalanobis distance, so minimizing cost and running nearest-neighbor are the same thing for one target. The point of the reframing is what happens with more than one target. With N tracks and M returns the costs form an $N \times M$ matrix, and association becomes the problem of choosing at most one return per track and at most one track per return so as to minimize the total cost, a two-dimensional assignment problem solved optimally by the Hungarian or auction algorithms rather than by greedy nearest-neighbor. And because committing to a single hard assignment throws away information, the probabilistic methods, probabilistic data association (PDA) and its joint form (JPDA), and multiple-hypothesis tracking (MHT), keep a weighted set of associations instead of one. Those are the subject of Module 8; single-target association is the foundation they all rest on.

The origins of gating

Gating and correlation logic are among the oldest ideas in automated tracking, developed through the 1960s and 1970s as radar systems moved from manual plotting to computer tracking and had to decide, scan by scan, which returns continued which tracks. The statistical distance at the center of it is the Mahalanobis distance, introduced by the Indian statistician Prasanta Chandra Mahalanobis in 1936 and reused here as the natural association metric because it coincides with the measurement likelihood. R. W. Sittler's 1964 paper is an early formal treatment of the data-association problem for surveillance tracking. The full lineage of the probabilistic methods that build on this foundation, and the people behind them, is taken up in Module 8, where Bar-Shalom's probabilistic data association and Reid's multiple-hypothesis tracking get their proper treatment.

Key takeaways

Data association is what connects a real, cluttered sensor to the Day 1 filters. The returns in a scan have uncertain origin, so the tracker must decide which one, if any, updates the track. Gating removes the implausible cheaply, using the innovation covariance and a chi-square threshold set from a chosen gate probability. Among the survivors the correct metric is the Mahalanobis distance, which normalizes the innovation by its uncertainty and equals the measurement likelihood exponent, so it ranks returns by how probable they are rather than by raw distance; against an elongated covariance it and the Euclidean rule can pick different returns. The nearest-neighbor rule takes the most likely survivor, which is cheap and optimal for a single target but degrades as clutter density rises and the nearest return is increasingly a false one. Writing the choice as a cost minimization changes nothing for one target but is exactly the form that scales to the multi-target assignment and probabilistic methods of Module 8. Module 7 first turns to the logic that starts and ends tracks, and to the range-rate measurement that sharpens them.

Exercise 6: Gating and Nearest-Neighbor in Clutter

Exercise 6: Gating and Nearest-Neighbor in Clutter (25 min)

No coding required. Start the companion app from the project root:

```
conda activate radar
streamlit run exercises/ex06_association.py
```

The app runs a constant-velocity target embedded in Poisson clutter, with an ellipsoidal gate around the prediction and nearest-neighbor association. You control the clutter density, the detection probability, the gate probability, and the gate shape, and you see the gate, the returns, and which one is associated, along with the running mis-association rate.

Work through these steps and note what you see:

1. Start with low clutter. Confirm nearest-neighbor almost always picks the true return,

and watch the track stay locked on.

2. Raise the clutter density in steps and record the mis-association rate at each. Where does it start climbing steeply?
3. Lower the detection probability P_D . How does missing the target more often change the mis-association rate at a fixed clutter density?
4. Widen the gate by raising the gate probability P_G . Does a bigger gate help or hurt in clutter, and why?
5. Switch the gate from ellipsoidal to rectangular. Does the mis-association rate change much, and what does that tell you about where the extra clutter the rectangle admits actually lands?

Deliverable: the mis-association-rate-versus-clutter-density curve, with a short paragraph explaining why the single-target nearest-neighbor rule breaks down as clutter density rises, and what a multi-target tracker would need to do instead.

Multi-Target Tracking Training Workshop

Module 7 · Track Management & Range-Rate (Doppler)

2026-09-17

Table of contents

7	Track Management & Range-Rate (Doppler)	3
	Learning objectives	3
	Where Module 6 left us	3
7.1	The track lifecycle	3
	The lifecycle as a state machine	4
7.2	M-of-N initiation logic	4
	Tuning the confirmation rule	5
7.3	Sequential probability ratio test (SPRT)	5
7.4	Track deletion	6
7.5	Measured range-rate (Doppler)	6
	Range-rate geometry	6
7.6	Range-rate in the filter	6
	The gain from range-rate	7
	The person behind it	7
	Key takeaways	8
	Exercise 7: Track Management and Range-Rate	9

Module 7: Track Management & Range-Rate (Doppler)

Student Guide

Learning objectives

Module 6 solved association for a single target on one scan: gate the returns, score them by Mahalanobis distance, and pick one. This module covers the two processing layers that turn that per-scan decision into a tracker that lives over time. The first is track management: the logic that decides when a sequence of detections is real enough to start a track, and when a run of misses means the track is dead. The second is the use of measured range-rate, the Doppler channel that most modern radars deliver alongside range and angle, to sharpen the estimate. The two are paired because Doppler feeds directly back into gating and confirmation timing: a good radial-velocity estimate tightens the gate and shortens the time to a confident decision.

Where Module 6 left us

The association machinery of Module 6 assumed a track already existed and asked which return should update it. Two questions were left open. When an unassociated return appears, it might be the first detection of a new target or it might be clutter; committing a confirmed track to every such return would flood the display with false tracks, while ignoring them would miss real targets. And when a track's gate comes up empty scan after scan, the track must eventually be declared dead and removed. Both are sequential decision problems on a stream of detections and misses, and both are what track management solves.

7.1 The track lifecycle

A real tracker keeps every candidate in one of three states. A tentative track is a hypothesis, born the moment an unassociated return appears; it is carried and updated but not shown to the operator. If subsequent scans keep feeding it detections, the confirmation logic promotes it to a

confirmed track, the kind that drives a display symbol and a fire-control or air-traffic decision. If instead its gate keeps coming up empty, the deletion logic eventually moves it to deleted and frees the resources. The whole design trades two error costs against each other. Set the confirmation bar too high and real targets take too long to confirm or never do, a missed track; set it too low and random clutter sequences confirm as false tracks that clutter the picture and waste attention. Every rule that follows, M-of-N, the SPRT, and the deletion criteria, is a way to place and tune that trade.

The lifecycle as a state machine

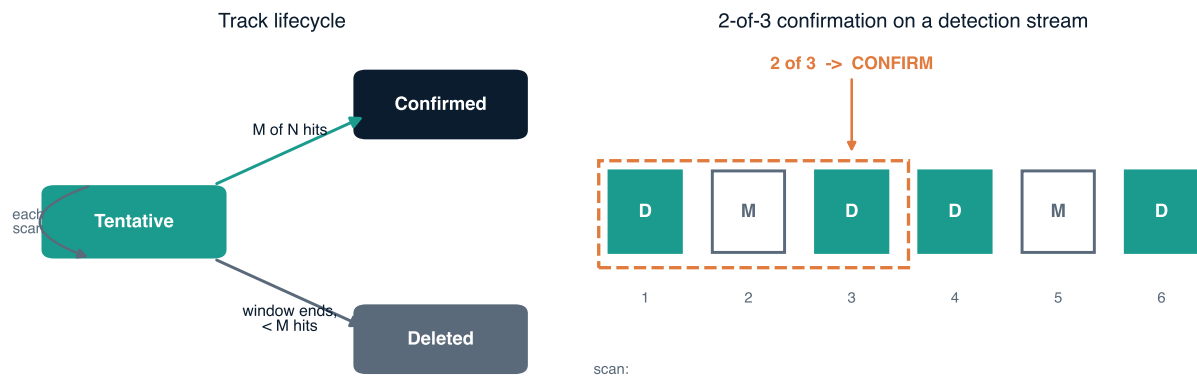


Figure 7.1: The track lifecycle and an M-of-N = 2-of-3 confirmation rule. A tentative track is updated each scan; it is promoted to confirmed once M of the last N scans produced a gated detection, and deleted if the window closes with fewer than M. The example stream D, M, D reaches two detections in three scans and confirms.

7.2 M-of-N initiation logic

The simplest confirmation rule is the sliding window. After an initiating detection, the tentative track is confirmed as soon as it accumulates M gated detections within a window of the next N scans. Modelling each scan as an independent Bernoulli trial with per-scan success probability p , the probability of confirming is the binomial tail

$$P(\text{confirm}) = \sum_{k=M}^N \binom{N}{k} p^k (1-p)^{N-k}.$$

The same formula does double duty. Evaluated at $p = P_D$, the target's per-scan detection probability, it is the probability a real target confirms. Evaluated at $p = P_{FA}$, the probability that clutter falls in the tentative track's gate on a given scan, it is the probability a clutter sequence confirms into a false track. The pair (M, N) is the design knob: raising the bar shrinks the false-track

probability much faster than it shrinks the target-confirmation probability, but it never shrinks only one, which is the trade the next slide plots.

Tuning the confirmation rule

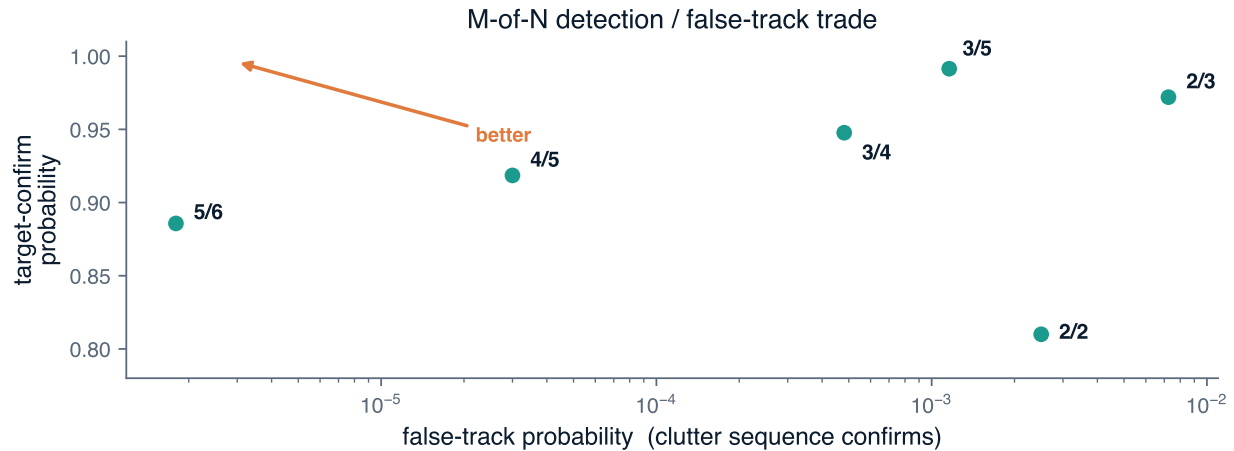


Figure 7.2: The M-of-N detection / false-track trade, at target detection probability $P_D = 0.9$ and clutter-in-gate probability $P_{FA} = 0.05$. Each rule is one point; the ideal is the upper-left corner (confirm every target, admit no false tracks). A 2-of-3 rule confirms almost every target but admits the most false tracks; a 4-of-5 or 5-of-6 rule crushes false tracks at the cost of a few real targets; 3-of-5 is a common compromise.

7.3 Sequential probability ratio test (SPRT)

The M-of-N rule always waits a fixed number of scans. The sequential probability ratio test, due to Abraham Wald, instead accumulates evidence and stops the instant it is decisive, which is faster on average. It maintains a running log-likelihood ratio between two hypotheses, target present (H_1) and clutter only (H_0). Each scan updates it by the log-ratio of the two models' likelihoods: a gated detection adds $\ln(P_D/P_{FA})$, a miss adds $\ln((1 - P_D)/(1 - P_{FA}))$. The score is compared to two thresholds. If $L_k \geq A = \ln \frac{1-\beta}{\alpha}$ the track is confirmed; if $L_k \leq B = \ln \frac{\beta}{1-\alpha}$ it is deleted; between the two, the test takes another scan. Here α is the tolerated probability of confirming clutter and β the tolerated probability of deleting a real target, so the thresholds are set directly from the error budget rather than by trial and error. Wald and Wolfowitz proved the test optimal: among all tests meeting the same two error probabilities, the SPRT minimizes the expected number of scans to a decision. With $P_D = 0.9$, $P_{FA} = 0.05$, and $\alpha = \beta = 0.05$, the thresholds are $A = +2.94$ and $B = -2.94$, a detection is worth $+2.89$ and a miss -2.25 , so two clean detections confirm; the average sample number is well below the five scans a fixed 3-of-5 window always spends.

7.4 Track deletion

Ending a track is the same sequential decision run the other way. When a confirmed track's gate is empty on a scan, the filter cannot update, so it coasts: it applies the prediction step only, and the covariance $\mathbf{P}$ grows because process noise is added with no measurement to pull it back. A track cannot coast forever. Three deletion criteria are common and often combined. The simplest is a missed-update count: delete after K consecutive coasts, the mirror image of the M-of-N confirmation rule. The principled version is the SPRT's lower threshold: the same accumulated score that confirmed the track, once it falls to B , deletes it. The third is covariance-based: because a coasting track's position uncertainty and therefore its gate grow every scan, the track eventually becomes useless even before the count expires, so it is deleted when the position standard deviation or the gate volume exceeds a set limit. As with confirmation, the tuning trades a fading target lost too early against stale tracks left cluttering the display too long.

7.5 Measured range-rate (Doppler)

Modern radars are largely pulse-Doppler or FMCW, which means each detection carries a measured range-rate in addition to range and angle. Range-rate is the rate of change of range, and geometrically it is the target's velocity projected onto the line of sight. For a sensor at the origin and a target with state $[x, \dot{x}, y, \dot{y}]$,

$$\dot{r} = \frac{\mathbf{r} \cdot \mathbf{v}}{\|\mathbf{r}\|} = \frac{x\dot{x} + y\dot{y}}{\sqrt{x^2 + y^2}},$$

and it is what produces the Doppler frequency shift $f_d = -2\dot{r}/\lambda$ that the signal processor actually estimates. Like the bearing measurement of Modules 3 and 4, range-rate is a nonlinear function of a Cartesian state: it mixes position and velocity and divides by range. So it cannot be handled by the linear Kalman update, and the same extended-Kalman machinery applies, a first-order linearization about the current estimate through the measurement Jacobian.

Range-rate geometry

7.6 Range-rate in the filter

There are three levels at which range-rate is used. If the tracker works in a one dimensional range / range-rate frame, along a fixed line of sight, the measurement is simply the velocity state itself, which is linear, so a constant-gain α - β filter or a small two-state Kalman filter suffices; this is common in the range gate of a monopulse tracker. When the geometry is only locally curved, a Kalman filter on a linearized model works. The general case, a 2-D or 3-D Cartesian state, needs

Range-rate sees only the along-line-of-sight velocity

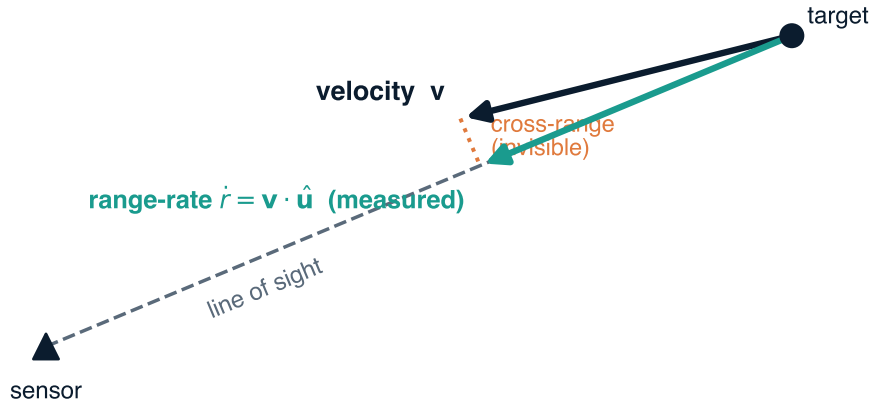


Figure 7.3: Range-rate is the target velocity projected onto the line of sight. Only the along-line-of-sight (radial) component of the velocity produces a Doppler shift and is measured; the cross-range (tangential) component is invisible to the range-rate channel. This is why range-rate constrains the radial velocity and says nothing about the tangential velocity.

the extended Kalman filter, with the range-rate measurement Jacobian

$$\mathbf{H}_{\dot{r}} = \begin{bmatrix} \frac{\dot{x} r^2 - (x\dot{x} + y\dot{y})x}{r^3} & \frac{x}{r} & \frac{\dot{y} r^2 - (x\dot{x} + y\dot{y})y}{r^3} & \frac{y}{r} \end{bmatrix},$$

evaluated at the predicted state, exactly the EKF update pattern from Module 3. The payoff is specific: because range-rate directly observes the velocity component along the line of sight, it sharpens the radial velocity estimate and, through the prediction step, the range. It contributes nothing to the tangential velocity, the across-line-of-sight motion the Doppler channel cannot see. The next slide quantifies both halves of that statement.

The gain from range-rate

The person behind it

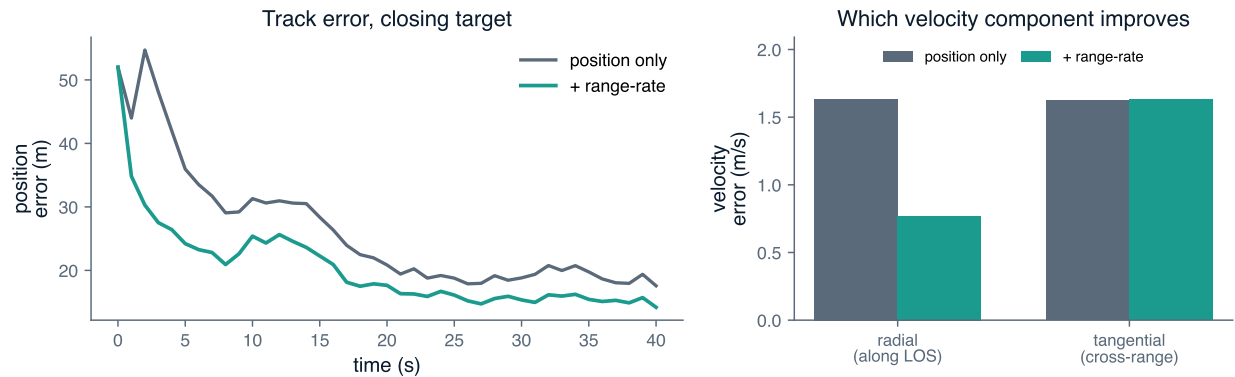


Figure 7.4: Track error with and without the range-rate measurement, over a 200-run Monte-Carlo on a closing target (position noise 40 m, range-rate noise 2 m/s). Left: adding range-rate lowers the position error throughout the run. Right: the velocity error split into radial and tangential components shows why, the radial (along line-of-sight) velocity error falls by about half, while the tangential (cross-range) velocity error is unchanged, because range-rate observes only the radial component.



THE PERSON BEHIND IT

Abraham Wald (1902-1950)

Romanian-born mathematician who developed the sequential probability ratio test at Columbia's Statistical Research Group during World War II, and proved (with Jacob Wolfowitz) that among all tests meeting given error probabilities the SPRT minimizes the expected number of observations. That optimality is why score-based track confirmation stops as soon as the evidence is decisive rather than waiting out a fixed window. Wald also essentially founded statistical decision theory (loss, risk, and minimax rules), lent his name to the Wald test, and is popularly known for the survivorship-bias insight on where to armor returning aircraft.

Key takeaways

Track management is the sequential-decision layer that wraps the per-scan association of Module 6. A track passes through tentative, confirmed, and deleted states, and the whole design is the trade between missed tracks and false tracks. M-of-N confirmation places that trade with a binomial sliding window, where tightening the rule cuts the false-track probability faster than it cuts target confirmation. The SPRT replaces the fixed window with a running likelihood-ratio score and two thresholds set from the error budget, deciding as soon as the evidence is decisive and, by Wald's theorem, using the fewest scans on average. Deletion is the same logic in reverse, through coasting, a missed-update count, the SPRT's lower threshold, or a covariance limit. The Doppler half of the module adds the measured range-rate, the velocity projected onto the line of sight; it is nonlinear in the Cartesian state and enters through the same EKF machinery as bearing, and it sharpens the radial velocity estimate by roughly half while leaving the tangential velocity, which it cannot

observe, untouched.

Exercise 7: Track Management and Range-Rate

Exercise 7: Track Management and Range-Rate (two parts, 15 min each)

No coding required. Start the companion app from the project root:

```
conda activate radar
streamlit run exercises/ex07_track_mgmt.py
```

Use the selector at the top to switch between the two parts.

Part A - M-of-N initiation (15 min). The app evaluates the detection / false-track trade for a family of (M, N) rules at a chosen target detection probability P_D and clutter-in-gate probability P_{FA} , and animates a detection stream walking the confirmation logic.

1. Set a false-track budget (say 10^{-3} per tentative track) and a confirmation floor (say confirm $\geq 95\%$ of real targets). Find the (M, N) rules that meet both.
2. Lower P_D . Which rules still clear the confirmation floor, and what does that cost in false-track rate?
3. Raise the clutter level P_{FA} . How must the rule tighten to hold the same false-track budget?

Part B - range-rate in the EKF (15 min). The app runs an EKF on a target, with the range-rate measurement switchable on or off, and reports position and velocity error split into radial and tangential components.

1. With range-rate off, note the position and velocity RMSE.
2. Switch range-rate on. By how much does the position error drop? The velocity error?
3. Look at the radial vs tangential velocity error. Which one improves, and which is unchanged? Explain the result from the measurement geometry.
4. Change the range-rate noise. How good does the Doppler measurement have to be before it stops helping?

Deliverables: (a) the detection / false-track trade-off plot with the chosen (M, N) marked and a sentence justifying it against the budget; (b) the before/after error plot with a short paragraph on why range-rate improves the radial velocity and not the tangential.

Multi-Target Tracking Training Workshop

Module 8 · Multi-Target Association: NN, Assignment, PDA & MHT

2026-09-17

Table of contents

8	Multi-Target Association: NN, Assignment, PDA & MHT	3
	Learning objectives	3
	Where Module 6 left us	3
8.1	The multi-target association problem	3
8.2	Global nearest neighbor and the assignment problem	4
	Optimal vs greedy assignment	4
8.3	Probabilistic Data Association (PDA)	5
	PDA vs GNN in clutter	5
8.4	Joint PDA (JPDA)	5
8.5	PDA/JPDA implementation issues	6
	Track coalescence	6
8.6	Multiple Hypothesis Tracking (MHT)	6
	An MHT hypothesis tree	7
8.7	Keeping MHT tractable	7
	The people behind it	7
	Key takeaways	8
	Exercise 8: Assignment and Soft Association	8

Module 8: Multi-Target Association: NN, Assignment, PDA & MHT

Student Guide

Learning objectives

Module 6 solved association for one target on one scan, and showed nearest-neighbor breaking down as clutter fills the gate. This module is the escalation ladder that answer builds: from global nearest-neighbor, up through the optimal assignment problem, to probabilistic association (PDA and JPDA), to hypothesis-based tracking (MHT). Each rung exists because the previous one fails in a specific way, and seeing those failures is the point. It is the heaviest module in the course, and everything rests on Day 1's innovation covariance and the M6 gate and Mahalanobis metric.

Where Module 6 left us

The single-target rule of Module 6 gated the returns around one prediction and chose the nearest. With several targets the problem changes in kind, not just in size. A single return can fall in the gates of two or more tracks, and two tracks can compete for the same return, so the choices are coupled: assigning a measurement to one track removes it from another. The association must therefore be made jointly across all tracks and measurements on the scan, which is where the assignment problem, and everything past it, comes from.

8.1 The multi-target association problem

On a single scan there are n tracks and m gated returns, and an assignment must respect two exclusivity constraints: each return updates at most one track, and each track is updated by at most one return (a missed detection and a false alarm are the “none” options). A choice satisfying both is a feasible joint association event. The number of such events grows super-exponentially with n and m ; counting the weighted events is evaluating a matrix permanent, which, unlike the determinant, has no efficient exact algorithm. So in anything but a sparse scene the events cannot

all be enumerated, and every method in this module is a different way of coping: pick the single best event (GNN), average over events per target (PDA/JPDA), or defer across scans and prune (MHT).

8.2 Global nearest neighbor and the assignment problem

Global nearest neighbor makes one hard decision per scan, but the best one across all tracks at once. Score each track-measurement pair by the negative log-likelihood of that pairing, which is the Module 6 Mahalanobis term plus the Gaussian normalizer and a detection-probability offset,

$$c_{ij} = \frac{1}{2} \nu_{ij}^\top \mathbf{S}_i^{-1} \nu_{ij} + \frac{1}{2} \ln |2\pi \mathbf{S}_i| - \ln P_D,$$

and choose the one-to-one assignment $\{a_{ij}\}$ that minimizes the total cost $\sum_{ij} a_{ij} c_{ij}$. This is the classical assignment problem, solved exactly in $O(n^3)$ by the Munkres (Hungarian) algorithm, or by the auction algorithm, which scales better on large sparse problems. The tempting shortcut, greedy nearest, takes the globally cheapest cell and repeats; it is fast but suboptimal, because a cheap early pick can force an expensive one later, as the next slide shows.

Optimal vs greedy assignment

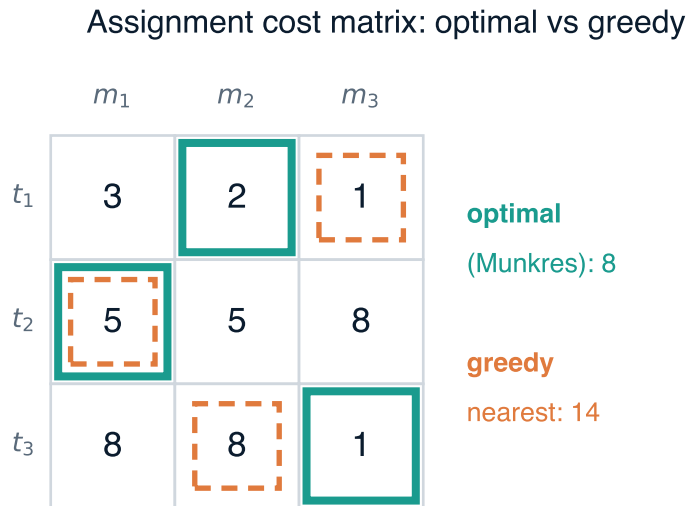


Figure 8.1: A measurement-to-track cost matrix with the optimal (Munkres) assignment and the greedy nearest pick highlighted. Greedy takes the globally cheapest cell first and is forced into a total cost of 14, where the optimal assignment costs 8. Across random matrices greedy averages about 18% above optimal.

8.3 Probabilistic Data Association (PDA)

Probabilistic Data Association keeps the single-target setting but refuses to commit. Instead of picking one gated return, it forms the probability β_i that each gated return is the target, from the return's Gaussian likelihood weighted by P_D , with a further weight β_0 for the event that none of them is the target (the clutter density λ and $1 - P_D$ set that one). The filter then updates with the probability-weighted combined innovation $\nu = \sum_i \beta_i \nu_i$, and inflates the covariance by a spread-of-innovations term that accounts for the association uncertainty. The effect is that no single false return can capture the track: a spurious return in the gate is down-weighted by its low likelihood, so in clutter PDA holds the target where a hard nearest-neighbor pick would occasionally jump onto a false return and lose it.

PDA vs GNN in clutter



Figure 8.2: One target in clutter tracked by a hard GNN pick versus a soft PDA update, over a 120-run Monte-Carlo. Left: the mean position error grows faster for GNN as it occasionally locks onto a false return. Right: GNN loses the track in about 40% of runs, PDA in about 23%. Soft association roughly halves the loss rate at this clutter level.

8.4 Joint PDA (JPDA)

PDA computes each target's weights as if it were the only target in the scene, which is wrong precisely when it matters, when two tracks' gates overlap and compete for the same returns. Joint PDA fixes the bookkeeping. It enumerates the feasible joint events over all targets at once (each return to at most one target, each target to at most one return), computes each event's probability from the per-pairing likelihoods, and then marginalizes to get the per-target-per-measurement weight β_{ij} , the total probability of all joint events that assign return j to target i . Because the enumeration enforces exclusivity, a return that falls between two tracks is split between them rather than fully claimed by both, so JPDA weights are the PDA idea done jointly and consistently across the overlapping tracks.

8.5 PDA/JPDA implementation issues

Two practical costs come with soft association. The first is track coalescence: when targets are close, the shared returns pull the estimates toward one another. Running PDA independently per target is the worst case, since both filters weight the same central returns and drift onto the centreline; JPDA's joint constraint reduces this but does not eliminate it, which is why coalescence-avoiding variants such as JPDA* exist. The second cost is computation: the number of feasible joint events is exponential in the number of overlapping tracks and returns, so full enumeration is infeasible in dense scenes. The standard mitigation is clustering: use the cheap gate to partition tracks and returns into clusters of mutually overlapping objects, and enumerate events only within each cluster, since tracks in different clusters are independent problems.

Track coalescence

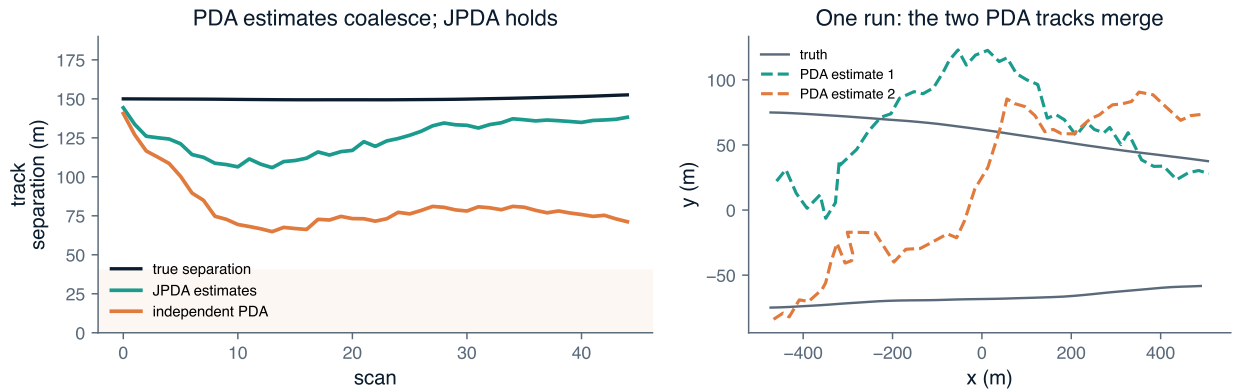


Figure 8.3: Two closely-spaced parallel targets (true separation about 150 m) in clutter. Left: the estimated track separation over the run. Independent PDA collapses toward zero, the two estimates coalescing onto the centreline, while JPDA's joint weighting holds them near the true separation. Right: one run's tracks, showing the two PDA estimates merging in the middle.

8.6 Multiple Hypothesis Tracking (MHT)

PDA and JPDA still collapse the ambiguity every scan into a single weighted update. If the blend was wrong, because two interpretations were genuinely indistinguishable on that scan, there is no way to revisit it once the estimate has moved on. Multiple Hypothesis Tracking takes the opposite stance: defer the decision. It maintains a tree of association hypotheses, each hypothesis being one consistent history of measurement-to-track assignments over the recent scans, and scores each by its accumulated likelihood. A one-scan ambiguity that a filter would have to guess is simply carried as two branches until a later scan makes one far more likely, at which point the other is pruned. Two formulations are standard: hypothesis-oriented MHT enumerates global hypotheses over all

tracks, while track-oriented MHT maintains a tree of track hypotheses and forms global hypotheses only when a decision is needed, which is the more scalable and more common modern form.

An MHT hypothesis tree

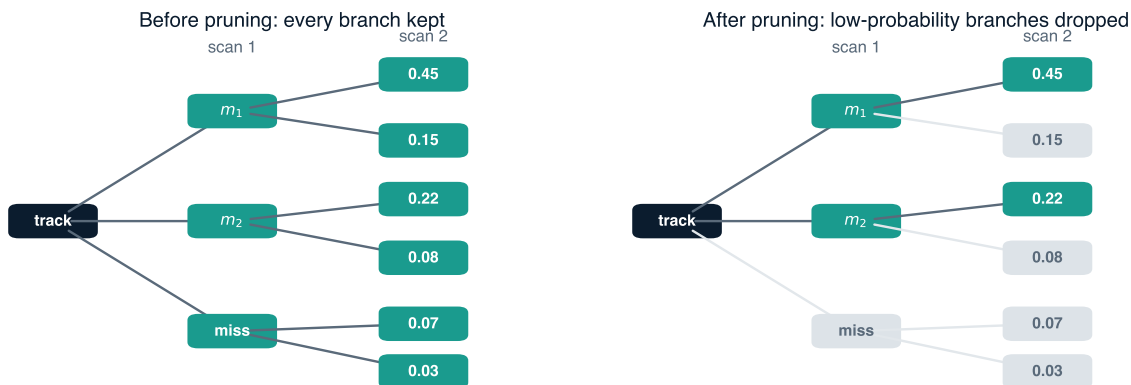


Figure 8.4: A multiple-hypothesis tree over two scans. Each scan branches the track on every feasible association (a measurement or a miss), and each branch carries its accumulated-likelihood score. Left: the full tree. Right: after pruning, only the high-probability branches are kept, which is what stops the tree from exploding.

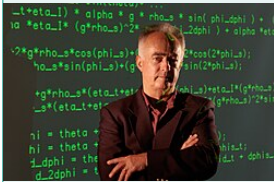
8.7 Keeping MHT tractable

Deferring decisions is only useful if the hypothesis tree stays bounded, and four controls keep it so. Pruning drops any hypothesis whose probability falls below a threshold, or keeps only the top k . Merging combines hypotheses that have become effectively identical, for instance two histories that differ only in an old assignment and now share the same track states. N-scan pruning fixes the association decisions older than N scans back, committing the past while keeping the recent window open, which bounds the tree depth. And gating, the same Module 6 test, keeps the tree from ever branching on a measurement that could not plausibly belong to the track. With these, MHT is not a combinatorial thought experiment but the basis of many fielded multi-target trackers.

The people behind it

THE PERSON BEHIND IT

Yaakov Bar-Shalom (b. 1941)



Israeli-American engineer at the University of Connecticut and the central figure in modern data association. He originated Probabilistic Data Association and, with Fortmann and Tse, Joint PDA, and his books (with Fortmann, Li, and Kirubarajan) are the standard references for estimation, tracking, and association. He also co-developed the IMM estimator of Module 5. Multiple Hypothesis Tracking is due to Donald B. Reid, whose 1979 paper "An Algorithm for Tracking Multiple Targets" introduced the deferred- decision hypothesis-tree formulation that MHT trackers still use.

Key takeaways

Multi-target association is a joint assignment problem: the choices are coupled across tracks, and the number of feasible joint events grows like a matrix permanent, so exact enumeration is hopeless in dense scenes. Global nearest neighbor makes the single best hard decision each scan by solving the assignment optimally with Munkres or auction, which a greedy pick cannot match. PDA keeps a single track but softens the decision, weighting every gated return by its association probability, and roughly halves the track-loss rate in clutter relative to a hard pick. JPDA carries that soft weighting out jointly across overlapping tracks so a shared return is split rather than double-counted. Both soft methods bring track coalescence and the cost of feasible- event enumeration, handled by clustering and by coalescence-avoiding variants. MHT takes the last step, deferring the association across scans as a scored hypothesis tree, and pruning, merging, N-scan pruning, and gating are what keep that tree, and the computation, bounded.

Exercise 8: Assignment and Soft Association

Exercise 8: Assignment and Soft Association (two parts, 15 min each)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex08_multitarget.py
```

Use the selector at the top to switch between the two parts.

Part A - assignment vs greedy (15 min). The app builds a measurement-to-track cost matrix from a small multi-target scene, solves the optimal assignment (`scipy.optimize.linear_sum_assignment`), and shows it against greedy nearest-neighbor.

1. Compare the optimal total cost to the greedy total. When are they equal, and when does greedy pay a penalty?
2. Increase the scene's ambiguity (closer, overlapping tracks). Does the greedy penalty grow?

3. Find a matrix where greedy's first (cheapest) pick is exactly the one that forces the expensive swap.

Part B - PDA vs GNN continuity (15 min). The app runs crossing targets in clutter, tracked by GNN or PDA, and reports track error and loss.

1. With GNN, raise the clutter density until tracks start to be lost.
2. Switch to PDA at the same clutter. How much does the loss rate drop?
3. Lower the detection probability P_D . Which method degrades faster, and why?
4. Bring the targets closer together. Watch for coalescence, and note what a joint (JPDA) method would add here.

Deliverables: (a) the assignment comparison (optimal vs greedy cost and associations) with a sentence on when greedy's shortcut costs a swap; (b) the track-continuity / error plot with a short paragraph on why soft association survives clutter that breaks the hard pick.

Multi-Target Tracking Training Workshop

Module 9 · Coordinate Conversions & Sensor Motion Compensation

2026-09-17

Table of contents

9	Coordinate Conversions & Sensor Motion Compensation	3
	Learning objectives	3
	Where Module 8 left us	3
9.1	Why conversion matters for fusion	3
9.2	From a sensor's frame to a global frame	4
	The RUV to ENU to ECEF chain	4
9.3	The conversion-bias problem	4
9.4	Debiased, consistent converted measurements	5
	Bias and consistency, measured	5
9.5	Sensor motion compensation	6
	Uncompensated vs compensated	6
	Converted-measurement filtering: the origin	6
	Key takeaways	7
	Exercise 9: Debiased Converted Measurements	7

Module 9: Coordinate Conversions & Sensor Motion Compensation

Student Guide

Learning objectives

Day 1 and Day 2 tracked with a single sensor, mostly in that sensor's own native frame. Day 3 is about combining sensors, and the first requirement is mechanical but easy to get wrong: every sensor's data must be expressed in one common frame, with uncertainties that are honest about what the conversion did to them. This module is the bridge. It has little new estimation theory and a lot of getting the geometry and the statistics of the frame change right, because a bias or a mis-stated covariance introduced here propagates straight into every fused track that follows.

Where Module 8 left us

Modules 6 through 8 built the association and multi-target machinery, but always for a single sensor's stream of measurements. Day 3 combines multiple sensors: fusing their tracks, aligning them in space and time, and bounding the achievable performance. None of that is meaningful until the sensors' data share a common frame, so this module comes first. And the step is not just bookkeeping: the polar-to-Cartesian conversion at the heart of it is nonlinear, so done naively it biases both the estimate and its stated uncertainty, which is the problem the module solves.

9.1 Why conversion matters for fusion

A single-sensor tracker can happily work in the sensor's own measurement frame, but the moment two sensors are to be combined, their measurements must be expressed in a common coordinate frame so that "the same place" means the same numbers to both. Fusion needs more than a common frame, though: it weights each sensor's contribution by its stated covariance, so those covariances must be consistent, neither optimistic nor pessimistic about the true error. A converted measurement that is biased, or that reports a covariance smaller than its real spread, will be over-

weighted in the fusion and will pull the combined track off the target. Getting the conversion and its covariance right is therefore the precondition for everything on Day 3.

9.2 From a sensor's frame to a global frame

The conversion is a chain. A sensor's native measurement is polar, or RUV (range, azimuth, elevation, or range and two direction cosines). The first step converts it to the sensor's local East-North-Up Cartesian frame by the spherical-to-Cartesian relations $E = r \cos el \sin az$, $N = r \cos el \cos az$, $U = r \sin el$. The second step carries ENU to a global earth-centered earth-fixed (ECEF) frame by a rotation set by the sensor's geodetic latitude and longitude, followed by a translation to the sensor's own ECEF position, $\mathbf{p}_{\text{ECEF}} = \mathbf{p}_0 + \mathbf{R}(\text{lat}, \text{lon}) \mathbf{p}_{\text{ENU}}$. Every sensor runs its own chain with its own rotation and translation, and the result is that all of their data live in one common global frame where they can be compared and fused. (We use a spherical earth here; a precise system uses the WGS-84 ellipsoid, which changes the lat/lon-to-ECEF relation but not the structure of the chain.)

The RUV to ENU to ECEF chain

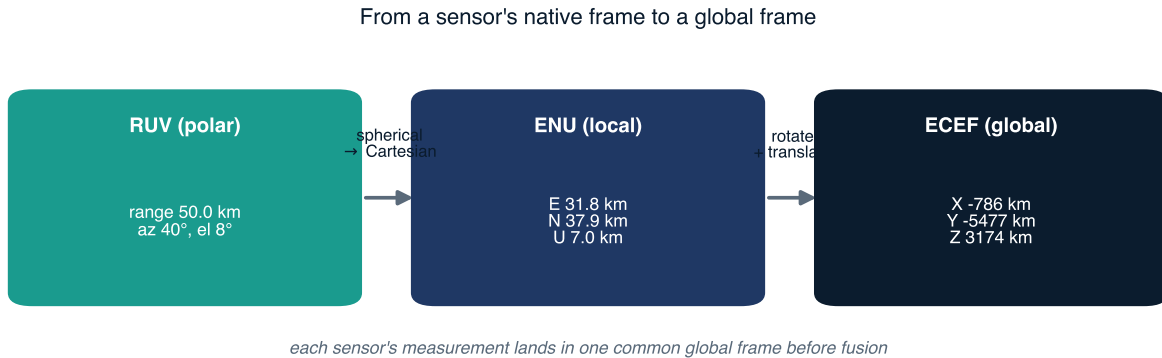


Figure 9.1: One target measurement carried through the transformation chain: from the sensor's native polar/RUV frame, to the local East-North-Up Cartesian frame by the spherical-to-Cartesian conversion, to the global earth-centered earth-fixed frame by a rotation set by the sensor's latitude and longitude and a translation to its position. Every sensor's chain lands its data in the same global frame before fusion.

9.3 The conversion-bias problem

The trouble is that the conversion is nonlinear, so an unbiased polar measurement does not give an unbiased Cartesian one. Averaging over the bearing noise, the converted coordinate is pulled toward the sensor: $E[r \cos \theta] = e^{-\sigma_\theta^2/2} r_t \cos \theta_t$, a multiplicative bias that shrinks the range component. On top of the mean bias, the covariance obtained by linearizing the conversion (the Jacobian mapping,

which is what an EKF uses) is inconsistent: it does not match the true spread of the converted point, and in practice it is over-confident, reporting a smaller error than the conversion actually has. Both effects grow with the angular error σ_θ and with range, since the cross-range uncertainty is $r \sigma_\theta$. An over-confident, biased measurement is exactly what corrupts a fused filter, so the conversion must be corrected before it is used.

9.4 Debiased, consistent converted measurements

The fix has two parts. First, undo the mean bias by multiplying the naive conversion by $e^{\sigma_\theta^2/2}$, the multiplicative unbiased converted measurement $\mathbf{z}_u = e^{\sigma_\theta^2/2} [r \cos \theta, r \sin \theta]^\top$, whose expectation is the true Cartesian position. Second, replace the linearized covariance with the exact covariance of that converted measurement, the closed-form terms derived by Lerro and Bar-Shalom, which correctly account for the nonlinearity and are functions of $r, \theta, \sigma_r, \sigma_\theta$. With a debiased measurement and a consistent covariance in hand, the target can be tracked by an ordinary linear Kalman filter in Cartesian coordinates, which is the converted-measurement Kalman filter. The payoff is consistency: the normalized error of the debiased conversion matches its stated covariance (a NEES of 2 for a two-dimensional measurement) at all angular-error levels, where the naive conversion becomes badly over-confident, as the next slide quantifies.

Bias and consistency, measured

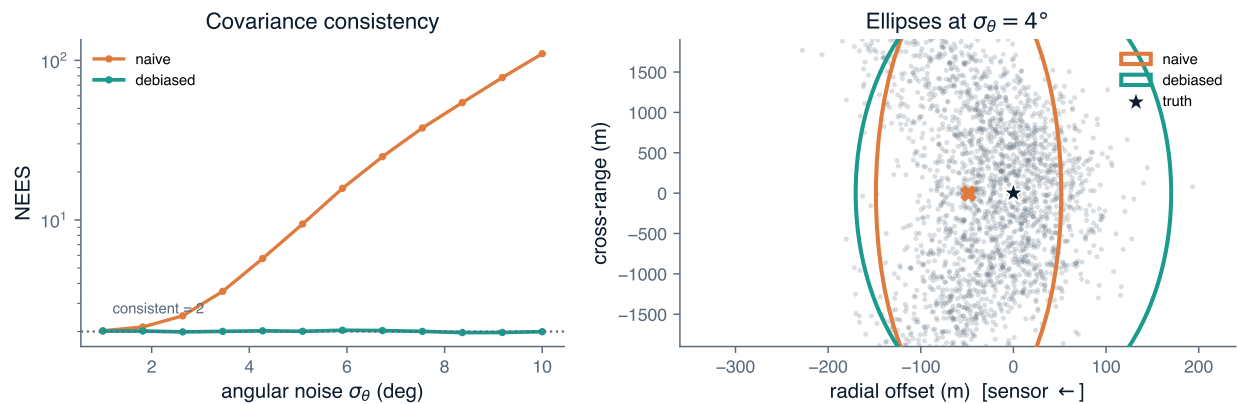


Figure 9.2: Left: the converted-measurement consistency (NEES, which should equal 2 for a two-dimensional measurement) versus angular noise, at 20 km range. The naive Jacobian covariance becomes badly over-confident, its NEES climbing past 100, while the debiased conversion holds NEES at 2. Right: the covariance ellipses at a moderate angular error, in the local radial and cross-range frame. The naive mean is pulled toward the sensor (a range-direction bias) and its ellipse is off-center; the debiased ellipse is centered on the truth and consistent.

9.5 Sensor motion compensation

The final complication is that real sensors are rarely fixed at a known origin: they sit on ships, aircraft, and vehicles that move and rotate. A measurement is taken in the platform's frame at the instant of the measurement, so converting it to the global frame requires the platform's position and attitude at that time, $\mathbf{p}_{\text{global}} = \mathbf{p}_{\text{plat}}(t) + \mathbf{R}(\text{att}(t)) \mathbf{p}_{\text{local}}$. Using a stale or nominal platform state, or ignoring the platform motion entirely, injects a bias that looks like target motion. Combining measurements taken at different times adds a second requirement, time-alignment: each measurement must be predicted to a common reference time before it is fused, or the platform's and target's motion between the measurement times will be misattributed. The next slide shows what happens when the platform motion is simply ignored: the reconstructed track drifts by the platform's own displacement.

Uncompensated vs compensated

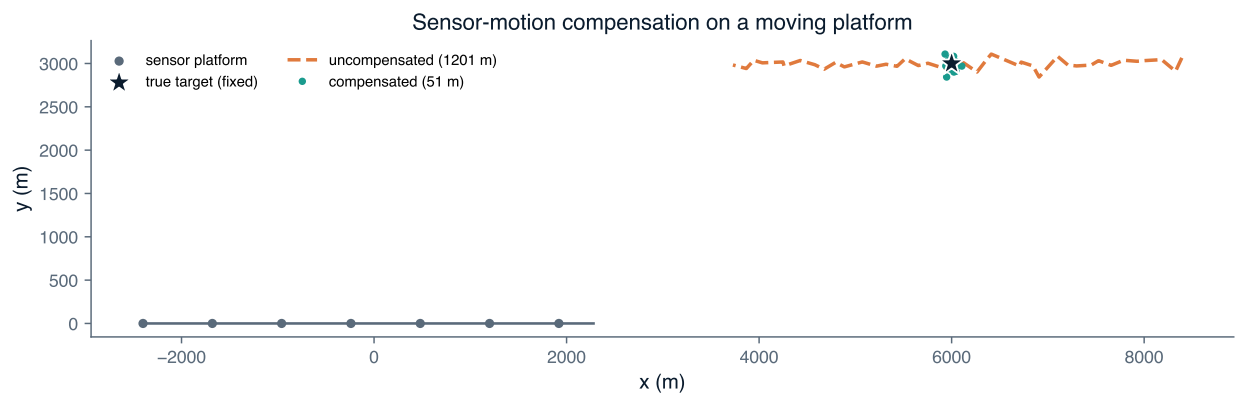


Figure 9.3: A moving sensor platform measures a fixed target. Ignoring the platform's motion, the reconstructed track drifts across the scene by the platform's own displacement (a large motion-induced error). Compensating, by adding the platform's position at each measurement time, recovers the true fixed target to within the measurement noise.

Converted-measurement filtering: the origin

The debiased converted measurement and its consistent covariance were put on a firm footing by D. Lerro and Y. Bar-Shalom in their 1993 paper “Tracking with debiased consistent converted measurements versus EKF,” which showed that a linear filter fed the debiased converted measurement and its correct covariance outperforms the extended Kalman filter for radar-to-Cartesian tracking, precisely because it does not lie about its own accuracy. It is the standard approach for converting range/angle measurements into a Cartesian tracker. Bar-Shalom appears yet again here, after the IMM of Module 5 and the data-association work of Module 8; his full biography is on the Module 8 card.

Key takeaways

Fusion begins with geometry: every sensor's data must be carried into one common global frame, with covariances that are consistent, before any combination is meaningful. The frame chain runs from the sensor's native polar/RUV measurement, through the local East-North-Up Cartesian frame by the spherical-to-Cartesian conversion, to the global ECEF frame by a rotation set by the sensor's latitude and longitude and a translation to its position. The catch is that the polar-to-Cartesian conversion is nonlinear: done naively it biases the estimate toward the sensor and reports an over-confident, inconsistent covariance, both growing with angular error and range. The debiased converted measurement removes the bias and its exact covariance restores consistency (NEES of 2), which is the converted-measurement Kalman filter of Lerro and Bar-Shalom. Finally, a moving sensor must be compensated with its position and attitude at each measurement time and its measurements time-aligned to a common reference, or the track inherits a motion-induced bias. With all of this correct, the sensors are ready to be fused, which is where the rest of Day 3 begins.

Exercise 9: Debiased Converted Measurements

Exercise 9: Debiased Converted Measurements (25 min)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex09_conversion.py
```

The app draws a Monte-Carlo cloud of a target's converted position from noisy polar measurements, and overlays the naive and the debiased converted-measurement covariance ellipses, reporting the NEES (normalized estimation-error squared, which should equal 2 for a two-dimensional measurement) for each. You control the range, the angular noise σ_θ , and the range noise σ_r .

Work through these steps and note what you see:

1. Start with a small σ_θ . Both ellipses roughly cover the cloud and both NEES values are near 2. Note the tiny bias.
2. Raise σ_θ . Watch the naive converted mean pull toward the sensor and the naive NEES climb well above 2, while the debiased NEES stays near 2.
3. Increase the range at fixed σ_θ . How does the cross-range spread, and the bias, change?
4. Change the range noise σ_r . Which NEES does it affect, and why is the angular noise the one that drives the inconsistency?

Deliverable: the naive-vs-debiased covariance-ellipse comparison with the two NEES values, and a short paragraph explaining why the naive converted covariance is inconsistent and what the debiasing terms fix.

Multi-Target Tracking Training Workshop

Module 10 · Multi-Sensor Fusion: JDL Model & Architectures

2026-09-17

Table of contents

10 Multi-Sensor Fusion: JDL Model & Architectures	3
Learning objectives	3
Where Module 9 left us	3
10.1 The JDL data fusion model	3
The JDL level stack	4
10.2 Fusion levels in practice	4
10.3 Report-to-track (centralized) fusion	5
10.4 Track-to-track (distributed) fusion	5
The two architectures side by side	5
10.5 The cross-covariance problem	5
10.6 Covariance intersection	6
Naive fusion vs covariance intersection, measured	7
10.7 Choosing an architecture	7
The JDL model and covariance intersection: the origins	7
Key takeaways	8
Exercise 10: Correlated Track Fusion	8

Module 10: Multi-Sensor Fusion: JDL Model & Architectures

Student Guide

Learning objectives

Module 9 delivered the precondition for fusion: every sensor's data carried into one common, consistent frame. This module makes the first genuine combination of sensors, not just measurements from one sensor, and it sets the vocabulary and the architecture for the rest of Day 3. Two things to take away. First, the JDL data fusion model, the field's shared map of what "data fusion" means and where target tracking sits inside it. Second, the single architectural fork that defines every real fusion system: fuse the raw reports at a central node, or let each sensor form its own tracks and fuse those. The distributed choice buys autonomy and bandwidth but opens the correlated-error problem, which is the technical heart of the module and the reason covariance intersection exists.

Where Module 9 left us

Module 9 was the bridge: it put every sensor's measurements into a common global frame with consistent covariances, and it showed that a naive polar-to-Cartesian conversion lies about its own accuracy unless it is debiased. With the frame problem solved, we can finally combine sensors. This module gives that combination its framework (the JDL model) and its two architectures (centralized report fusion and distributed track fusion), and it confronts the problem that makes distributed fusion subtle: the local tracks being fused are not statistically independent, so combining them as if they were produces an over-confident result. The same theme as M9, in a new guise: a fused estimate that lies about its accuracy is the thing that corrupts the system.

10.1 The JDL data fusion model

The Joint Directors of Laboratories (JDL) model is the common vocabulary for data fusion. It organizes fusion into levels of increasing abstraction. Level 0 is signal or sub-object processing,

extracting detections and features from raw sensor data. Level 1, object refinement, is where our work lives: association, tracking, and identification, turning detections into labelled tracks with state and covariance. Level 2, situation assessment, interprets the relationships among those tracks (formations, engagements, patterns of activity). Level 3, impact assessment, reasons about intent, threat, and what happens next. Level 4, process refinement, closes the loop by managing the sensors and tasking, feeding decisions back down to how the system collects data. The value of the model is not doctrine for its own sake: it lets a designer place their own component, a tracker, precisely, and see what it consumes (Level 0 detections in a common frame) and what consumes it (the situation and impact levels above).

The JDL level stack

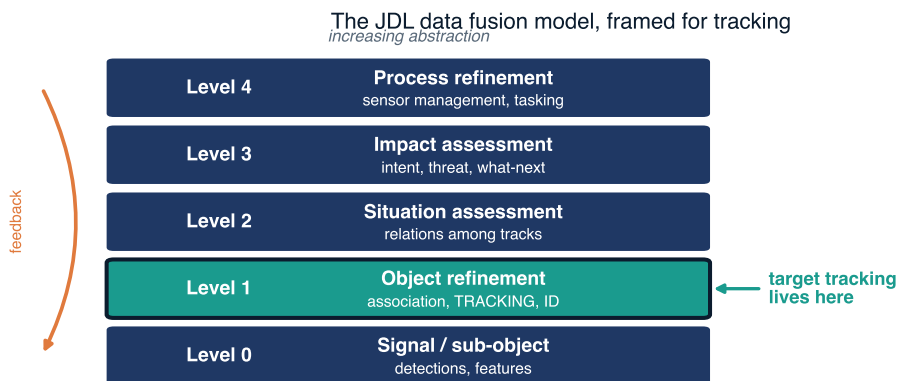


Figure 10.1: The JDL data fusion model as a stack of levels of increasing abstraction, annotated for a target-tracking audience. Level 0 extracts detections and features; Level 1 (object refinement) is where association, tracking, and identification live; Levels 2 and 3 assess the situation and its impact; Level 4 (process refinement) manages and tasks the sensors, the feedback loop back to collection. Target tracking is a Level 1 function that consumes Level 0 detections and feeds the levels above.

10.2 Fusion levels in practice

It helps to translate the higher JDL levels into what they mean for a tracker. Object assessment (Level 1) is the tracks we produce: each target's estimated state, its covariance, and its identity. Situation assessment (Level 2) looks across tracks for relationships, a group moving in formation, one track closing on another, a pattern that means something more than any single track. Impact assessment (Level 3) reasons about consequence: intent, threat, and prediction of what happens next. The practical point for this course is that everything above Level 1 depends on the quality and, crucially, the honesty of the Level-1 tracks. A track whose covariance is over-confident does not just mis-state one target's uncertainty; it corrupts the situation and impact reasoning built on

top of it. That is why the architecture and the correlated-error problem in the rest of this module matter beyond the tracker itself.

10.3 Report-to-track (centralized) fusion

The first architecture is report-to-track, or centralized, fusion. Every sensor sends its raw detections (reports) to a single central node, which runs one tracker on the pooled stream of measurements from all sensors. Because no sensor forms its own tracks, no information is thrown away and no measurement is counted twice: the central tracker sees the full evidence and is, in principle, statistically optimal. That optimality is the appeal. The costs are practical. The communication load is high, since every detection from every sensor must be shipped every scan; the central node is a single point of failure, so losing it loses the whole track picture; and its compute scales with the total detection load across all sensors. Centralized fusion is the right choice when bandwidth and a reliable central resource are available and optimality matters.

10.4 Track-to-track (distributed) fusion

The second architecture is track-to-track, or distributed, fusion. Here each sensor runs its own local tracker and ships finished tracks, a state estimate and its covariance, to a fusion node that combines them into a global track. The advantages are the mirror image of centralized fusion's costs. Bandwidth is low, because tracks update far more slowly than raw detections arrive; the system is autonomous and degrades gracefully, since each sensor keeps tracking even if the fusion node or another sensor is lost; and dissimilar sensors integrate cleanly, because each contributes a track in the common frame rather than raw measurements in its own units. The catch, and it is the technical heart of the module, is that the local tracks are not statistically independent. They observe the same target, driven by the same motion, often started from a shared prior, so their errors are correlated. Combining them as if they were independent, the obvious thing to do, is wrong, and the next section shows exactly how it fails.

The two architectures side by side

10.5 The cross-covariance problem

Two local tracks of the same target are correlated because they share error sources. The same target motion drives both trackers, so their process noise is common; they are often initialized from a shared prior or a common reference track; and any error in the shared sensor-registration or timing enters both. The strength of that coupling is the cross-covariance $\mathbf{P}_{12} = E[\tilde{\mathbf{x}}_1 \tilde{\mathbf{x}}_2^T]$, and for correlated tracks it is not zero. The naive fusion rule, the information-form sum $\mathbf{P}_{\text{naive}} = (\mathbf{P}_1^{-1} + \mathbf{P}_2^{-1})^{-1}$, is derived assuming the two estimates are independent, that is, $\mathbf{P}_{12} = 0$. When they are in fact correlated, that rule is over-confident: it produces a fused covariance smaller than the true error of

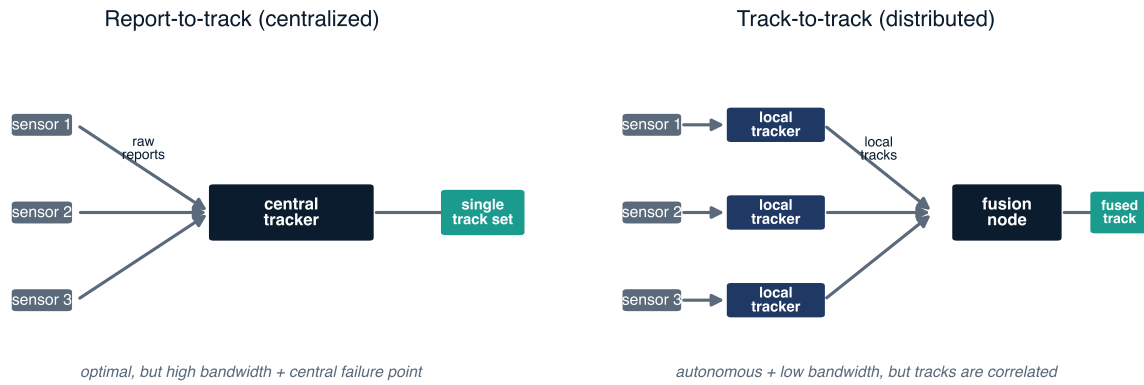


Figure 10.2: The two fusion architectures as data-flow diagrams. Left, report-to-track (centralized): every sensor ships raw reports to one central tracker that produces a single track set: statistically optimal, but bandwidth-hungry with a central point of failure. Right, track-to-track (distributed): each sensor runs a local tracker and ships finished tracks to a fusion node: autonomous and low-bandwidth, but the local tracks are correlated and cannot be fused as if independent.

the fused estimate. Worse, it is blind to the correlation, it returns the same covariance whatever the true $\mathbf{P}_{12}$ is, so it cannot even detect the problem. A fused track that reports a smaller error than it has is exactly what over-weights that track in any downstream reasoning, and the optimism compounds every fusion cycle.

10.6 Covariance intersection

There are two remedies. If the cross-covariance can be estimated, the Bar-Shalom/Campo formula gives the exact fused covariance accounting for $\mathbf{P}_{12}$, $\mathbf{P} = \mathbf{P}_1 - (\mathbf{P}_1 - \mathbf{P}_{12})(\mathbf{P}_1 + \mathbf{P}_2 - \mathbf{P}_{12} - \mathbf{P}_{12}^\top)^{-1}(\mathbf{P}_1 - \mathbf{P}_{12})^\top$, and with $\mathbf{P}_{12} = 0$ it collapses back to the naive result. In practice $\mathbf{P}_{12}$ is rarely known in a distributed system, which is where covariance intersection comes in. CI fuses two estimates as a convex combination in information space, $\mathbf{P}_{CI}^{-1} = \omega \mathbf{P}_1^{-1} + (1 - \omega) \mathbf{P}_2^{-1}$, with the weight $\omega \in [0, 1]$ chosen to minimize the size of the fused covariance (its trace or determinant). Its defining property is that it is consistent for any unknown correlation between the two estimates: whatever the true $\mathbf{P}_{12}$, the CI covariance is guaranteed not to be optimistic. The cost is that CI is conservative, it returns a larger covariance than the truly optimal one, trading tightness for a guarantee of never lying. In a distributed system where the correlation is unknown, that is exactly the trade you want. The next slide measures both effects.

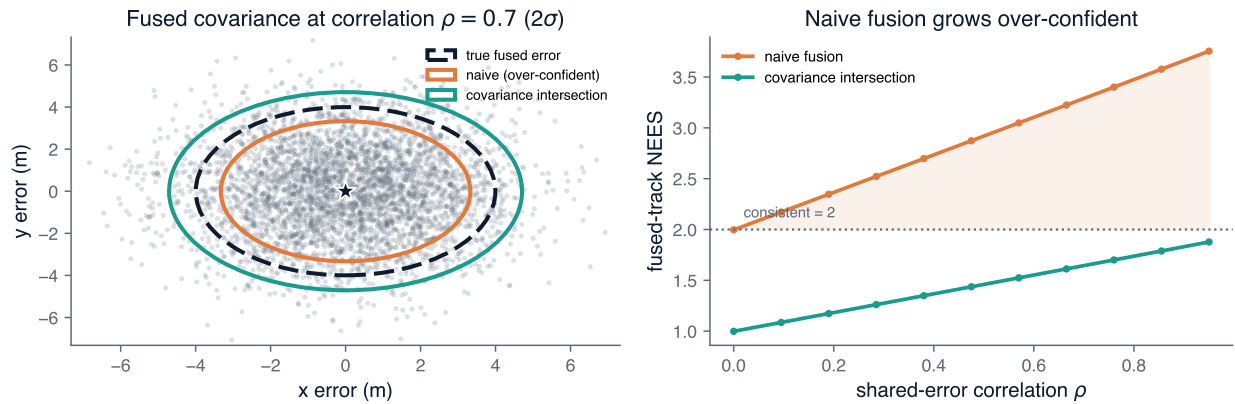


Figure 10.3: Two correlated local tracks of one target, fused. Left: at a shared-error correlation of 0.7, the naive independence-assuming fusion (its 2-sigma ellipse) is too small to cover the actual fused-error cloud, while covariance intersection returns a conservative ellipse that contains both the cloud and the true fused-error ellipse. Right: the fused-track consistency (NEES, which should equal 2) versus the correlation. Naive fusion climbs well above 2 as the shared error grows (over-confident, the shaded gap), while covariance intersection stays at or below 2 (consistent, never optimistic) for every correlation.

Naive fusion vs covariance intersection, measured

10.7 Choosing an architecture

The architecture choice is a trade among a few axes. On bandwidth and latency, the distributed approach wins, because tracks are far cheaper to communicate than raw reports. On statistical optimality, the centralized approach wins, because it uses the full evidence with no double-counting and no correlation to manage. On autonomy and robustness, the distributed approach wins again, since there is no single point of failure and sensors keep working independently. The distributed approach's price is that it must handle the correlated-error problem explicitly, with covariance intersection or an estimated cross-covariance, whereas the centralized approach sidesteps it by never forming local tracks in the first place. Because each pure architecture sacrifices something, real systems are usually hybrid: hierarchical or federated designs that track locally for autonomy and bandwidth, then fuse with a correlation-aware rule. The next module makes this concrete for the two most common cases, fusing multiple radars and fusing multiple angle-only sensors.

The JDL model and covariance intersection: the origins

Two short attributions. The JDL data fusion model was formulated by Franklin E. White and the U.S. Joint Directors of Laboratories Data Fusion Subpanel around 1991, and revised several times since; its contribution was to give a fragmented field a shared vocabulary and the level structure that lets engineers place their work and interfaces cleanly. Covariance intersection was introduced by

Simon Julier and Jeffrey Uhlmann in 1997 as a fusion rule that stays consistent when the correlation between two estimates is unknown, which is the normal situation in distributed tracking; it is now the standard robust choice for track-to-track fusion. Julier and Uhlmann appear here for the second time in the course, after the unscented transform of Module 3, where their full biography is given; covariance intersection is arguably their most directly useful contribution for a fusion audience.

Key takeaways

Multi-sensor fusion has a framework and a fork. The framework is the JDL model, which places target tracking at Level 1, object refinement, feeding the situation and impact levels above and closing back through sensor management at Level 4. The fork is the architecture: report-to-track fusion pools every sensor's raw detections into one central tracker, statistically optimal but bandwidth-hungry and dependent on a single central node; track-to-track fusion lets each sensor form its own tracks and combines them, autonomous and light on bandwidth. The distributed choice carries the module's central lesson: the local tracks share error sources, so their cross-covariance is nonzero, and fusing them as if they were independent produces a track that is over-confident and blind to the correlation. Covariance intersection fixes this by fusing consistently for any unknown correlation, at the price of a conservative covariance, which is the right trade when the correlation cannot be estimated. Real systems blend the two into hybrid designs. The next module turns this framework into the concrete recipes for fusing multiple radars and multiple angle-only sensors.

Exercise 10: Correlated Track Fusion

Exercise 10: Correlated Track Fusion (25 min)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex10_fusion.py
```

The app fuses two local tracks of the same target that share a tunable fraction of their error. It draws the two input covariance ellipses, the naive (independence-assuming) fused ellipse, the covariance-intersection ellipse, and the true fused-error ellipse over a Monte-Carlo cloud, and reports the NEES (normalized estimation-error squared, which should equal 2 for a consistent two-dimensional estimate) for the naive and the CI result. You control the shared-error correlation ρ , the two input covariances, and the CI weight ω (optimized automatically, with a manual override).

Work through these steps and note what you see:

1. Start at $\rho = 0$ (independent tracks). Both NEES values are near or below 2, and the naive ellipse covers the cloud. Note that naive fusion is the tightest here, it is correct when the tracks really are independent.
2. Raise ρ . Watch the true fused-error cloud grow past the naive ellipse while the naive

ellipse stays put, and the naive NEES climb above 2. The CI ellipse always contains the cloud and its NEES stays at or below 2.

3. Vary the CI weight ω manually. What does the optimizer's choice do to the shape and size of the CI ellipse, and why does the optimum minimize its area?
4. Make the two input covariances very unequal. Which track does CI lean toward, and how does the naive fusion's over-confidence compare?

Deliverable: the ellipse comparison with the naive and CI NEES values, and a short paragraph explaining what makes the two local tracks correlated and why covariance intersection is the safe default in a distributed fusion system.

Multi-Target Tracking Training Workshop

Module 11 · Fusion of Multiple Radars & Multiple Angle-Only Sensors

2026-09-17

Table of contents

11 Fusion of Multiple Radars & Angle-Only Sensors	3
Learning objectives	3
Where Module 10 left us	3
11.1 Fusing multiple radars	4
11.2 Colocated vs non-colocated radars	4
Two radars fuse: the geometry	4
11.3 Fusing multiple angle-only sensors: triangulation	4
Bearings crossing to a fix	5
11.4 Angle-only correlation: ghost targets	5
Four crossings, two of them ghosts	5
11.5 Geometry and accuracy: GDOP	5
The GDOP field	7
Triangulation and cross-fixing: the origin	7
Key takeaways	8
Exercise 11: Radar Fusion and Triangulation	8

Module 11: Fusion of Multiple Radars & Angle-Only Sensors

Student Guide

Learning objectives

Module 10 gave the framework and the architectures for fusion: the JDL model, the choice between report-to-track and track-to-track, and the correlated-error caution that led to covariance intersection. This module applies all of that to the two sensor networks a real system actually fields, taken as a parallel pair: first many radars, then many angle-only sensors. Radars each produce a full position estimate, so fusing them is combining consistent estimates and the reward is geometric, two radars viewing from different angles collapse the cross-range error. Angle-only sensors give bearing but no range, so a single one cannot fix a target at all; here fusion is not an improvement but the mechanism, two or more bearings are triangulated into a position. The angle-only case adds a wrinkle the radar case does not have: with several targets, bearings must be associated across sensors before triangulating, or the crossings produce ghost targets. Both cases end at the same place, geometry (GDOP) sets the achievable accuracy.

Where Module 10 left us

Module 10 was the general theory of combining sensors: where tracking sits on the JDL model, the report-to-track and track-to-track architectures, and the fact that local tracks are correlated, which is why covariance intersection is the safe default. This module makes that concrete for the two homogeneous cases. Fusing several radars is the M10 problem directly, with a geometric bonus that makes it worthwhile. Fusing several angle-only sensors is different in kind: because a bearing carries no range, no single sensor produces a track at all, and combining sensors is the only way to recover position. We take the two in parallel and finish with the geometry, GDOP, that governs both.

11.1 Fusing multiple radars

Each radar measures range and angle and, after the debiased conversion of Module 9, produces a full position estimate with a consistent covariance. A radar network is therefore exactly the fusion problem of Module 10: pool every radar's detections into one central tracker (report-to-track), or let each radar form its own track and fuse the tracks (track-to-track). The Module 10 caution carries over without change: two radars tracking the same target, often from a shared prior, produce correlated tracks, so they are combined with covariance intersection or an estimated cross-covariance rather than the naive independence-assuming rule that would be over-confident. What is new here, and what makes a radar network worth building, is geometric, and it is the subject of the next section.

11.2 Colocated vs non-colocated radars

The value of a radar network is geometric. A single radar measures range precisely but angle only to σ_θ , so its position-error ellipse is thin along the line of sight and fat across it, the cross-range error being $r\sigma_\theta$, which at long range dominates the total. Two radars viewing the same target from different bearings have their fat cross-range axes pointing in different directions, so fusing them crosses each radar's precise range axis against the other's, and the fused ellipse collapses to something far smaller than either input. The benefit depends on the angular separation of the looks, not merely on having two radars: widely separated looks give a large gain, near-collinear looks give almost none. Non-colocated radars, the usual case, must first be brought into a common frame (Module 9) and time-aligned before fusing; colocated radars share a frame and clock but, sitting at nearly the same viewing angle, gain little of the geometric diversity that makes fusion pay.

Two radars fuse: the geometry

11.3 Fusing multiple angle-only sensors: triangulation

Angle-only sensors, passive electro-optical or infrared trackers, ESM receivers, or a radar in a passive mode, measure bearing but not range. A single such sensor therefore defines only a line of position through the target, not a point: it cannot fix the target on its own. This flips the role of fusion. For radars, fusion improved an estimate each sensor could already make; for angle-only sensors, fusion is the only way to get a position at all. Two or more sensors on a baseline cross their bearing lines to a triangulated, or cross-fixed, position. Because each bearing is uncertain by σ_θ , each line of position is really a thin wedge, and the fix is the region where the wedges overlap, an error ellipse whose size and orientation are set by the sensors' angular noise and, crucially, by the geometry of the crossing.

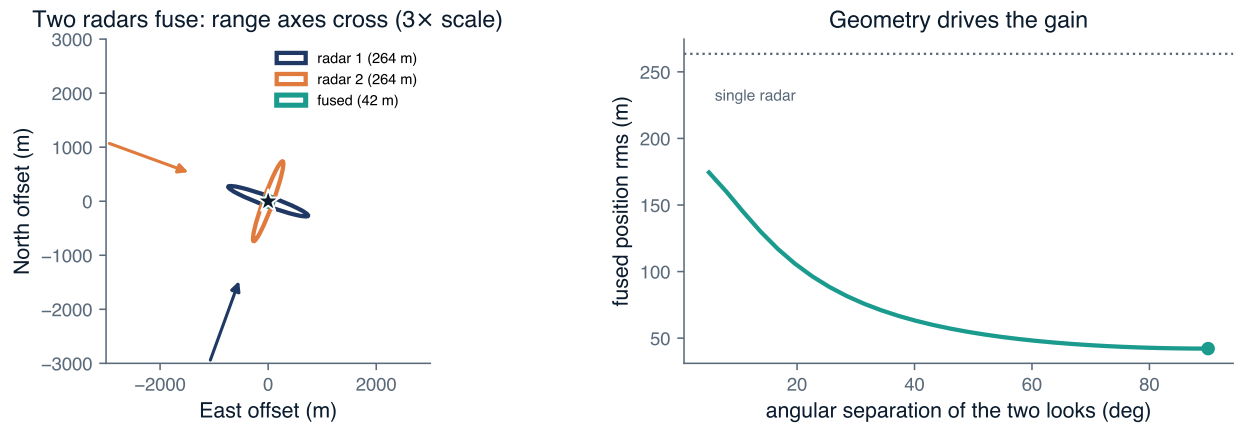


Figure 11.1: Left: two radars view one target from different bearings. Each radar's converted-measurement covariance is thin along its line of sight and fat across it (the cross-range term dominates at range); fusing crosses the two precise range axes, so the fused ellipse (shown at 3x scale for visibility) collapses to a fraction of either input. Right: the fused position rms versus the angular separation of the two looks. The gain is geometric, near the single-radar accuracy when the looks are nearly collinear and collapsing as the separation approaches ninety degrees.

Bearings crossing to a fix

11.4 Angle-only correlation: ghost targets

The triangulation picture is clean for one target, but with several targets it hides a trap. Each sensor now reports several bearings, one per target, and it does not know which of the other sensor's bearings belongs to the same target. Every bearing from sensor A can be crossed with every bearing from sensor B, and for two sensors and two targets the four crossings are two real target positions plus two ghost targets, phantom fixes formed by crossing the bearing of one target from sensor A with the bearing of a different target from sensor B. The ghosts are not noise; they are exact intersections of the wrong pair of lines, indistinguishable from real fixes on the two-sensor geometry alone. Resolving them is the data-association problem of Modules 6 through 8 applied across sensors rather than across time: a third sensor viewing from another angle has bearings that pass through the real targets but miss the ghosts, and target motion over time is consistent for the real tracks but not for the phantoms. Either cue rejects the ghosts.

Four crossings, two of them ghosts

11.5 Geometry and accuracy: GDOP

Both cases, radar and angle-only, are governed by geometric dilution of precision. The achievable accuracy of a fused fix depends not only on each sensor's noise but on the geometry: the cut angle

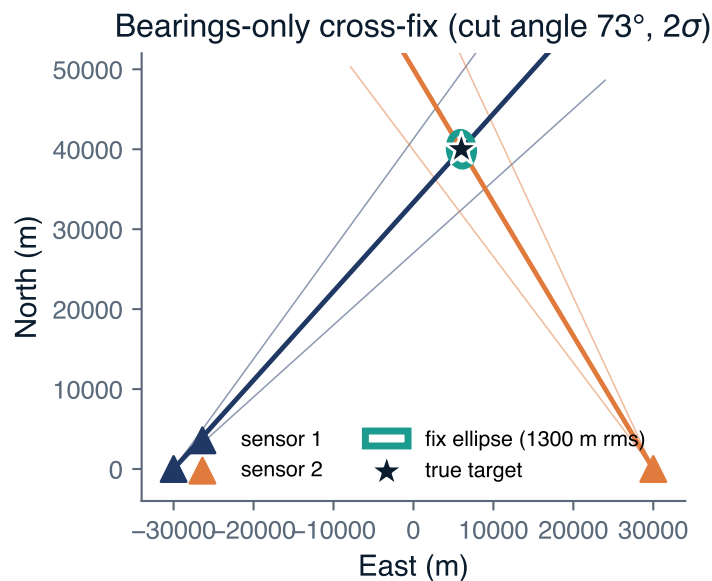


Figure 11.2: Two angle-only sensors on a baseline cross their bearing lines to a target. Each bearing carries an angular-uncertainty wedge (shown exaggerated), and the overlap of the two wedges is the fix error ellipse (drawn at two sigma). With no range from either sensor, this crossing is the only source of a position, and its accuracy is set by where and at what angle the lines cross.

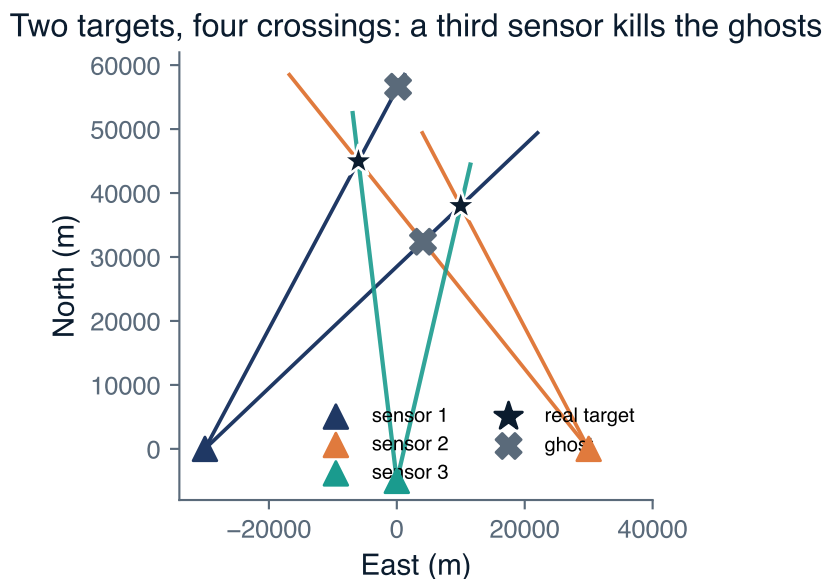


Figure 11.3: Two angle-only sensors and two real targets produce four bearing crossings: two are the true target positions (stars) and two are ghost targets (crosses), intersections of bearings that belong to different targets. A third sensor, viewing from another angle, has bearings that pass through the two real targets and miss both ghosts, resolving the ambiguity, the cross-sensor version of the data association of Modules 6 to 8.

at which the lines of position cross, the length of the baseline relative to the target range, and the target's aspect. A cut angle near ninety degrees gives a compact, well-conditioned fix; as the angle shrinks toward zero, the fix stretches along the nearly parallel lines and the error grows without bound, a large along-line uncertainty that no amount of per-sensor precision removes. GDOP is the single scalar that summarizes this geometric factor, and mapping it over a region is the tool for siting sensors: it shows where a given sensor layout produces good fixes and where the geometry goes bad. The lesson is the same in both halves of the module, geometry, not sensor count or sensor precision alone, sets the fused accuracy.

The GDOP field

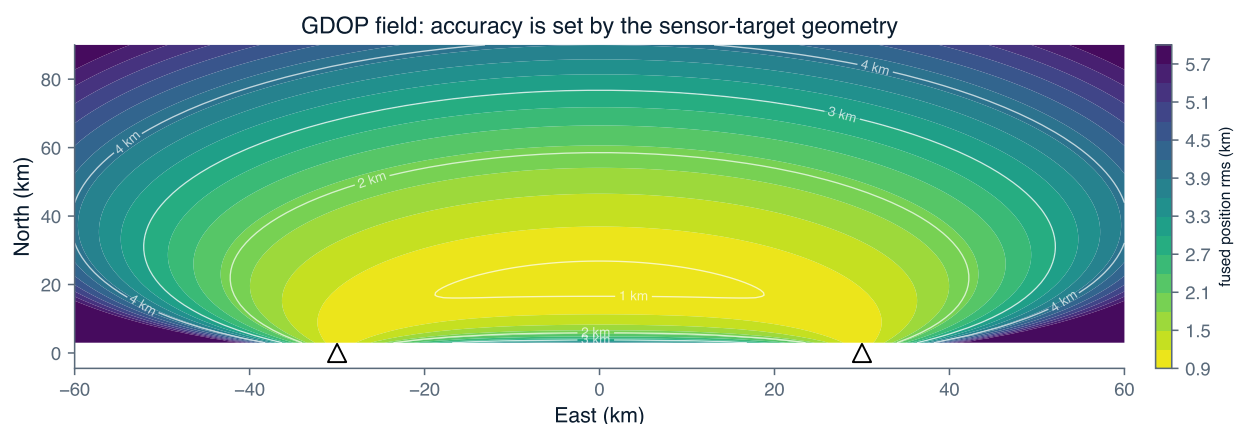


Figure 11.4: A map of fused position accuracy (a GDOP field) over target location for a fixed pair of angle-only sensors (triangles). A well of good accuracy sits above the baseline, where the two bearings cross near a right angle; the error grows rapidly toward the baseline and out to the sides, where the geometry becomes collinear. The field, not the individual sensor noise, is what a designer optimizes when siting sensors.

Triangulation and cross-fixing: the origin

Fixing a position by crossing bearing lines is far older than target tracking. Cross-bearing fixing is classical surveying and navigation practice, taking bearings to two or more known landmarks (or, in reverse, from two known stations to an unknown point) and crossing the resulting lines of position. Multi-sensor angle-only tracking inherits that geometry directly; what it adds is the multi-target complication, the ghost-association problem, and the systematic use of GDOP to reason about and site the sensors. It is a method with a long lineage rather than the work of a single named figure, so we note the origin briefly here rather than with a full biography.

Key takeaways

The two homogeneous fusion cases rhyme. Fusing many radars is the Module 10 problem with a geometric reward: each radar already gives a full position, and combining radars that view from different bearings crosses their precise range axes to collapse the cross-range error, a gain that depends on the angular separation of the looks and evaporates when they are near-collinear. Fusing many angle-only sensors is different in kind: a bearing carries no range, so no single sensor fixes a target, and triangulation of two or more bearings is the mechanism that produces a position at all, with an accuracy set by the crossing geometry. The angle-only case adds the ghost-target problem, phantom crossings of bearings from different targets, resolved by associating bearings across sensors with a third sensor or with motion over time. Both cases are ruled by GDOP: the cut angle and the baseline, not sensor count or per-sensor precision alone, determine the fused accuracy, which is why sensor siting is a geometry problem. Module 12 adds the harder, heterogeneous case, fusing radar with angle-only sensors, where sensor registration and alignment bias bite hardest.

Exercise 11: Radar Fusion and Triangulation

Exercise 11: Radar Fusion and Triangulation (two parts, 15 min each)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex11_fusion.py
```

Use the selector at the top to switch between the two parts.

Part A - multi-radar fusion and geometry (15 min). The app fuses two radar tracks of one target and draws each radar's ellipse and the fused ellipse, reporting the fused position rms.

1. Start with the two radars viewing from about ninety degrees apart. Note how much smaller the fused ellipse is than either radar alone.
2. Bring the two look angles close together (near-collinear). What happens to the fused rms, and why does the gain almost disappear?
3. Switch fusion from naive to covariance intersection. Which is tighter, and which is the safe choice for two correlated radar tracks (Module 10)?

Part B - triangulation and ghost targets (15 min). The app places two angle-only sensors and one or more targets, triangulates the crossings, and shows the fix ellipse and any ghosts.

1. With one target, shrink the baseline (or move the target off to the side) and watch the fix ellipse stretch as the cut angle drops. Relate this to GDOP.
2. Add a second target. Identify the two ghost crossings among the four intersections.
3. Turn on the third sensor. Which crossings does it confirm, and how does that reject the ghosts?

Deliverables: (a) the radar-fusion covariance plot at a good and a poor geometry, with one line on why look-angle separation matters; (b) the triangulation/ghost plot with and without the third sensor, identifying which crossings are ghosts and how the third sensor rejects them.

Multi-Target Tracking Training Workshop

Module 12 · Radar/Angle-Only Fusion & Sensor Alignment

2026-09-17

Table of contents

12 Radar/Angle-Only Fusion & Sensor Alignment	3
Learning objectives	3
Where Module 11 left us	3
12.1 Fusing radar with angle-only: complementary observability	4
Radar range meets passive bearing	4
12.2 Colocated vs non-colocated radar/angle-only	5
12.3 The sensor alignment (registration) problem	5
12.4 Bias sources and their impact	5
One bias, two tracks	6
12.5 Bias estimation and correction	6
Correcting the bias	7
12.6 Residual bias and downstream association	7
Multi-sensor registration: the origin	7
Key takeaways	8
Exercise 12: Sensor Registration	8

Module 12: Radar/Angle-Only Fusion & Sensor Alignment

Student Guide

Learning objectives

Module 11 fused like with like: many radars, then many angle-only sensors. This module fuses radar WITH angle-only, a genuinely complementary pair, the radar supplies the range the passive sensor lacks and the passive sensor supplies precise bearing to pin the radar's coarse cross-range. Together they fix a target neither can fix well alone. It then confronts the problem that most often wrecks fusion across dissimilar, widely separated sensors: sensor alignment, or registration. A small systematic error in where a sensor is, which way it points, or when it measures, shifts its track, and once that shift exceeds the association gate the fusion sees one target as two. Registration lives here because the heterogeneous, wide-baseline case is exactly where bias errors do the most damage. The module ends by tying the residual, whatever registration leaves behind, back to the gating and association of Days 2 and 3.

Where Module 11 left us

Module 11 handled the homogeneous fusion cases and closed on the geometric lesson that GDOP, not sensor count, sets the fused accuracy. This module takes the harder, heterogeneous case, a radar and an angle-only sensor, whose measurement types are different but complementary. Getting the gain from that pair requires a non-colocated baseline, and that baseline is precisely where a small alignment error between the two sensors does the most harm. So the second half of the module is sensor registration: what misalignment does to a fused picture, where the biases come from, how to estimate and remove them from common targets, and what the unavoidable residual does downstream.

12.1 Fusing radar with angle-only: complementary observability

A radar and an angle-only sensor are a complementary pair. The radar measures range with high precision but resolves bearing only to its beamwidth, so its position estimate is thin along the line of sight and long across it, a cross-range smear that grows with range. An angle-only sensor is the opposite: it measures bearing precisely but carries no range at all, so on its own it gives only a line of position. Fuse the two, taking the radar's range and a non-colocated angle-only sensor's bearing, and each supplies exactly what the other lacks: the passive bearing pins down the cross-range direction the radar measures poorly, collapsing the fused fix to a fraction of the radar-only ellipse. The association step is the cross-sensor idea of Module 11, now applied between two different measurement types that first have to be expressed in the common frame of Module 9.

Radar range meets passive bearing

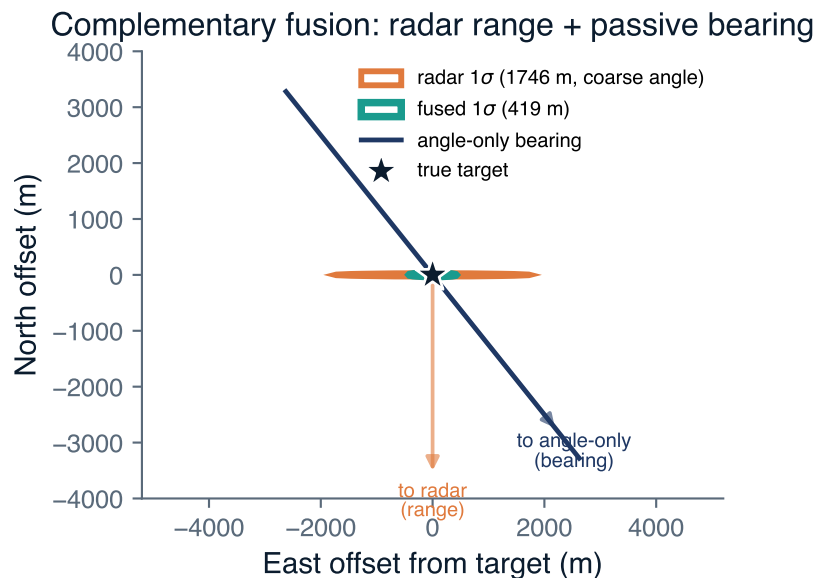


Figure 12.1: A range-dominant radar (precise range, coarse beam) has a long cross-range smear, drawn here as a flat one-sigma ellipse about the target. A non-colocated angle-only sensor's precise bearing (the line) crosses that smear and pins the cross-range the radar cannot resolve, collapsing the fused one-sigma ellipse to a small fix. The two sensors sit far off-panel; their directions are marked. Complementary observability: range from the radar, cross-range from the passive bearing.

12.2 Colocated vs non-colocated radar/angle-only

Where the angle-only sensor sits relative to the radar decides whether the fusion helps. Colocate them and the passive sensor looks along nearly the same line as the radar, so its bearing merely duplicates the radar's own bearing and does nothing for the cross-range the radar is poor at. The gain comes from a non-colocated pair viewing the target from different angles, so the passive bearing constrains a direction the radar cannot, the same geometric-diversity lesson as Module 11. That separation is not free: the two sensors must be brought into a common frame and time-aligned (Module 9), and the wider the baseline, the more a small alignment error between them displaces the fused picture. So the very geometry that makes heterogeneous fusion worthwhile is the geometry in which registration errors do the most damage, which is the subject of the rest of the module.

12.3 The sensor alignment (registration) problem

Registration is the task of expressing every sensor's measurements in one common frame with their systematic errors removed, so that "the same target" lands at the same place for all sensors. When registration is wrong, a single true target is reported at slightly different positions by different sensors, and the fusion mishandles it in one of two ways. If the offset is small, it mis-associates, blurring nearby real targets together or inflating the apparent measurement error. If the offset is large enough, the fusion fails to associate one target's two reports at all and splits the target into two separate fused tracks, along with the ghosting that follows. The crucial property is that misregistration is a systematic error, not random noise: it does not average out over time, and it biases every fused track in the same direction, which is why it is so corrosive and why it must be estimated and removed rather than merely tolerated.

12.4 Bias sources and their impact

The registration error budget has four kinds of bias. A location bias is an error in the sensor's own surveyed or GPS position; it shifts every report by the same vector. An orientation bias is a boresight or azimuth offset in which way the sensor points; it rotates every bearing. A timing bias is an unsynchronized clock, which on a moving target looks like a position error proportional to the target's speed and the time offset. And measurement biases are fixed range or angle offsets in the sensor itself. Each maps to a shift of the sensor's track. The azimuth or orientation bias is the classic and most instructive case, because its effect grows with range: an azimuth error $\Delta\theta$ displaces the track cross-range by $R\Delta\theta$, so a fraction of a degree becomes hundreds of metres at tens of kilometres. The next figure shows that shift splitting a target in two.

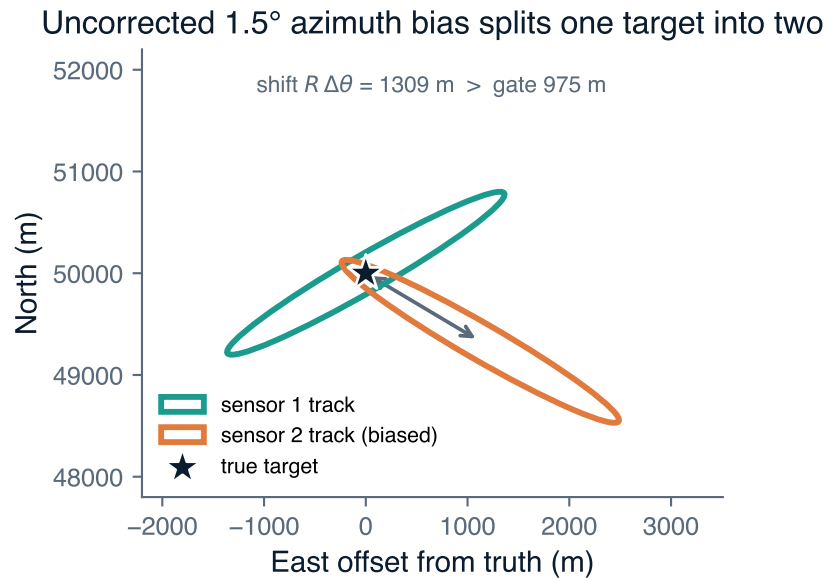


Figure 12.2: One true target seen by two radars, one carrying an uncorrected 1.5 degree azimuth bias. The biased sensor's track is displaced cross-range by R times the bias, here about 1300 metres at 50 km, which exceeds the cross-sensor association gate (about 975 metres). The fusion therefore fails to associate the two reports and splits the single target into two separate tracks. Because the bias is systematic, the displacement is the same on every scan and does not average away.

One bias, two tracks

12.5 Bias estimation and correction

The bias is estimated from targets that two or more sensors see in common. Because both sensors observe the same true target, the systematic difference between their reports is an estimate of the relative bias between them, uncontaminated by the target's unknown position. Averaging that difference over many common observations, a least-squares or grid-based fit, recovers the bias, and its estimate error falls as $\sigma_\theta/\sqrt{N}$ in the number of common observations: a handful of common targets already pins a fraction-of-a-degree bias to within a tenth of a degree. Applying the estimated correction re-registers the sensors, the displaced track moves back onto the truth, and the two fused tracks that the bias had split re-merge into one. The next figure shows the re-merge and the estimate converging on the injected bias as more common observations are used.

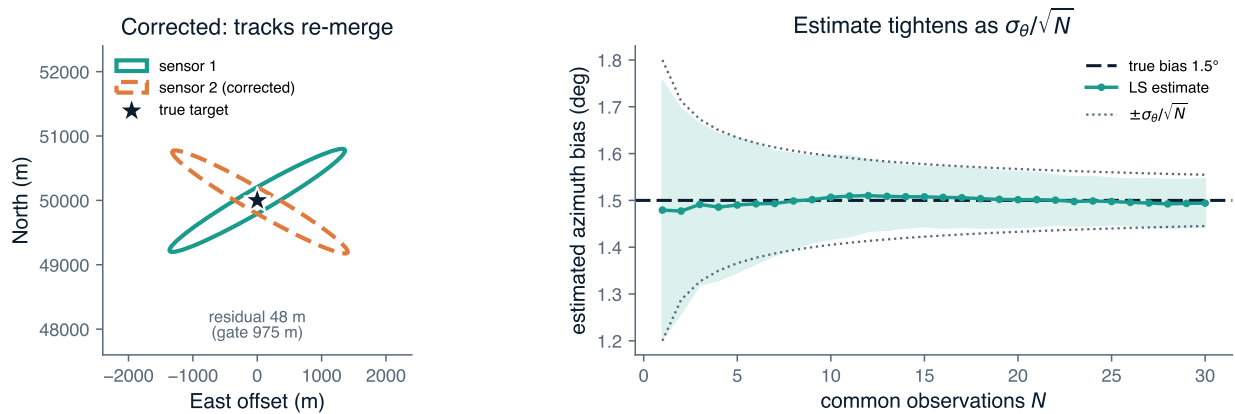


Figure 12.3: Left: after the least-squares bias estimate is removed, the previously displaced sensor-2 track re-merges onto the true target, the residual (here about 50 metres) well inside the association gate. Right: the least-squares azimuth-bias estimate converging to the injected 1.5 degree truth as the number of common observations grows, with the estimate spread shrinking as $\sigma_\theta/\sqrt{N}$. A few common targets suffice to pin the bias tightly enough to re-register the sensors.

Correcting the bias

12.6 Residual bias and downstream association

Registration is never perfect. Whatever the method, a residual bias of order $\sigma_\theta/\sqrt{N}$ remains, and it feeds straight back into the gating and data association of Modules 6 through 8. If the residual shift is still larger than the association gate, the fusion continues to split or mis-associate tracks, just less often than before correction. If the residual is within the gate, it no longer splits tracks but it inflates the effective measurement error, so under clutter the mis-association rate rises because more spurious measurements fall inside the enlarged gate. The practical rule follows directly: register until the residual track shift is well inside the association gate, and then budget the remaining residual as part of the measurement covariance so that the gating and association machinery accounts for it honestly rather than being surprised by it. Good fusion is therefore inseparable from good registration; the association algorithms of the earlier modules assume a registered picture, and this module is what earns that assumption.

Multi-sensor registration: the origin

Estimating inter-sensor biases from targets seen in common is a body of technique that grew out of netting multiple radars together for air defense, where tracks from separate sites had to be merged into one air picture. The standard treatments are least-squares and grid-based (often called gridlock) registration, both of which use common targets as references to solve for the relative location, orientation, timing, and measurement biases among the sensors. It is a well-developed

engineering method rather than the work of a single named figure, so we note its origin briefly here in the same spirit as the earlier short notes, without a full biography.

Key takeaways

Fusing a radar with an angle-only sensor is the complementary case: the radar contributes precise range, the passive sensor precise bearing, and together they fix a target neither resolves well alone, provided they view from a non-colocated baseline. That same baseline is where alignment errors bite hardest. Misregistration, a systematic bias in a sensor's location, orientation, timing, or measurements, displaces its track by an amount that for the classic azimuth bias grows as $R \Delta\theta$, and once the displacement exceeds the association gate the fusion splits one target into two. Because the bias is systematic it does not average out; it must be estimated, from targets seen in common by two or more sensors, and removed. A least-squares fit recovers the bias with an error that falls as $\sigma_\theta/\sqrt{N}$, and the correction re-merges the split tracks. What registration leaves behind, the residual, then feeds the gating and association of Days 2 and 3, so it must be driven well inside the gate and budgeted into the measurement covariance. Registered fusion is the precondition the earlier association modules quietly assumed. Module 13 closes the course with attribute fusion and the performance metrics that measure how well all of this actually works.

Exercise 12: Sensor Registration

Exercise 12: Sensor Registration (30 min)

No coding required. Start the companion app from the project root:

```
conda activate radar
streamlit run exercises/ex12_registration.py
```

The app tracks one target with two sensors, injects a known azimuth bias into sensor 2, and shows the fused picture. You control the injected bias, the number of common observations N used to estimate it, the target range, and the sensor noise; the app draws the biased (split) and corrected (merged) fused tracks and reports the estimated bias against the truth and the residual shift against the association gate.

Work through these steps and note what you see:

1. Start with a small bias (a few tenths of a degree). The two tracks stay gated together as one target, but the fused estimate is degraded. Note the shift against the gate.
2. Raise the bias until the track splits into two. At what shift, relative to the gate, does the split happen? Relate it to $R \Delta\theta$.
3. Estimate the bias with a small N , then a large one. Watch the estimate tighten toward the truth as $\sigma_\theta/\sqrt{N}$, and the corrected tracks re-merge.
4. Increase the range at a fixed bias. Why does the same angular bias split the tracks more

easily at longer range?

Deliverable: the before/after fused-track plot (split, then re-merged) and the estimated-bias-versus-truth value, with a short paragraph on why registration must drive the residual shift well inside the association gate.

Multi-Target Tracking Training Workshop

Module 13 · Attribute Fusion & Performance Metrics

2026-09-17

Table of contents

13 Attribute Fusion & Performance Metrics	3
Learning objectives	3
Where Module 12 left us	3
13.1 Fusing target type and attribute data	4
13.2 Bayesian attribute fusion	4
Bayesian class fusion in action	4
13.3 Dempster-Shafer evidential fusion	4
Dempster-Shafer vs Bayes	5
13.4 Fuzzy-logic attribute fusion	6
13.5 Performance metrics	6
Three views of one run	6
13.6 Composite metrics and evaluation practice	6
OSPA over time	7
The people behind it	7
A note on fuzzy logic: Lotfi Zadeh	7
Key takeaways	8
Course wrap-up	8
Exercise 13: Attribute Fusion and Scoring	9

Module 13: Attribute Fusion & Performance Metrics

Student Guide

Learning objectives

The course closes on the two things every earlier module left implicit: what a target IS, not just where it is, and whether any of the machinery actually worked. The first half adds the non-kinematic dimension, fusing target type and attribute declarations, and lays out the three standard frameworks for it: Bayesian class probability, Dempster-Shafer evidential belief, and fuzzy-logic membership. The second half is the evaluation framework, the metrics that score a tracker's accuracy, consistency, and continuity, and the composite OSPA distance that folds localization and cardinality errors into one number. It is the capstone: attribute fusion complements every kinematic method in the course, and the metrics are the instruments that told us, all along, that the debiased converter was consistent where the naive one was not, that PDA survived clutter the hard pick could not, and that covariance intersection stayed honest where naive fusion lied.

Where Module 12 left us

Module 12 finished the kinematic fusion story by registering the sensors so their position data combine without bias. Two things remain. First, sensors report more than position: they classify. Fusing those noisy type and attribute declarations is a different problem from continuous state estimation, over a discrete hypothesis set, and it has its own frameworks. Second, across twelve modules we have made many comparative claims, this filter beats that one, this association method survives clutter, this fusion rule stays consistent, and we have been implicitly relying on metrics to justify them. The capstone makes both explicit: how to fuse attributes, and how to measure tracking performance so that "X beats Y" is a measurement, not an assertion.

13.1 Fusing target type and attribute data

Every sensor reports more than a position. A radar estimates size and motion class, an ESM receiver identifies an emitter type, an electro-optical sensor classifies a shape: all of these are attributes, and fusing them answers “what is this target” alongside the “where” that the rest of the course addressed. Attribute fusion complements kinematic tracking in a concrete way: it disambiguates association. Two tracks that are momentarily identical in position and velocity, and so hard to keep apart kinematically, are trivially separable if one is classified as a transport and the other as a fighter. But attribute declarations are noisy and discrete: a classifier says “fighter” when the truth is “transport” some fraction of the time. Fusing them is therefore a problem of combining discrete evidence over a hypothesis set, which is a different mathematical setting from the continuous state estimation of Days 1 and 2, and it has three standard frameworks.

13.2 Bayesian attribute fusion

The natural first framework is Bayesian. Represent belief as a probability distribution over the class set, start from a prior, and update it with each declaration using the classifier’s confusion matrix as the likelihood, $P(\text{declaration} \mid \text{class})$. Because independent declarations multiply, the sequential posterior is $P(c \mid z_{1:k}) \propto P(z_k \mid c) P(c \mid z_{1:k-1})$, a running product normalized to sum to one. The posterior for the true class climbs toward certainty at a rate set entirely by how discriminating the classifier is: a sharp classifier (a confusion matrix close to the identity) converges in a handful of declarations, a poor one needs many more or never resolves. The framework is clean, well-understood, and gives a single probability per class, which is also its limitation, as the next framework highlights.

Bayesian class fusion in action

13.3 Dempster-Shafer evidential fusion

Dempster-Shafer evidence theory generalizes the Bayesian picture by assigning mass to subsets of the class set, not only to single classes. Mass on the whole frame represents ignorance explicitly, so a sensor early in a track can say “I do not yet know” rather than being forced to spread a uniform prior. Belief in a hypothesis is the mass committed to its subsets (a lower bound), plausibility is the mass not committed against it (an upper bound), and the pair forms an interval $[\text{Bel}, \text{Pl}]$ that brackets the Bayesian point. Evidence is combined by Dempster’s rule, which intersects the focal sets of the two mass functions and renormalizes by the total conflicting mass. That renormalization is the framework’s famous weakness: when two sources strongly conflict, the normalization by a near-total conflict produces a counterintuitive result. In Zadeh’s example, two sensors that each nearly rule out a class, committing 0.99 to different classes and only 0.01 to a third, combine to certify that third class with mass one. The interval representation is valuable; the combination rule

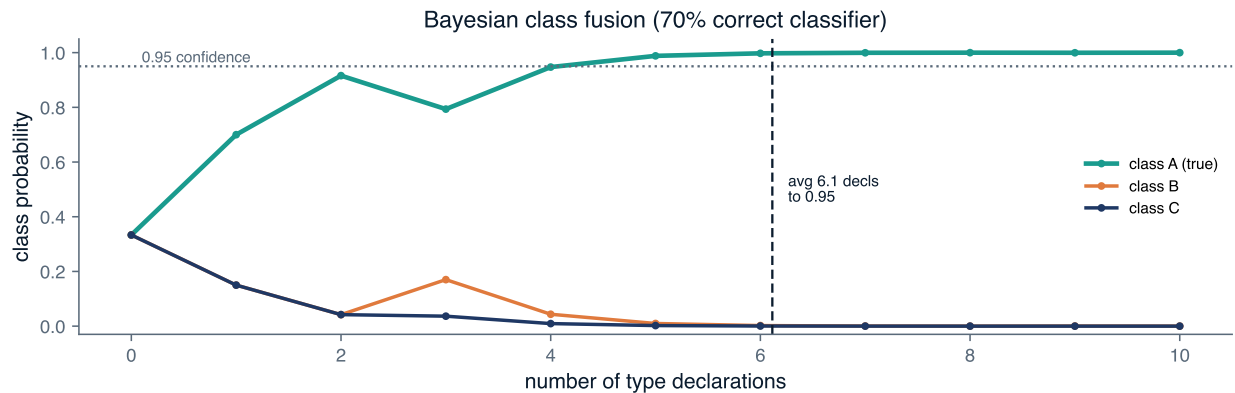


Figure 13.1: Sequential Bayesian class probabilities as noisy type declarations arrive from a 70-percent-correct classifier. The true class rises to near-certainty while the alternatives decay; on average about six declarations are needed to pass 0.95 confidence. A sharper classifier converges faster, a weaker one slower, at a rate set by the confusion matrix.

must be used with care.

Dempster-Shafer vs Bayes

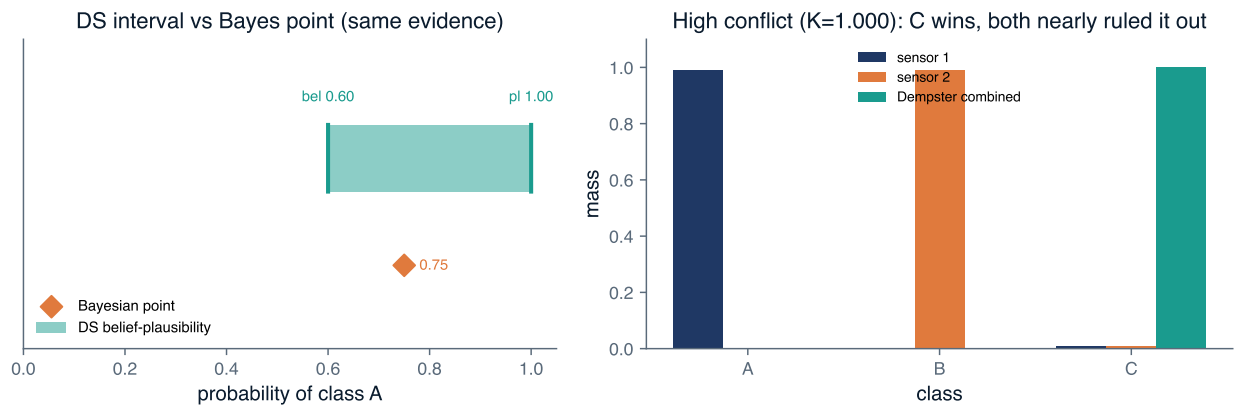


Figure 13.2: Left: a Dempster-Shafer belief-plausibility interval for a class (here $[0.60, 1.00]$, the mass committed to it and the mass not committed against it) versus a single Bayesian point probability on the same evidence, Dempster-Shafer represents the remaining ignorance explicitly. Right: the high-conflict (Zadeh) case, where two sensors commit 0.99 to different classes and only 0.01 each to class C, yet Dempster's rule, normalizing by the near-total conflict, certifies class C, the one both sensors nearly ruled out.

13.4 Fuzzy-logic attribute fusion

The third framework, fuzzy logic, suits attributes that are graded and linguistic rather than crisply probabilistic: “large”, “fast”, “high-flying”. A membership function maps a measured feature, say speed or radar cross-section, to a degree of membership between zero and one in each fuzzy class, and does so with soft, overlapping boundaries rather than a hard threshold. A rule base then combines memberships across several features, using fuzzy operators (minimum and maximum, or product and sum) to fire rules like “if fast and small then fighter”, and defuzzifies the result into a class decision. Fuzzy fusion trades the probabilistic rigor of Bayes and the evidential structure of Dempster-Shafer for a natural, transparent way to encode expert rules and handle vague, overlapping categories. It is common where the attributes are inherently linguistic and where a domain expert’s rules are the best available knowledge. The three frameworks are complementary tools; the right one depends on whether the evidence is probabilistic, set-valued, or linguistic.

13.5 Performance metrics

Measuring a tracker takes several metrics, because a tracker can fail in several distinct ways. Accuracy is the obvious one: position and velocity root-mean-square error against truth, and, from Day 1, the normalized estimation-error squared (NEES) that checks consistency, whether the filter’s stated covariance actually matches its error, so it is neither over-confident nor pessimistic. Continuity metrics catch the failures that accuracy misses: a track swap (the label jumps from one target to another), fragmentation (one true target broken into several short-lived tracks), and purity (the fraction of a track’s life spent on its dominant truth target). Timeliness is the track-initiation time, how many scans it takes to confirm a new target, the M7 trade in metric form. Each names a failure mode a real system produces, and none subsumes the others, which is exactly why a single composite metric is needed to compare methods fairly.

Three views of one run

13.6 Composite metrics and evaluation practice

Two honest failures motivate a composite metric. RMSE, computed only over matched tracks, ignores targets that were missed or invented; a pure count of tracks ignores where those tracks are. The Optimal Sub-Pattern Assignment (OSPA) distance folds both into one number: it solves an optimal assignment between the truth set and the estimate set (with a cutoff c that bounds any single miss and an order p), and its value decomposes into a localization term, the assigned distances, and a cardinality term, a penalty for every unmatched track. A missed or spurious track raises OSPA even when the matched tracks are perfectly placed, which is exactly what RMSE cannot see. Evaluation practice then closes the loop: score over many Monte-Carlo runs rather than one, average the metrics, and compare competing methods on identical scenarios and identical

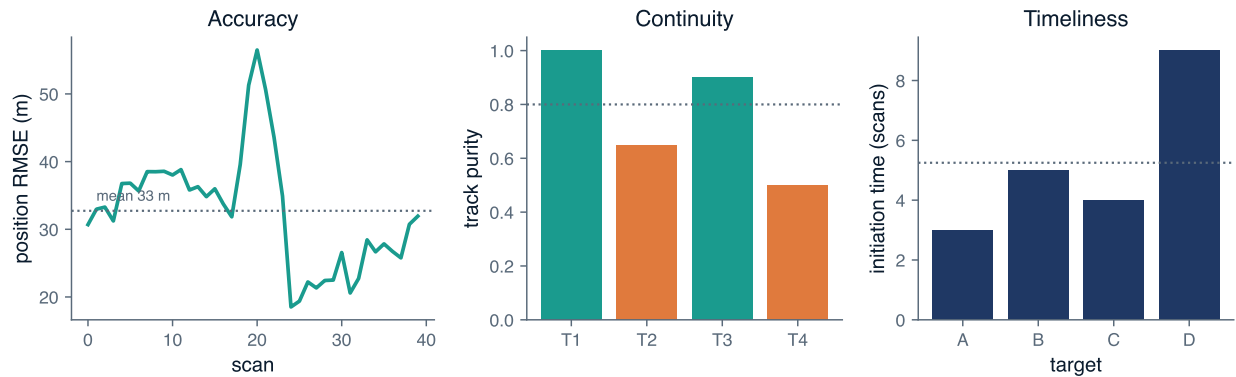
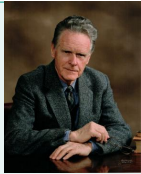


Figure 13.3: One multi-target run scored three ways. Left, accuracy: position RMSE over time, with the spike where a maneuver stresses the filter. Middle, continuity: per-track purity, the fraction of each track's life on its dominant truth (two tracks fall below the 0.8 line, flagging swaps or contamination). Right, timeliness: the initiation time per target, how many scans to confirm each. Three distinct failure modes, three metrics; none captures the others.

noise draws so the comparison is fair, and always report consistency (NEES) alongside error so a method is not rewarded for being confidently wrong.

OSPA over time

The people behind it



THE PERSON BEHIND IT

Arthur P. Dempster & Glenn Shafer (evidence theory, 1967/1976)

Arthur Dempster, at Harvard, developed the theory of upper and lower probabilities in the 1960s, generalizing Bayesian probability to intervals that represent partial ignorance. Glenn Shafer reframed it in his 1976 book *A Mathematical Theory of Evidence*, introducing belief functions and the belief-plausibility interval. The Dempster-Shafer framework is the standard alternative to Bayesian fusion where evidence is set-valued or ignorance must be modeled explicitly, and its combination rule, with the well-known high-conflict pitfall, is a fixture of sensor fusion.



A note on fuzzy logic: Lotfi Zadeh

Fuzzy attribute fusion rests on the work of Lotfi A. Zadeh, of the University of California, Berkeley, who introduced fuzzy sets in 1965 and developed fuzzy logic in the years after. Zadeh's idea was to let membership in a set be a matter of degree, a value in the interval from zero to one, rather than a crisp yes or no, which is exactly what the membership functions of section 13.4 encode. Zadeh is also the source of the high-conflict counterexample that exposes the pitfall in Dempster's

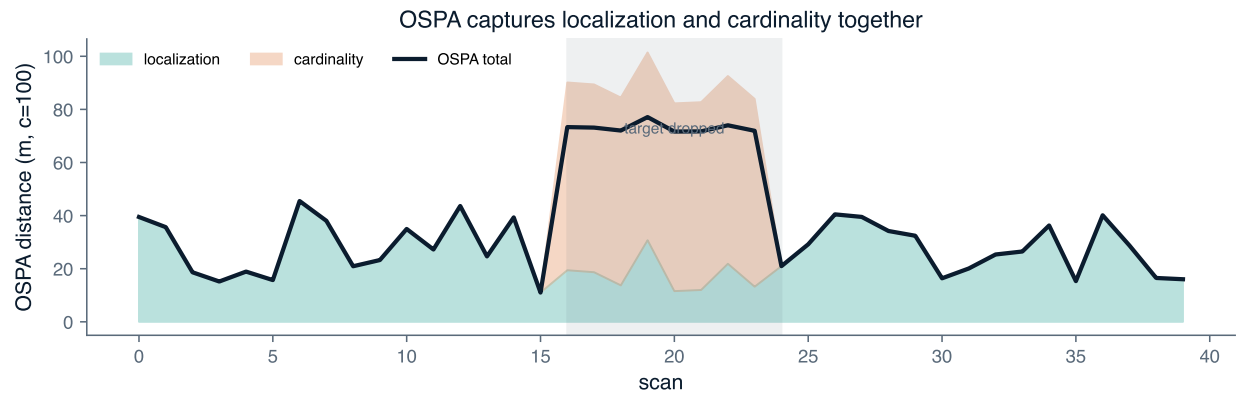


Figure 13.4: OSPA distance over a two-target run (cutoff 100 m). While both targets are tracked, OSPA follows the localization error alone (the lower band). When one target is dropped for a stretch, OSPA jumps, the cardinality term (the upper band) accounting for the unmatched target, and settles again when the track is re-acquired. RMSE over the matched track alone would show none of this; OSPA captures localization and cardinality together.

combination rule, so he appears twice in this module, as the origin of the fuzzy framework and as its sharpest critic of the evidential one.

Key takeaways

The capstone adds the two dimensions the kinematic modules left implicit. Attribute fusion answers what a target is, complementing where it is and disambiguating association, and it has three standard frameworks: Bayesian class probability, which converges to a point posterior at a rate set by the classifier; Dempster-Shafer evidence theory, which brackets belief and plausibility and models ignorance explicitly but must be used carefully under high conflict; and fuzzy logic, which handles graded, linguistic attributes with membership functions and expert rules. Performance evaluation supplies the instruments: RMSE and NEES for accuracy and consistency, purity, swaps, and fragmentation for continuity, initiation time for timeliness, and OSPA as the composite that captures localization and cardinality together, all averaged over Monte-Carlo runs on identical scenarios. These are the tools that make “X beats Y” a measurement.

Course wrap-up

It is worth closing by naming what these metrics were measuring all along. Every comparative claim in the workshop was, underneath, one of these numbers. When Module 9 said the debiased converted measurement was consistent and the naive one was not, that was a NEES of two versus a NEES climbing into the hundreds. When Module 8 said probabilistic data association survived clutter that broke the nearest-neighbor pick, that was a track-loss rate roughly halved and an

RMSE more than halved. When Module 10 said covariance intersection stayed honest where naive fusion did not, that was a fused-track NEES held at or below two against one that grew past three. The three days trace a single arc: estimate a single target's state (Day 1), associate measurements to tracks among many targets (Day 2), and fuse across sensors and measure the result (Day 3). The estimator, the association logic, and the fusion rule were the methods; the metrics of this final module were how we knew, at each step, that the better method was actually better. That is the discipline the workshop set out to build.

Exercise 13: Attribute Fusion and Scoring

Exercise 13: Attribute Fusion and Scoring (two parts, 15 min each)

No coding required. Start the companion app from the project root:

```
conda activate radar
```

```
streamlit run exercises/ex13_attribute_metrics.py
```

Use the selector at the top to switch between the two parts.

Part A - Bayes vs Dempster-Shafer (15 min). The app feeds a stream of noisy type declarations into a Bayesian class-probability update and a Dempster-Shafer combination.

1. With a decent classifier, watch the Bayesian posterior for the true class converge. How many declarations to pass 0.95?
2. Lower the classifier accuracy. Does convergence slow, and can it pick the wrong class for a while?
3. Turn on the high-conflict toggle (the Zadeh case). What does Dempster's rule do, and why is the result counterintuitive?

Part B - scoring a run (15 min). The app scores a supplied multi-target run with RMSE, track purity, and OSPA over time.

1. Read off the RMSE and the per-track purity. Which track has a swap?
2. Inject a dropped or spurious track. Watch OSPA jump while RMSE (over the matched track) barely moves. Why?
3. Change the OSPA cutoff c . How does it trade localization sensitivity against the cardinality penalty?

Deliverables: (a) the class-confidence comparison including the high-conflict case, with a line on when each framework is appropriate; (b) the metrics report (RMSE, purity, OSPA) with a short paragraph on why a composite metric is needed and what a fair Monte-Carlo evaluation requires.

Multi-Target Tracking Training Workshop

Formula Reference – every equation, every term

2026-09-23

Formulas by module

1	Module 1 – Introduction & Tracking Fundamentals	3
2	Module 2 – Linear Filtering: Kalman & Other Linear Filters	4
3	Module 3 – Nonlinear Filters: EKF, Unscented & Particle	6
4	Module 4 – Angle-Only Tracking	8
5	Module 5 – Maneuvering Targets: Adaptive & Multiple-Model	9
6	Module 6 – Single-Target Correlation & Association	10
7	Module 7 – Track Management & Range-Rate (Doppler)	11
8	Module 8 – Multi-Target Association: NN, Assignment, PDA & MHT	12
9	Module 9 – Coordinate Conversions & Sensor Motion Compensation	13
10	Module 10 – Multi-Sensor Fusion: JDL Model & Architectures	14
11	Module 11 – Fusion of Multiple Radars & Angle-Only Sensors	15
12	Module 12 – Radar/Angle-Only Fusion & Sensor Alignment	16
13	Module 13 – Attribute Fusion & Performance Metrics	17

Module 1 – Introduction & Tracking Fundamentals

State-space model [eq-statespace]

$$\mathbf{x}_k = \mathbf{F} \mathbf{x}_{k-1} + \mathbf{w}_{k-1}, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k$$

$\mathbf{x}_k$ target state at step k (position m, velocity m/s); $\mathbf{F}$ state-transition matrix (dimensionless); $\mathbf{w}_{k-1}$ process noise; $\mathbf{z}_k$ measurement; $h(\cdot)$ measurement function; $\mathbf{v}_k$ measurement noise.

Process and measurement noise [eq-processnoise]

$$\mathbf{w}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}), \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$$

$\mathbf{Q}$ process-noise covariance (units of state²); $\mathbf{R}$ measurement-noise covariance (units of measurement²).

Radar measurement (range, azimuth, elevation) [eq-radarmeas]

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2}, \quad \theta = \text{atan2}(p_y, p_x), \quad \phi = \arcsin\left(\frac{p_z}{r}\right)$$

r range (m); θ azimuth (rad); ϕ elevation (rad); p_x, p_y, p_z target position components in the sensor frame (m).

Range rate (radial velocity) [eq-rangerate]

$$\dot{r} = \frac{\mathbf{p} \cdot \mathbf{v}}{\|\mathbf{p}\|}$$

$\dot{r}$ range rate (m/s); $\mathbf{p}$ relative position vector (m); $\mathbf{v}$ relative velocity vector (m/s).

Bearing (angle-only) [eq-bearing]

$$\beta = \text{atan2}(p_y, p_x)$$

β bearing angle to target (rad).

Module 2 – Linear Filtering: Kalman & Other Linear Filters

Ordinary and weighted least squares [eq-ols, eq-wls]

$$\hat{\mathbf{x}} = (\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{z}, \quad \hat{\mathbf{x}} = (\mathbf{H}^\top \mathbf{R}^{-1} \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{z}$$

$\mathbf{H}$ measurement (design) matrix; $\mathbf{z}$ stacked measurements; $\mathbf{R}$ measurement-noise covariance (weighting).

Recursive least squares (gain and update) [eq-rls-gain, eq-rls-update]

$$\mathbf{K}_k = \mathbf{P}_{k-1} \mathbf{h}_k (\mathbf{h}_k^\top \mathbf{P}_{k-1} \mathbf{h}_k + r_k)^{-1}$$
$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_{k-1} + \mathbf{K}_k (z_k - \mathbf{h}_k^\top \hat{\mathbf{x}}_{k-1}), \quad \mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{h}_k^\top) \mathbf{P}_{k-1}$$

$\mathbf{K}_k$ gain; $\mathbf{P}_k$ estimate-error covariance; $\mathbf{h}_k$ measurement row vector; r_k scalar measurement variance; z_k scalar measurement.

Kalman filter: predict [eq-predict-mean, eq-predict-cov]

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F} \hat{\mathbf{x}}_{k-1|k-1}, \quad \mathbf{P}_{k|k-1} = \mathbf{F} \mathbf{P}_{k-1|k-1} \mathbf{F}^\top + \mathbf{Q}$$

Kalman filter: innovation and covariance [eq-innov-cov]

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_{k|k-1}, \quad \mathbf{S}_k = \mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R}$$

$\mathbf{y}_k$ innovation (measurement residual); $\mathbf{S}_k$ innovation covariance.

Kalman filter: gain and update [eq-update-state, eq-update-cov]

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k$$
$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_{k|k-1}$$

Joseph-form covariance update (numerically stable) [eq-joseph]

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K} \mathbf{H}) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K} \mathbf{H})^\top + \mathbf{K} \mathbf{R} \mathbf{K}^\top$$

Valid for any gain $\mathbf{K}$; guarantees a symmetric positive-semidefinite $\mathbf{P}_{k|k}$.

Alpha-beta filter (steady-state constant-gain) [eq-alphabeta]

$$\hat{p} = p^- + \alpha (z - p^-), \quad \hat{v} = v^- + \frac{\beta}{T} (z - p^-)$$

p^-, v^- predicted position (m) and velocity (m/s); α, β fixed gains (dimensionless); T update interval

(s); z position measurement (m).

Module 3 – Nonlinear Filters: EKF, Unscented & Particle

Nonlinear state-space model [eq-nl-model]

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}) + \mathbf{w}_{k-1}, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k$$

$f(\cdot)$ nonlinear dynamics; $h(\cdot)$ nonlinear measurement function.

EKF linearization (measurement Jacobian) [eq-ekf-lin]

$$h(\mathbf{x}) \approx h(\hat{\mathbf{x}}_{k|k-1}) + \mathbf{H}_k (\mathbf{x} - \hat{\mathbf{x}}_{k|k-1}), \quad \mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}}$$

$\mathbf{H}_k$ Jacobian of h evaluated at the prediction; $\mathbf{F}_k = \partial f / \partial \mathbf{x}$ is the dynamics Jacobian.

EKF predict / innovation / update [eq-ekf-predict, eq-ekf-innov, eq-ekf-update]

$$\begin{aligned} \hat{\mathbf{x}}_{k|k-1} &= f(\hat{\mathbf{x}}_{k-1|k-1}), & \mathbf{P}_{k|k-1} &= \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^\top + \mathbf{Q} \\ \mathbf{y}_k &= \mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1}), & \mathbf{S}_k &= \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R} \\ \mathbf{K}_k &= \mathbf{P}_{k|k-1} \mathbf{H}_k^\top \mathbf{S}_k^{-1}, & \hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k, & \mathbf{P}_{k|k} &= (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \end{aligned}$$

Range-bearing measurement Jacobian [eq-rb-jacobian]

$$\mathbf{H} = \begin{bmatrix} p_x/r & 0 & p_y/r & 0 \\ -p_y/r^2 & 0 & p_x/r^2 & 0 \end{bmatrix}$$

State ordered $[p_x, \dot{p}_x, p_y, \dot{p}_y]^\top$; $r = \sqrt{p_x^2 + p_y^2}$ (m). Top row is $\partial r / \partial \mathbf{x}$, bottom row $\partial \theta / \partial \mathbf{x}$.

Unscented transform: sigma points and weights [eq-sigma]

$$\mathcal{X}_0 = \bar{\mathbf{x}}, \quad \mathcal{X}_i = \bar{\mathbf{x}} + \left(\sqrt{(n + \kappa) \mathbf{P}} \right)_i, \quad \mathcal{X}_{i+n} = \bar{\mathbf{x}} - \left(\sqrt{(n + \kappa) \mathbf{P}} \right)_i$$

$$W_0 = \frac{\kappa}{n + \kappa}, \quad W_i = \frac{1}{2(n + \kappa)}$$

n state dimension; κ scaling parameter; $(\sqrt{\cdot})_i$ the i -th matrix-square-root column.

Unscented transform: recovered moments [eq-ut-moments]

$$\bar{\mathbf{y}} = \sum_{i=0}^{2n} W_i \mathcal{Y}_i, \quad \mathbf{P}_{yy} = \sum_{i=0}^{2n} W_i (\mathcal{Y}_i - \bar{\mathbf{y}})(\mathcal{Y}_i - \bar{\mathbf{y}})^\top$$

$\mathcal{Y}_i = g(\mathcal{X}_i)$ transformed sigma points; $\bar{\mathbf{y}}$ transformed mean; $\mathbf{P}_{yy}$ transformed covariance.

UKF update [eq-ukf-update]

$$\mathbf{K}_k = \mathbf{P}_{xz} \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k, \quad \mathbf{P}_{k|k} = \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top$$

$\mathbf{P}_{xz}$ state-measurement cross-covariance from the sigma points.

Particle filter: posterior, weights, effective sample size [eq-pf-posterior, eq-sis, eq-neff]

$$p(\mathbf{x}_k \mid \mathbf{z}_{1:k}) \approx \sum_{i=1}^N w_k^i \delta(\mathbf{x}_k - \mathbf{x}_k^i)$$

$$w_k^i \propto w_{k-1}^i \frac{p(\mathbf{z}_k \mid \mathbf{x}_k^i) p(\mathbf{x}_k^i \mid \mathbf{x}_{k-1}^i)}{q(\mathbf{x}_k^i \mid \mathbf{x}_{k-1}^i, \mathbf{z}_k)}, \quad N_{\text{eff}} = \frac{1}{\sum_{i=1}^N (w_k^i)^2}$$

$\mathbf{x}_k^i$ the i -th particle; w_k^i its normalized weight; N particle count; $q(\cdot)$ proposal (importance) density; N_{eff} effective sample size (resample when it drops below a threshold).

Module 4 – Angle-Only Tracking

Bearing measurement from a moving observer [eq-ao-meas]

$$\beta_k = \text{atan2}(T_y - O_{y,k}, T_x - O_{x,k}) + v_k, \quad v_k \sim \mathcal{N}(0, \sigma_\beta^2)$$

$\mathbf{T} = (T_x, T_y)$ target position (m); $\mathbf{O}_k = (O_{x,k}, O_{y,k})$ observer position at step k (m); σ_β bearing noise std (rad).

Angle-only measurement Jacobian [eq-ao-jac]

$$\mathbf{H}_k = \begin{bmatrix} -\frac{\Delta y}{r^2} & 0 & \frac{\Delta x}{r^2} & 0 \end{bmatrix}, \quad \Delta = \mathbf{T} - \mathbf{O}_k, \quad r = \|\Delta\|$$

$\Delta = (\Delta x, \Delta y)$ observer-to-target offset (m); r range (m).

Pseudo-linear estimator constraint [eq-ple]

$$\sin \beta_k (T_x - O_{x,k}) - \cos \beta_k (T_y - O_{y,k}) = 0$$

Linear in the target coordinates; stacking over k yields a least-squares position fix (biased when noise correlates with the regressor).

Modified polar coordinates (state vector) [eq-mpc-state]

$$\mathbf{y} = [\beta, \dot{\beta}, \dot{r}/r, 1/r]^\top$$

Decouples the observable states $(\beta, \dot{\beta}, \dot{r}/r)$ from the single unobservable state $(1/r)$ until an observer maneuver makes range observable.

Module 5 – Maneuvering Targets: Adaptive & Multiple-Model

Innovation and innovation covariance [eq-innovation]

$$\nu_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}, \quad \mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R}$$

Normalized innovation squared (maneuver detector) [eq-nis]

$$\epsilon_k = \nu_k^\top \mathbf{S}_k^{-1} \nu_k$$

$\epsilon_k \sim \chi_{n_z}^2$ under the no-maneuver hypothesis; a sustained exceedance of the chi-square threshold flags a maneuver.

IMM mixing probabilities [eq-imm-mix]

$$\mu_{i|j} = \frac{\pi_{ij} \mu_i}{\bar{c}_j}, \quad \bar{c}_j = \sum_i \pi_{ij} \mu_i$$

π_{ij} mode-transition probability $i \rightarrow j$; μ_i prior mode probability; $\bar{c}_j$ predicted (normalizing) mode probability.

IMM mode likelihood and update [eq-imm-like, eq-imm-modeupdate]

$$\Lambda_j = \mathcal{N}(\nu_j; \mathbf{0}, \mathbf{S}_j), \quad \mu_j = \frac{\bar{c}_j \Lambda_j}{\sum_\ell \bar{c}_\ell \Lambda_\ell}$$

Λ_j mode- j filter likelihood; μ_j updated mode probability; the fused estimate is the mode-probability-weighted mixture of the mode estimates.

Module 6 – Single-Target Correlation & Association

Chi-square gate [eq-gate-chi2, eq-gate-gamma]

$$d^2 = \nu^\top \mathbf{S}_k^{-1} \nu \sim \chi_{n_z}^2, \quad \Pr(d^2 \leq \gamma) = P_G \implies \gamma = F_{\chi_{n_z}^2}^{-1}(P_G)$$

d^2 squared statistical distance; n_z measurement dimension; P_G gate probability; γ gate threshold; F^{-1} inverse chi-square CDF.

Mahalanobis distance and measurement likelihood [eq-maha, eq-likelihood]

$$d^2(\mathbf{z}) = \nu^\top \mathbf{S}_k^{-1} \nu, \quad \nu = \mathbf{z} - \hat{\mathbf{z}}_{k|k-1}, \quad \Lambda(\mathbf{z}) \propto \exp\left(-\frac{1}{2}d^2(\mathbf{z})\right)$$

$\hat{\mathbf{z}}_{k|k-1}$ predicted measurement; Λ Gaussian association likelihood.

Nearest-neighbor rule and assignment cost [eq-nn, eq-cost]

$$\mathbf{z}^* = \arg \min_{\mathbf{z}^j \in \text{gate}} (\mathbf{z}^j - \hat{\mathbf{z}}_{k|k-1})^\top \mathbf{S}_k^{-1} (\mathbf{z}^j - \hat{\mathbf{z}}_{k|k-1}), \quad c_j = \frac{1}{2}d^2(\mathbf{z}^j) + \frac{1}{2} \ln |2\pi \mathbf{S}_k|$$

$\mathbf{z}^*$ chosen measurement; c_j negative-log-likelihood assignment cost (the term $\frac{1}{2} \ln |2\pi \mathbf{S}_k|$ makes costs comparable across tracks with different $\mathbf{S}_k$).

Module 7 – Track Management & Range-Rate (Doppler)

M-of-N confirmation probability

$$P(\text{confirm}) = \sum_{k=M}^N \binom{N}{k} p^k (1-p)^{N-k}$$

M required hits within a window of N looks; p per-look detection probability.

Sequential probability ratio test (log-likelihood accumulation)

$$L_k = L_{k-1} + \begin{cases} \ln \frac{P_D}{P_{FA}} & \text{detection} \\ \ln \frac{1-P_D}{1-P_{FA}} & \text{miss} \end{cases}$$

L_k running log-likelihood ratio; P_D detection probability; P_{FA} false-alarm probability. Confirm when L_k crosses the upper threshold, delete when it crosses the lower.

Range rate and its measurement Jacobian

$$\dot{r} = \frac{x\dot{x} + y\dot{y}}{\sqrt{x^2 + y^2}}$$

$$\mathbf{H}_{\dot{r}} = \begin{bmatrix} \frac{\dot{x}r^2 - (x\dot{x} + y\dot{y})x}{r^3} & \frac{x}{r} & \frac{\dot{y}r^2 - (x\dot{x} + y\dot{y})y}{r^3} & \frac{y}{r} \end{bmatrix}$$

State ordered $[x, \dot{x}, y, \dot{y}]^\top$ (m, m/s); $r = \sqrt{x^2 + y^2}$ (m). Range rate is nonlinear in position, so it enters through the EKF.

Module 8 – Multi-Target Association: NN, Assignment, PDA & MHT

Assignment cost and the global assignment problem

$$c_{ij} = \frac{1}{2} \nu_{ij}^T \mathbf{S}_i^{-1} \nu_{ij} + \frac{1}{2} \ln |2\pi \mathbf{S}_i| - \ln P_D, \quad \min_{\{a_{ij}\}} \sum_{i,j} a_{ij} c_{ij}$$

c_{ij} cost of assigning measurement j to track i ; $a_{ij} \in \{0, 1\}$ one-to-one assignment indicator (solved by Munkres/Hungarian or auction).

Probabilistic Data Association (association weights)

$$\beta_i \propto P_D \mathcal{N}(\nu_i; \mathbf{0}, \mathbf{S}), \quad \beta_0 \propto \lambda (1 - P_D)$$

β_i posterior probability that measurement i is target-originated; β_0 probability that none is (all clutter); λ clutter spatial density (1/volume).

PDA combined innovation and update

$$\nu = \sum_i \beta_i \nu_i, \quad \hat{\mathbf{x}} += \mathbf{K} \nu$$

The state is updated with the probability-weighted average innovation (with an added spread-of-innovations covariance term).

JPDA marginal association probability

$$\beta_{ij} = \sum_{\text{events assigning } j \rightarrow i} P(\text{event})$$

Marginalizes the joint feasible-association events so shared measurements are split consistently among nearby tracks.

Module 9 – Coordinate Conversions & Sensor Motion Compensation

Polar (spherical) to local ENU

$$E = r \cos el \sin az, \quad N = r \cos el \cos az, \quad U = r \sin el$$

r range (m); az azimuth (rad); el elevation (rad); E, N, U east/north/up (m).

Local ENU to Earth-centered (ECEF)

$$\mathbf{p}_{\text{ECEF}} = \mathbf{p}_0 + \mathbf{R}(\text{lat}, \text{lon}) \mathbf{p}_{\text{ENU}}$$

$\mathbf{p}_0$ sensor origin in ECEF (m); $\mathbf{R}(\text{lat}, \text{lon})$ ENU-to-ECEF rotation matrix.

Conversion bias (why naive polar-to-Cartesian is biased)

$$E[r \cos \theta] = e^{-\sigma_\theta^2/2} r_t \cos \theta_t$$

The nonlinear conversion scales the true mean by $e^{-\sigma_\theta^2/2}$; σ_θ angle noise std (rad); r_t, θ_t true range and angle.

Debiased converted measurement

$$\mathbf{z}_u = e^{\sigma_\theta^2/2} \begin{bmatrix} r \cos \theta \\ r \sin \theta \end{bmatrix}$$

Multiplying by $e^{\sigma_\theta^2/2}$ removes the multiplicative bias so the converted measurement is unbiased for the converted-measurement Kalman filter.

Sensor motion compensation (moving platform)

$$\mathbf{p}_{\text{global}} = \mathbf{p}_{\text{plat}}(t) + \mathbf{R}(\text{att}(t)) \mathbf{p}_{\text{local}}$$

$\mathbf{p}_{\text{plat}}(t)$ platform position at measurement time (m); $\mathbf{R}(\text{att}(t))$ attitude (orientation) rotation at time t ; time-tagging each measurement is essential.

Module 10 – Multi-Sensor Fusion: JDL Model & Architectures

Naive (information-form) track fusion

$$\mathbf{P}_{\text{naive}} = (\mathbf{P}_1^{-1} + \mathbf{P}_2^{-1})^{-1}, \quad \hat{\mathbf{x}} = \mathbf{P}_{\text{naive}} (\mathbf{P}_1^{-1} \hat{\mathbf{x}}_1 + \mathbf{P}_2^{-1} \hat{\mathbf{x}}_2)$$

Optimal only when the two local estimates have independent errors; over-confident when they share process noise or a common prior.

Covariance intersection (robust to unknown correlation)

$$\mathbf{P}_{\text{CI}}^{-1} = \omega \mathbf{P}_1^{-1} + (1 - \omega) \mathbf{P}_2^{-1}, \quad \omega \in [0, 1]$$

ω chosen (often to minimize $\text{tr } \mathbf{P}_{\text{CI}}$ or $\det \mathbf{P}_{\text{CI}}$) yields a consistent fused covariance for any unknown cross-correlation.

Module 11 – Fusion of Multiple Radars & Angle-Only Sensors

Bearing lines and the least-squares triangulation fix

$$\sin \beta_i (x - O_{x,i}) - \cos \beta_i (y - O_{y,i}) = 0, \quad \hat{\mathbf{p}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{c}$$

Row i of $\mathbf{A}$ is $[\sin \beta_i, -\cos \beta_i]$ and $c_i = \sin \beta_i O_{x,i} - \cos \beta_i O_{y,i}$; β_i bearing from sensor i at position $\mathbf{O}_i$ (m); $\hat{\mathbf{p}}$ estimated target position (m). Two or more bearings intersect to a fix.

Cross-range error from a bearing error

$$\varepsilon_{\text{cross}} \approx r \sigma_\theta$$

A bearing std σ_θ (rad) at range r (m) maps to a cross-range position error $r\sigma_\theta$ (m); triangulation accuracy therefore degrades with range and improves with baseline.

Angle-only fix accuracy with N sensors

$$\sigma_{\text{fix}} \sim \frac{\sigma_\theta}{\sqrt{N}}$$

N independent bearing measurements; accuracy improves as $1/\sqrt{N}$ (before geometry, i.e. GDOP, is applied).

Geometric dilution of precision (GDOP)

$$\mathbf{P}_{\hat{\mathbf{p}}} = \sigma^2 (\mathbf{A}^\top \mathbf{A})^{-1}, \quad \text{GDOP} = \sqrt{\text{tr}[(\mathbf{A}^\top \mathbf{A})^{-1}]}$$

$\mathbf{A}$ measurement-geometry (Jacobian) matrix; σ per-measurement error; GDOP is the geometry factor multiplying measurement error into position error. Ghost (false-intersection) fixes are removed by cross-sensor association before triangulating.

Module 12 – Radar/Angle-Only Fusion & Sensor Alignment

Complementary observability (radar range + passive bearing)

$$\mathbf{z} = [r_{\text{radar}}, \beta_{\text{AO}}]^\top$$

Radar supplies range r (m), the passive sensor supplies bearing β (rad); together they fix a 2-D position that neither sensor resolves alone.

Registration (bias) model

$$\tilde{z}_s = z_s^{\text{true}} + b_s + v_s, \quad v_s \sim \mathcal{N}(0, \sigma_s^2)$$

$\tilde{z}_s$ reported measurement from sensor s ; b_s systematic bias (offset in range, azimuth, timing, or location); v_s random noise. Uncorrected b_s splits one target into multiple tracks.

Bias estimation from common targets (least squares)

$$\hat{\mathbf{b}} = (\mathbf{G}^\top \mathbf{G})^{-1} \mathbf{G}^\top \Delta \mathbf{z}$$

$\Delta \mathbf{z}$ measured inter-sensor differences on shared targets; $\mathbf{G}$ maps biases to those differences; $\hat{\mathbf{b}}$ estimated bias vector.

Bias-corrected measurement

$$\hat{z}_s^{\text{corr}} = \tilde{z}_s - \hat{b}_s$$

Subtracting the estimated bias re-registers the sensors so common targets overlay and downstream association succeeds.

Module 13 – Attribute Fusion & Performance Metrics

Bayesian class (attribute) update

$$P(c \mid z_{1:k}) \propto P(z_k \mid c) P(c \mid z_{1:k-1})$$

c candidate target class; $P(z_k \mid c)$ likelihood of the current declaration given class c (a confusion-matrix column); the posterior is recursive over declarations $z_{1:k}$.

Dempster-Shafer combination (evidential fusion)

$$m_{12}(A) = \frac{1}{1-K} \sum_{B \cap C = A} m_1(B) m_2(C), \quad K = \sum_{B \cap C = \emptyset} m_1(B) m_2(C)$$

$m(\cdot)$ mass (basic belief) assigned to a hypothesis set; K the conflict between sources; $1/(1-K)$ the normalization. Belief and plausibility bound the probability: $\text{Bel}(A) = \sum_{B \subseteq A} m(B)$, $\text{Pl}(A) = \sum_{B \cap A \neq \emptyset} m(B)$.

Fuzzy-logic combination and defuzzification

$$\mu_{A \wedge B}(x) = \min(\mu_A(x), \mu_B(x)), \quad \mu_{A \vee B}(x) = \max(\mu_A(x), \mu_B(x)), \quad x^* = \frac{\int x \mu(x) dx}{\int \mu(x) dx}$$

$\mu_A(x) \in [0, 1]$ membership degree; min/max implement fuzzy AND/OR; x^* is the centroid (defuzzified) output.

Tracking accuracy: RMSE and consistency (NEES)

$$\text{RMSE}_k = \sqrt{\frac{1}{M} \sum_{m=1}^M \|\hat{\mathbf{p}}_k^{(m)} - \mathbf{p}_k^{(m)}\|^2}, \quad \epsilon_k = (\mathbf{x}_k - \hat{\mathbf{x}}_k)^\top \mathbf{P}_k^{-1} (\mathbf{x}_k - \hat{\mathbf{x}}_k)$$

M Monte Carlo runs; $\hat{\mathbf{p}}$ estimated and $\mathbf{p}$ true position (m). A consistent filter has $E[\epsilon_k] \approx n$ (state dimension); the average NEES should fall within the chi-square bounds.

Track continuity metrics

$$\text{Purity} = \frac{\#\{\text{updates from the correct target}\}}{\#\{\text{all track updates}\}}, \quad \text{Completeness} = \frac{\#\{\text{time steps the target is tracked}\}}{\#\{\text{time steps the target exists}\}}$$

Track swaps and fragmentation are counted alongside these ratios; initiation time is the delay from target appearance to confirmed track.

Optimal subpattern assignment (OSPA) distance

$$\bar{d}_p^{(c)}(X, Y) = \left[\frac{1}{n} \left(\min_{\pi} \sum_{i=1}^m d^{(c)}(x_i, y_{\pi(i)})^p + c^p (n-m) \right) \right]^{1/p}$$

Sets X (truth, $|X| = n$) and Y (estimates, $|Y| = m$, $m \leq n$); $d^{(c)}(x, y) = \min(c, \|x - y\|)$ the cutoff distance; c cutoff (m); p order. OSPA jointly penalizes localization error and cardinality (missed/false-track) error in one number.